

Технологии разработки и тестирования программного обеспечения

- **Тестирование программного обеспечения** — процесс анализа программного средства и сопутствующей документации с целью выявления дефектов и повышения качества продукта.
- Хотя термина именно тестирование ПО не используется, есть просто тестирование, интеграция и прочее.
- На протяжении десятилетий развития разработки ПО к вопросам тестирования и обеспечения качества подходили очень и очень по-разному. Можно выделить несколько основных «эпох тестирования».

- **В 50–60-х годах** прошлого века процесс тестирования был предельно формализован, отделён от процесса непосредственной разработки ПО и «математизирован».
- Фактически тестирование представляло собой скорее отладку программ (debugging). Существовала концепция т.н. «исчерпывающего тестирования (exhaustive testing)» — проверки всех возможных путей выполнения кода со всеми возможными входными данными. Однако очень скоро было выяснено, что исчерпывающее тестирование невозможно, т.к. количество возможных путей и входных данных очень велико, а также при таком подходе сложно найти проблемы в документации.

- **В 70-х годах** фактически родились две фундаментальные идеи тестирования: тестирование сначала рассматривалось как процесс доказательства работоспособности программы в некоторых заданных условиях (positive testing), а затем — строго наоборот: как процесс доказательства неработоспособности программы в некоторых заданных условиях (negative testing). Это внутреннее противоречие не только не исчезло со временем, но и в наши дни многими авторами совершенно справедливо отмечается как две взаимодополняющие цели тестирования.
- Отметим, что *«процесс доказательства неработоспособности программы»* ценится чуть больше, т.к. не позволяет закрывать глаза на обнаруженные проблемы. Итак, ещё раз самое важное, что тестирование «приобрело» в 70-е годы:
- тестирование позволяет удостовериться, что программа соответствует требованиям;
- тестирование позволяет определить условия, при которых программа ведёт себя некорректно.

- **В 80-х годах** произошло ключевое изменение места тестирования в разработке ПО: вместо одной из финальных стадий создания проекта тестирование стало применяться на протяжении всего цикла разработки (software lifecycle) (
- также см. описание итерационной инкрементальной модели разработки ПО в главе «Модели разработки ПО»), что позволило в очень многих случаях не только быстро обнаруживать и устранять проблемы, но даже предсказывать и предотвращать их появление.
- В этот же период времени отмечено бурное развитие и формализация методологий тестирования и появление первых элементарных попыток автоматизировать тестирование.

- **В 90-х годах** произошёл переход от тестирования как такового к более все-объемлющему процессу, который называется «обеспечение качества (quality assurance)», охватывает весь цикл разработки ПО и затрагивает процессы планирования, проектирования, создания и выполнения тест-кейсов, поддержку имеющихся тест-кейсов и тестовых окружений.
- Тестирование вышло на качественно новый уровень, который естественным образом привёл к дальнейшему развитию методологий, появлению достаточно мощных инструментов управления процессом тестирования и инструментальных средств автоматизации тестирования, уже вполне похожих на своих нынешних потомков.

- **В нулевые годы** нынешнего века развитие тестирования продолжалось в контексте поиска всё новых и новых путей, методологий, техник и подходов к обеспечению качества. Серьёзное влияние на понимание тестирования оказало появление гибких методологий разработки и таких подходов, как «разработка под управлением тестированием (test-driven development, TDD)».
- Автоматизация тестирования уже воспринималась как обычная неотъемлемая часть большинства проектов, а также стали популярны идеи о том, что во главу процесса тестирования следует ставить не соответствие программы требованиям, а её способность предоставить конечному пользователю возможность эффективно решать свои задачи.

Процессы тестирования и разработки ПО

- **Модели разработки ПО**

- **Модель разработки ПО** (Software Development Model, SDM) — структура, систематизирующая различные виды проектной деятельности, их взаимодействие и последовательность в процессе разработки ПО. Выбор той или иной модели зависит от масштаба и сложности проекта, предметной области, доступных ресурсов и множества других факторов.
- Выбор модели разработки ПО серьёзно влияет на процесс тестирования, определяя выбор стратегии, расписание, необходимые ресурсы и т.д.
- Моделей разработки ПО много, но в общем случае классическими можно считать водопадную, v-образную, итерационную инкрементальную, спиральную и гибкую.

- **Водопадная модель** (waterfall model) сейчас представляет скорее исторический интерес, т.к. в современных проектах практически неприменима.
- Она предполагает однократное выполнение каждой из фаз проекта, которые, в свою очередь, строго следуют друг за другом (рисунок 2.1.а). Очень упрощённо можно сказать, что в рамках этой модели в любой момент времени команде «видна» лишь предыдущая и следующая фаза. В реальной же разработке ПО приходится «видеть весь проект целиком» и возвращаться к предыдущим фазам, чтобы исправить недоработки или что-то уточнить.



Рисунок 2.1.а — Водопадная модель разработки ПО

- К недостаткам водопадной модели принято относить тот факт, что участие пользователей ПО в ней либо не предусмотрено вообще, либо предусмотрено лишь косвенно на стадии однократного сбора требований. С точки зрения же тестирования эта модель плоха тем, что тестирование в явном виде появляется здесь лишь с середины развития проекта, достигая своего максимума в самом конце.
- Тем не менее водопадная модель часто интуитивно применяется при выполнении относительно простых задач, а её недостатки послужили прекрасным отправным пунктом для создания новых моделей. Также эта модель в несколько усовершенствованном виде используется на крупных проектах, в которых требования очень стабильны и могут быть хорошо сформулированы в начале проекта (аэрокосмическая область, медицинское ПО и т.д.)

- **V-образная модель** (V-model) является логическим развитием водопадной. Можно заметить (рисунок 2.1.b), что в общем случае как водопадная, так и v-образная модели жизненного цикла ПО могут содержать один и тот же набор стадий, но принципиальное отличие заключается в том, как эта информация используется в процессе реализации проекта.
- Очень упрощённо можно сказать, что при использовании v-образной модели на каждой стадии «на спуске» нужно думать о том, что и как будет происходить на соответствующей стадии «на подъёме». Тестирование здесь появляется уже на самых ранних стадиях развития проекта, что позволяет минимизировать риски, а также обнаружить и устранить множество потенциальных проблем до того, как они станут проблемами реальными

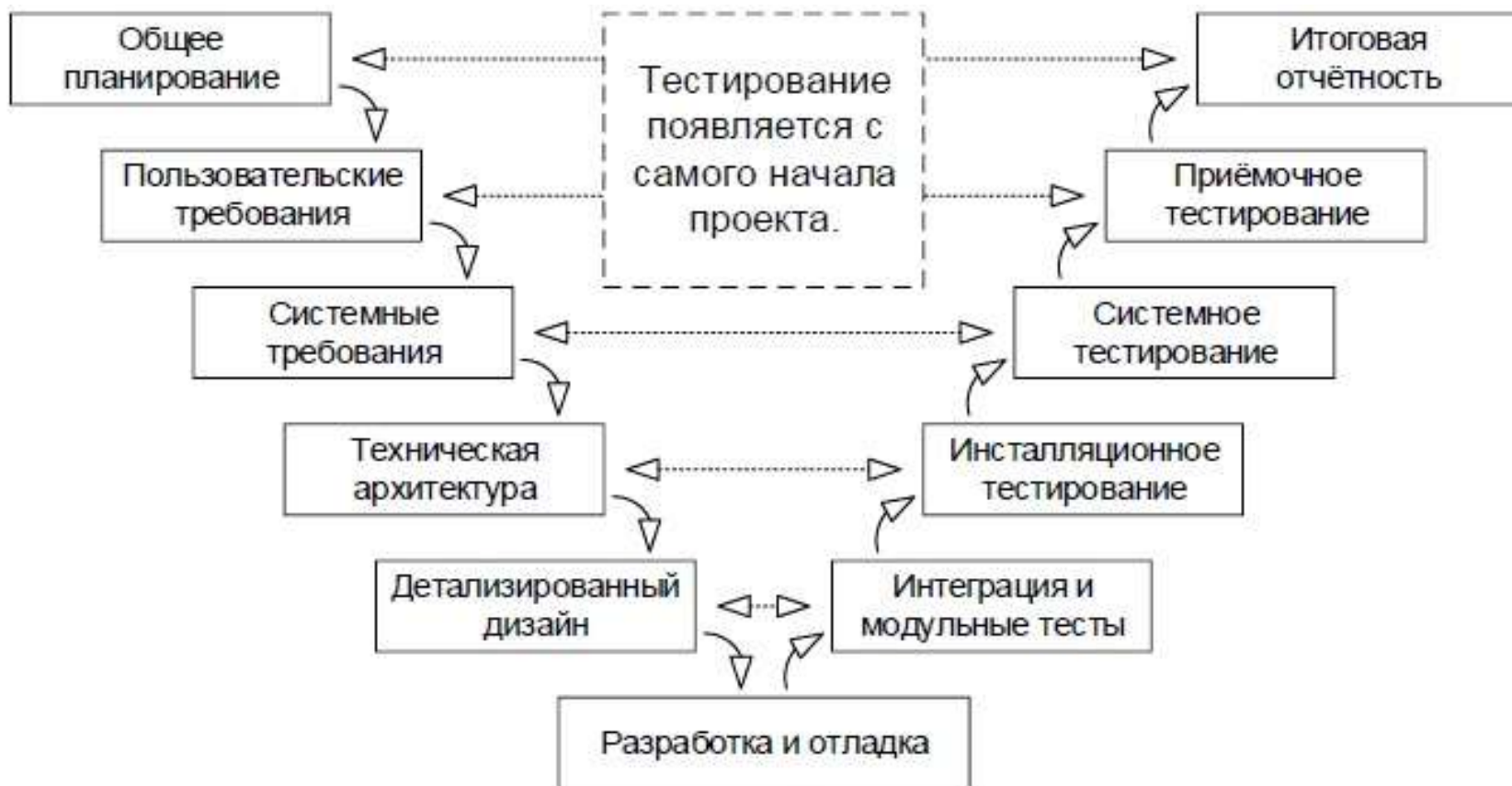


Рисунок 2.1.b — V-образная модель разработки ПО

- **Итерационная инкрементальная модель** (iterative model, incremental model) является фундаментальной основой современного подхода к разработке ПО. Как следует из названия модели, ей свойственна определённая двойственность
- с точки зрения жизненного цикла модель является **итерационной**, т.к. подразумевает многократное повторение одних и тех же стадий;
- с точки зрения развития продукта (приращения его полезных функций) модель является **инкрементальной**.
- Ключевой особенностью данной модели является разбиение проекта на относительно небольшие промежутки (итерации), каждый из которых в общем случае может включать в себя все классические стадии, присущие водопадной и v-образной моделям (рисунок 2.1.с). Итогом итерации является приращение (инкремент) функциональности продукта, выраженное в промежуточном билде (build).



Рисунок 2.1.с — Итерационная инкрементальная модель разработки ПО

- Длина итераций может меняться в зависимости от множества факторов, однако сам принцип многократного повторения позволяет гарантировать, что и тестирование, и демонстрация продукта конечному заказчику (с получением обратной связи) будет активно применяться с самого начала и на протяжении всего времени разработки проекта.
- Во многих случаях допускается распараллеливание отдельных стадий внутри итерации и активная доработка с целью устранения недостатков, обнаруженных на любой из (предыдущих) стадий.
- Итерационная инкрементальная модель очень хорошо зарекомендовала себя на объёмных и сложных проектах, выполняемых большими командами на протяжении длительных сроков. Однако к основным недостаткам этой модели часто относят высокие накладные расходы, вызванные высокой «бюрократизированностью» и общей громоздкостью модели.

- **Спиральная модель** (spiral model) представляет собой частный случай итерационной инкрементальной модели, в котором особое внимание уделяется управлению рисками, в особенности влияющими на организацию процесса разработки проекта и контрольные точки.
- Схематично суть спиральной модели представлена на рисунке 2.1.d. Обратите внимание на то, что здесь явно выделены четыре ключевые фазы:
 - проработка целей, альтернатив и ограничений;
 - анализ рисков и прототипирование;
 - разработка (промежуточной версии) продукта;
 - планирование следующего цикла.
- С точки зрения тестирования и управления качеством повышенное внимание рискам является ощутимым преимуществом при использовании спиральной модели для разработки концептуальных проектов, в которых требования естественным образом являются сложными и нестабильными (могут многократно меняться по ходу выполнения проекта).

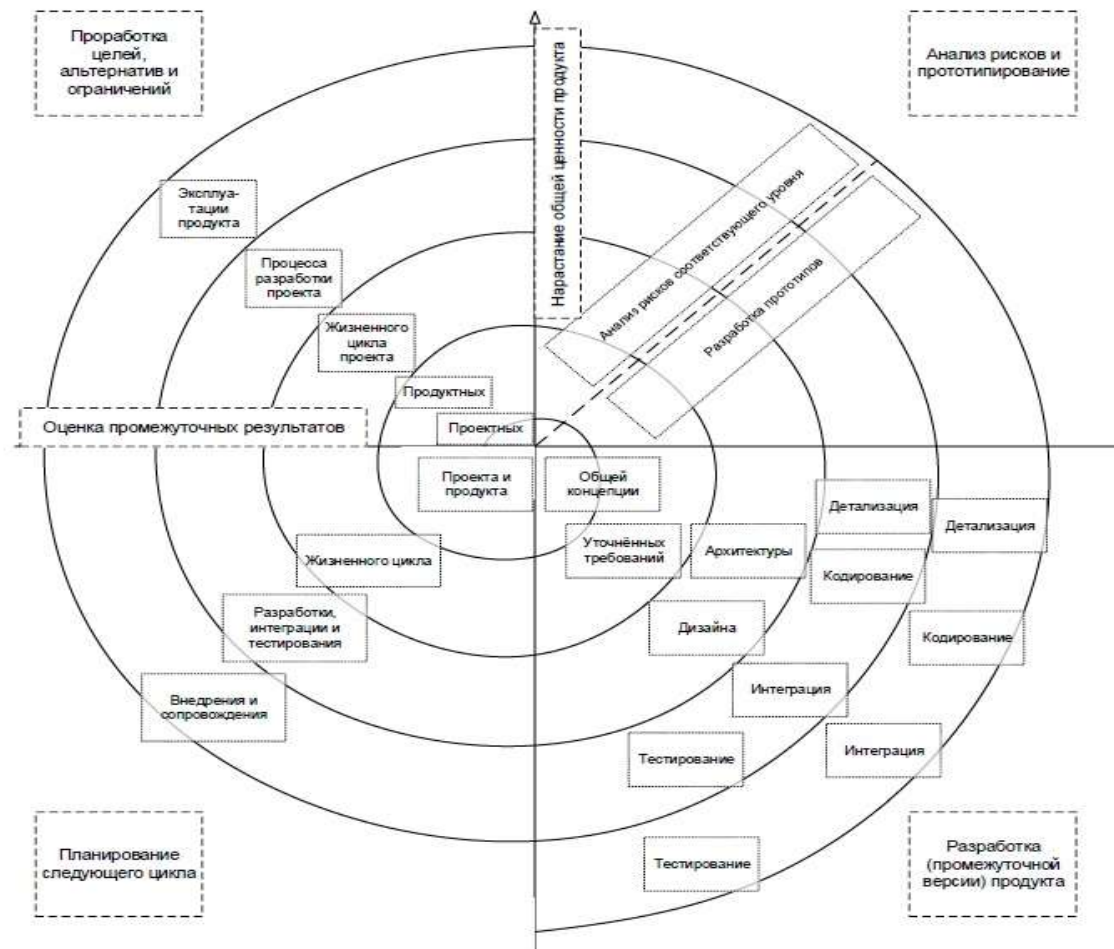


Рисунок 2.1.d — Спиральная модель разработки ПО

- **Гибкая модель** (agile model) представляет собой совокупность различных подходов к разработке ПО и базируется на т.н. «agile-манифесте»³⁴:
 - Люди и взаимодействие важнее процессов и инструментов.
 - Работающий продукт важнее исчерпывающей документации.
 - Сотрудничество с заказчиком важнее согласования условий контракта.
 - Готовность к изменениям важнее следования первоначальному плану.

Как несложно догадаться, положенные в основу гибкой модели подходы являются логическим развитием и продолжением всего того, что было за десятилетия создано и опробовано в водопадной, v-образной, итерационной инкрементальной, спиральной и иных моделях. Причём здесь впервые был достигнут ощутимый результат в снижении бюрократической составляющей и максимальной адаптации процесса разработки ПО к мгновенным изменениям рынка и требований заказчика.

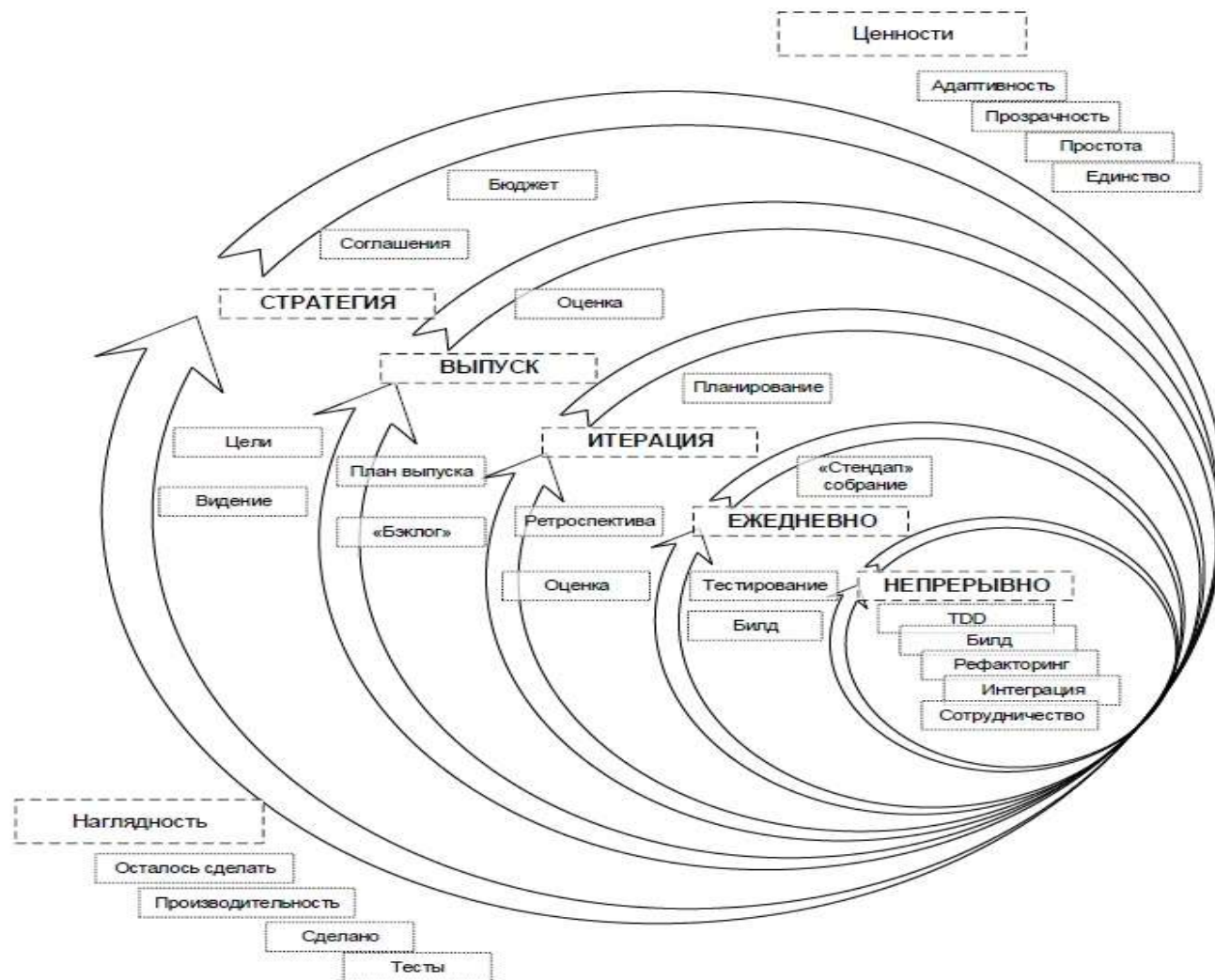


Рисунок 2.1.е — Суть гибкой модели разработки ПО

- Главным недостатком гибкой модели считается сложность её применения к крупным проектам, а также частое ошибочное внедрение её подходов, вызванное недопониманием фундаментальных принципов модели.
- Тем не менее можно утверждать, что всё больше и больше проектов начинают использовать гибкую модель разработки.

Тема 2. Интегрированная среда разработки ПО

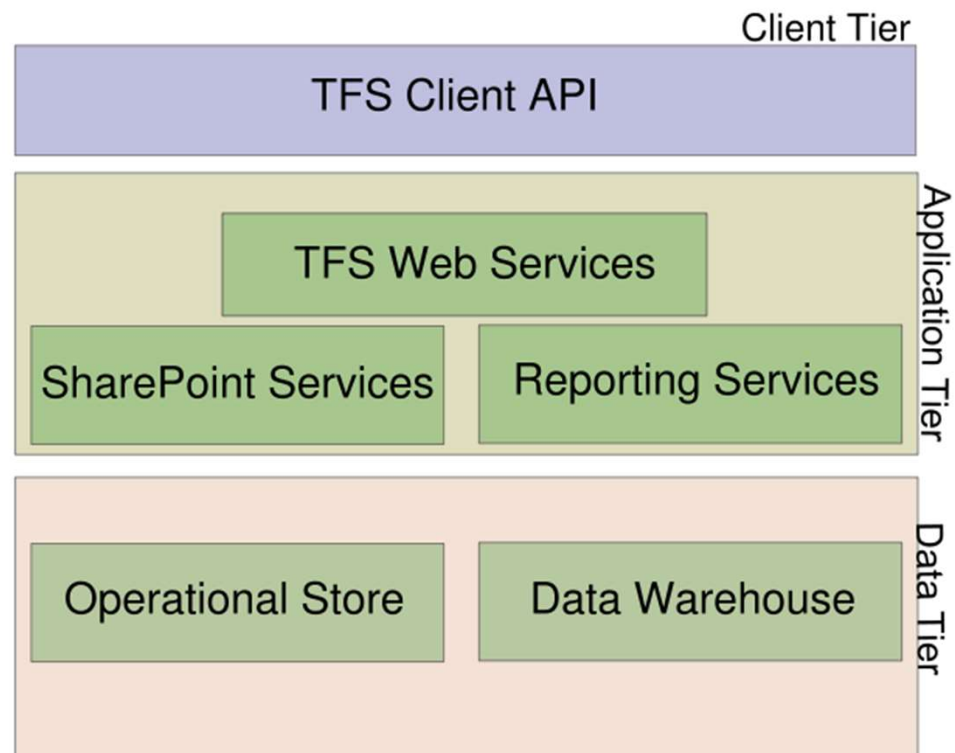
1. Основные компоненты среды разработки
2. Приёмы работы
3. Поддержка совместной работы над проектом

Интегрированная среда разработки - Integrated development environment (IDE)

Основные компоненты IDE

- редактор кода
- средства сборки проекта
- средства запуска проекта
- средства отладки
- средства тестирования
- поддержка совместной работы
- расширения

3-уровневая архитектура Team Foundation Server

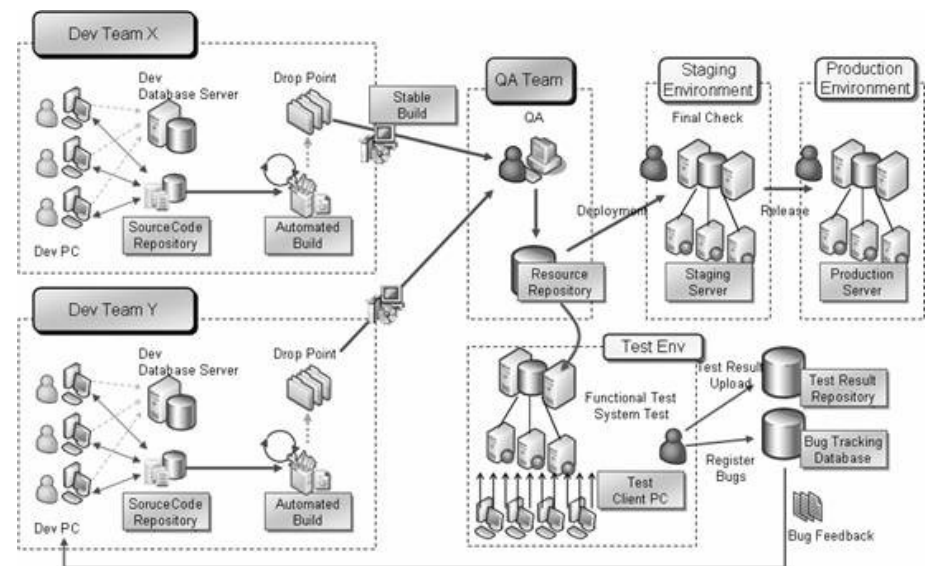
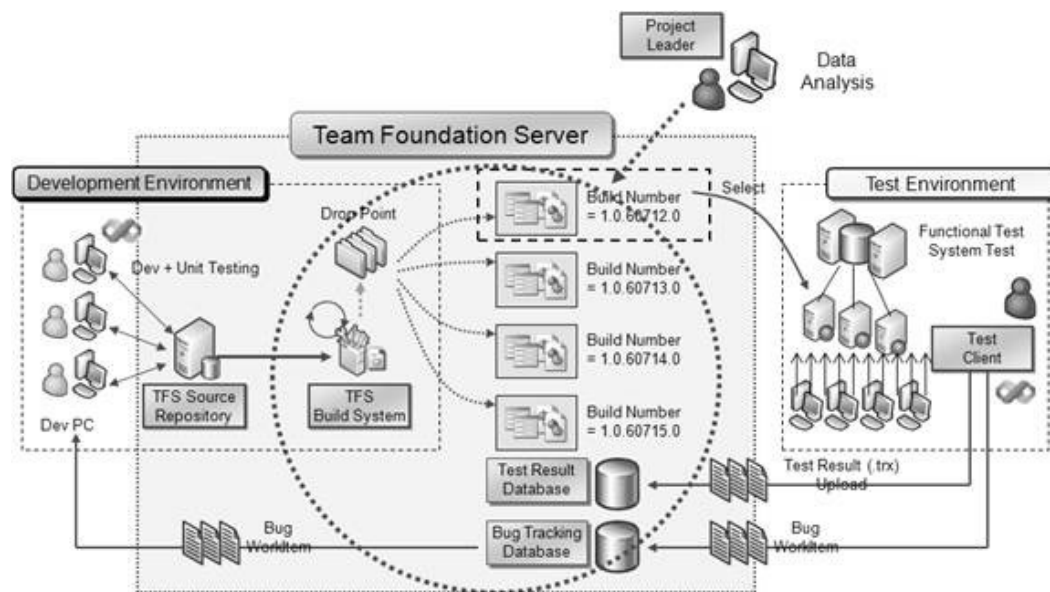


- Team Foundation Server работает по трёхуровневой
- *клиентский уровень,*
- *прикладной уровень*
- *уровень данных.*
- *Клиентский уровень* используется для создания и проектами, а также для доступа к хранимым и проекта. На этом уровне TFS не содержит никаких интерфейсов, но предоставляет веб-сервисы, которые использованы клиентскими приложениями для интеграции в функциональность TFS.
- Эти веб-сервисы используются такими приложениями, Team System для применения TFS в качестве серверной инфраструктуры хранилища информации или выделенного TFS управления приложениями

- Сами веб-сервисы находятся на *прикладном уровне*.
- *Прикладной уровень* также включает в себя веб-портал и репозиторий (хранилище) документации, поддерживаемые Windows SharePoint Services.
- Веб-портал, называемый *Team Project Portal* (портал командного проекта), выступает в роли центра взаимодействия для проектов, управляемых TFS. Репозиторий документов используется как для элементов проекта, так и для отслеживания ревизий (документирование изменений), а также для накопления и обработки данных и генерации отчетов. *Уровень данных*, основывающийся в первую очередь на установленном SQL Server обеспечивает сервисы постоянного хранения данных для репозитория документов. *Уровень данных* и *уровень приложений* могут существовать на различных физических или виртуальных серверах при использовании Windows Server 2003 или более специализированных версий. *Уровень данных* не

- Большая часть действий в Team Foundation Server происходит с «рабочими элементами».
- Рабочими элементами называются отдельные единицы (шаги) работы, выполняемые поочередно. Во многих источниках они отождествляются с элементами типа «ошибка» (bug) в системах отслеживания ошибок наподобие Bugzilla, то есть в этом случае рабочий элемент имеет поля Area (связанная область), Iteration (состояние), Assignee (связан с), Reported By (кем создан) для указания соответствующей информации, историю, приложенные файлы, а также множество прочих атрибутов.
- Рабочие элементы сами по себе могут быть нескольких типов, например *Ошибка*, *Задача*, *Требование качества*, *Сценарий* и т. д.. Выбранный фреймворк для любого взятого проекта в Team Foundation Server определяет какие типы рабочих элементов будут доступны и какими атрибутами каждый из типов будет обладать.
- Эти элементы внутри приложения хранятся в XML-формате, а их схема может быть легко модифицирована для добавления новых атрибутов к различным элементам или создания новых элементов на проектной основе. Каждый рабочий элемент имеет соответствующие методики контроля, которые определяют, кому какие именно элементы доступны и какие действия он с ними может производить (просмотр, редактирование, создание, удаление и др.). Также подразумевается использование уведомлений и возможности ведения логов, для отслеживания истории всего

Логическая структура



- Система управления версиями (VCS)
- также известная как система управления исходным кодом,
— это **программное обеспечение для отслеживания изменений в файловой системе и управления ими.**

О системе контроля версий

Что такое «система контроля версий» и почему это важно?

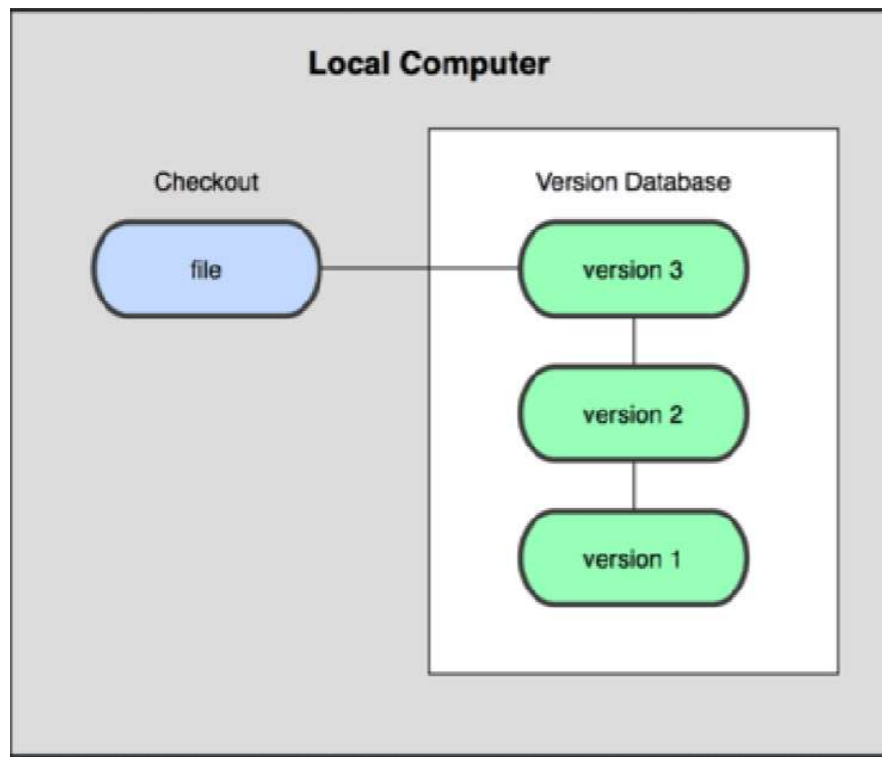
Система контроля версий — это система, записывающая изменения в файл или набор файлов в течение времени и позволяющая вернуться позже к определённой версии.

Для контроля версий файлов примером может быть использование исходного кода программного обеспечения, хотя на самом деле вы можете использовать контроль версий практически для любых типов файлов.

Если вы графический или web-дизайнер и хотите сохранить каждую версию изображения или макета (скорее всего, захотите), система контроля версий (далее СКВ) — как раз то, что нужно. Она позволяет вернуть файлы к состоянию, в котором они были до изменений, вернуть проект к исходному состоянию, увидеть изменения, увидеть, кто последний менял что-то и вызвал проблему, кто поставил задачу и когда и многое другое.

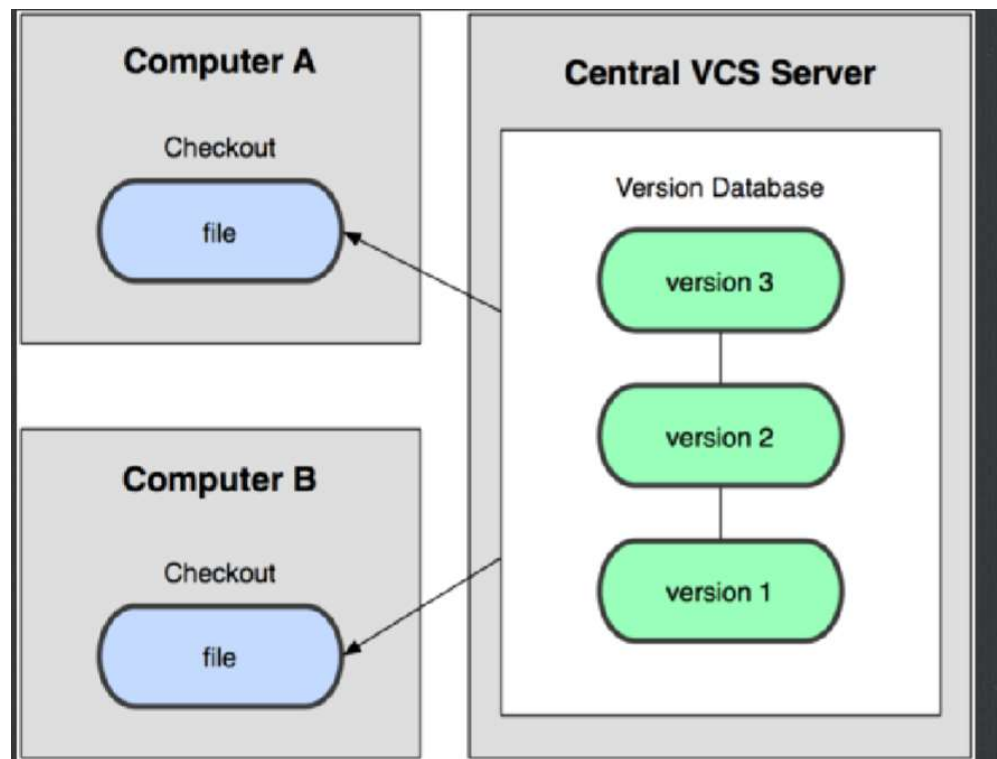
Использование СКВ также значит в целом, что, если вы сломали что-то или потеряли файлы, вы спокойно можете всё исправить. В дополнение ко всему вы получите всё это без каких-либо дополнительных усилий.

- **Локальные системы контроля версий**



- Многие люди в качестве метода контроля версий применяют копирование файлов в отдельный каталог (возможно даже, каталог с отметкой по времени, если они достаточно сообразительны).
- Данный подход очень распространён из-за его простоты, однако он невероятно сильно подвержен появлению ошибок. Можно легко забыть в каком каталоге вы находитесь и случайно изменить не тот файл или скопировать не те файлы, которые вы хотели.
- Для того, чтобы решить эту проблему, программисты давным-давно разработали локальные СКВ с простой базой данных, которая хранит записи о всех изменениях в файлах, осуществляя тем самым контроль ревизий.
- Одной из популярных СКВ была система RCS, которая и сегодня распространяется со многими компьютерами. [RCS](#) хранит на диске наборы патчей (различий между файлами) в специальном формате, применяя которые она может воссоздавать состояние каждого файла в заданный момент времени.

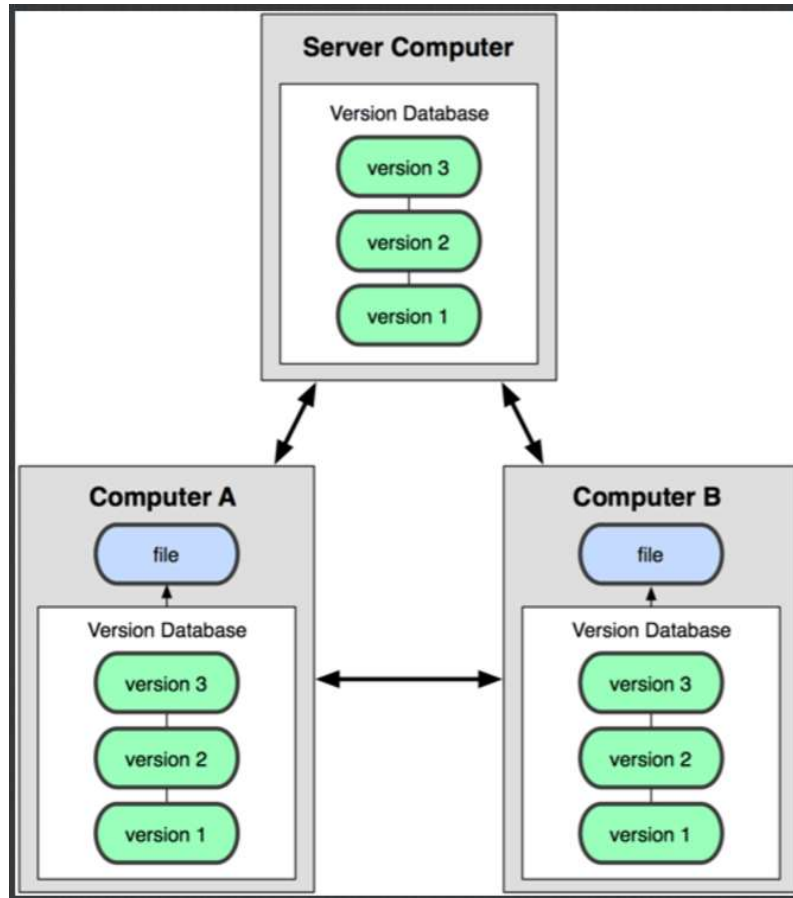
- **Централизованные системы контроля версий**



- Следующая серьёзная проблема, с которой сталкиваются люди, — это необходимость взаимодействовать с другими разработчиками. Для того, чтобы разобраться с ней, были разработаны централизованные системы контроля версий (ЦСКВ). Такие системы, как CVS, Subversion и Perforce, используют единственный сервер, содержащий все версии файлов, и некоторое количество клиентов, которые получают файлы из этого централизованного хранилища. Применение ЦСКВ являлось стандартом на протяжении многих лет.

- Такой подход имеет множество преимуществ, особенно перед локальными СКВ. Например, все разработчики проекта в определённой степени знают, чем занимается каждый из них. Администраторы имеют полный контроль над тем, кто и что может делать, и гораздо проще администрировать ЦСКВ, чем оперировать локальными базами данных на каждом клиенте.
- Несмотря на это, данный подход тоже имеет серьёзные минусы. Самый очевидный минус — это единая точка отказа, представленная централизованным сервером. Если этот сервер выйдет из строя на час, то в течение этого времени никто не сможет использовать контроль версий для сохранения изменений, над которыми работает, а также никто не сможет обмениваться этими изменениями с другими разработчиками. Если жёсткий диск, на котором хранится центральная БД, повреждён, а своевременные бэкапы отсутствуют, вы потеряете всё — всю историю проекта, не считая единичных снимков репозитория, которые сохранились на локальных машинах разработчиков. Локальные СКВ страдают от той же самой проблемы: когда вся история проекта хранится в одном месте, вы рискуете потерять всё.

- **Распределённые системы контроля версий**



- Здесь в игру вступают распределённые системы контроля версий (РСКВ). В РСКВ (таких как Git, Mercurial, Bazaar или Darcs) клиенты не просто скачивают снимок всех файлов (состояние файлов на определённый момент времени) — они полностью копируют репозиторий. В этом случае, если один из серверов, через который разработчики обменивались данными, умрёт, любой клиентский репозиторий может быть скопирован на другой сервер для продолжения работы. Каждая копия репозитория является полным бэкапом всех данных.
- Более того, многие РСКВ могут одновременно взаимодействовать с несколькими удалёнными репозиториями, благодаря этому вы можете работать с различными группами людей, применяя различные подходы единовременно в рамках одного проекта. Это позволяет применять сразу несколько подходов в разработке, например, иерархические модели, что совершенно невозможно в централизованных системах.

Git

Преимущества

- Скорость
- простой дизайн
- поддержка нелинейной разработки
- Распределённость
- поддержка больших проектов

- Git — это распределенная система контроля версий нашего кода.
- Для распределенных команд нужна какая-то система управления работы. Нужна, чтобы отслеживать изменения, которые происходят со временем.
- То есть шаг за шагом мы видим, какие файлы изменились и как. Особенно это важно, когда анализируешь, что было сделано в рамках одной задачи: это дает возможность возвращаться назад.
- Представим себе ситуацию: был работающий код, всё в нем было хорошо, но мы решили что-то улучшить, там подправить, сям подправить. Все ничего, но такое улучшение поломало половину функционала, сделало невозможным работу.

- Без Гита нужно было бы часами сидеть и вспоминать, как же все было изначально. А так мы просто откатываемся на коммит назад — и все.
- Или что делать, если есть два разработчика, которые делают одновременно свои изменения в коде?
- Без Гита это выглядит так: они скопировали код из оригинала, сделали что нужно. Наступает момент, и оба хотят добавить свои изменения в главную папку. И что делать в этой ситуации?.. Я даже не берусь оценить время, чтоб проделать эту работу. Таких проблем не будет вовсе, если пользоваться Гитом.

- **Основы Git**

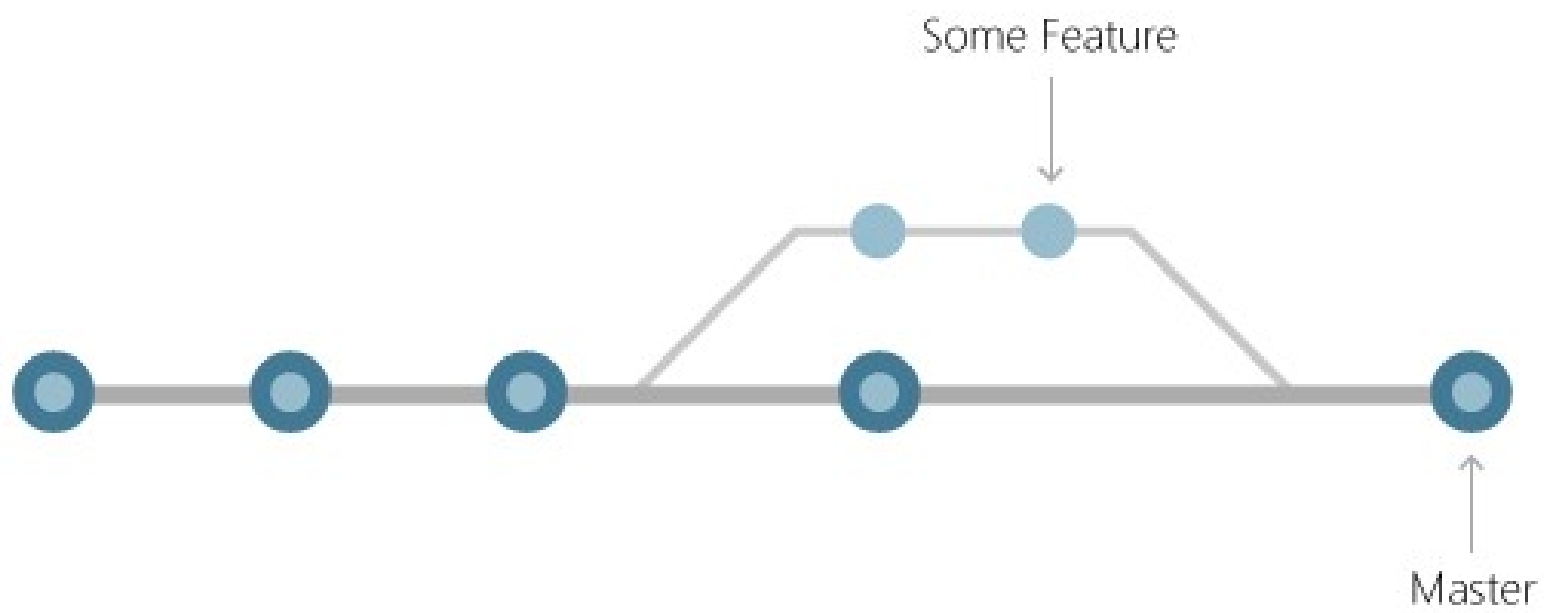
- При каждом сохранении работы Git создает фиксацию. Фиксация — это моментальный снимок всех файлов в определенный момент времени. Если файл не изменился с одной фиксации на следующую, Git использует ранее сохраненный файл. Эта конструкция отличается от других систем, которые хранят начальную версию файла и сохраняют запись изменений с течением времени.



- Фиксации создают ссылки на другие фиксации, формируя граф журнала разработки. Можно вернуть код к предыдущей фиксации, проверить, как файлы изменились с одной фиксации на следующую, и просмотреть сведения о том, где и когда были внесены изменения. Фиксации идентифицируются в Git уникальным криптографическим хэшем содержимого фиксации. Так как все хэшируется, невозможно вносить изменения, терять сведения или повреждены файлы без обнаружения Git.

- **Ветви**

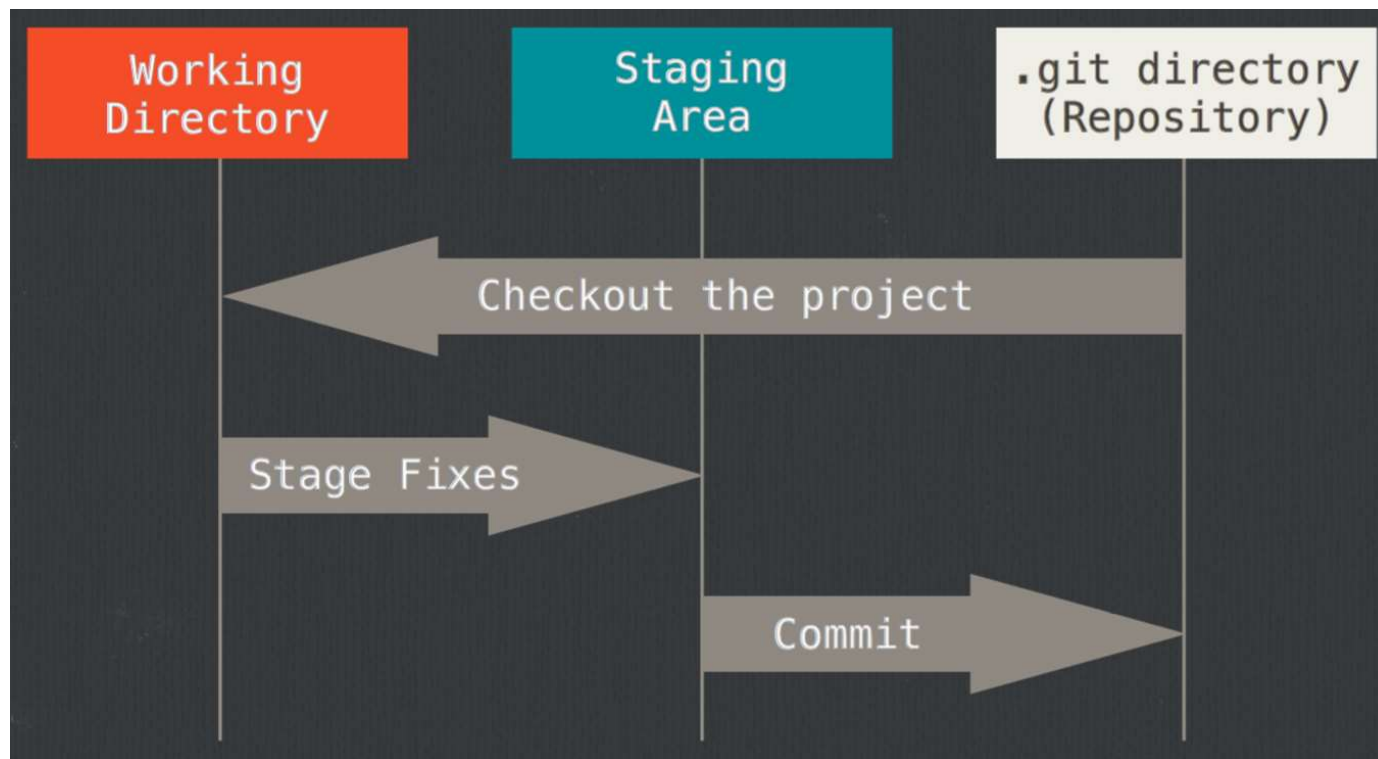
- Каждый разработчик сохраняет изменения в собственном локальном репозитории кода. В результате может быть много разных изменений, основанных на одной фиксации.
- Git предоставляет средства для изоляции изменений и последующего их слияния. Ветви, которые являются упрощенными указателями для выполнения работы, управляют этим разделением. После завершения работы, созданной в ветви, эту ветвь можно снова объединить с основной ветвью команды (или магистралью).



- **Файлы и фиксации**

- Файлы в Git находятся в одном из трех состояний: измененных, промежуточных или зафиксированных.
- При первом изменении файла изменения существуют только в рабочем каталоге. Они еще не являются частью фиксации или истории разработки. Разработчик должен *подготовить* измененные файлы для включения в фиксацию.
- Промежуточная область содержит все изменения, которые необходимо включить в следующую фиксацию. Когда разработчик доволен промежуточными файлами, файлы упаковываются в виде *фиксации* с сообщением, описывающим изменения. Эта фиксация становится частью истории разработки.

Секции Git-проекта



- У Git есть несколько состояний
- неотслеживаемое (untracked);
- измененное (modified);
- подготовленное (staged);
- закомиченное (committed).

Это состояния, в которых находятся файлы из нашего кода. То есть, их жизненный путь обычно выглядит так:

Файл, который создан и не добавлен в репозиторий, будет в состоянии untracked.

Делаем изменения в файлах, которые уже добавлены в гит репозиторий — находятся в состоянии modified.

Из тех файлов, которые мы изменили, выбираем только те (или все), которые нужны нам (например, скомпилированные классы нам не нужны), и эти классы с изменениями попадают в состояние staged.

Из заготовленных файлов из состояния staged создается коммит и переходит уже в гит репозиторий. После этого staged состояние — пустое. А вот modified еще может что-то содержать.

- **Что такое коммит**

- Коммит — это основной объект в управлении контроле версий. Он содержит все изменения за время этого коммита.
- Коммиты связаны между собой как односвязный список. А именно: Есть первый коммит. Когда создается второй коммит, то он (второй) знает, что идет после первого. И таким образом можно отследить информацию. Также у коммита есть еще своя информация, так называемые метаданные: уникальный идентификатор коммита, по которому можно его найти;
- имя автора коммита, который создал его;
- дата создания коммита;
- комментарий, который описывает, что было сделано во время этого коммита.

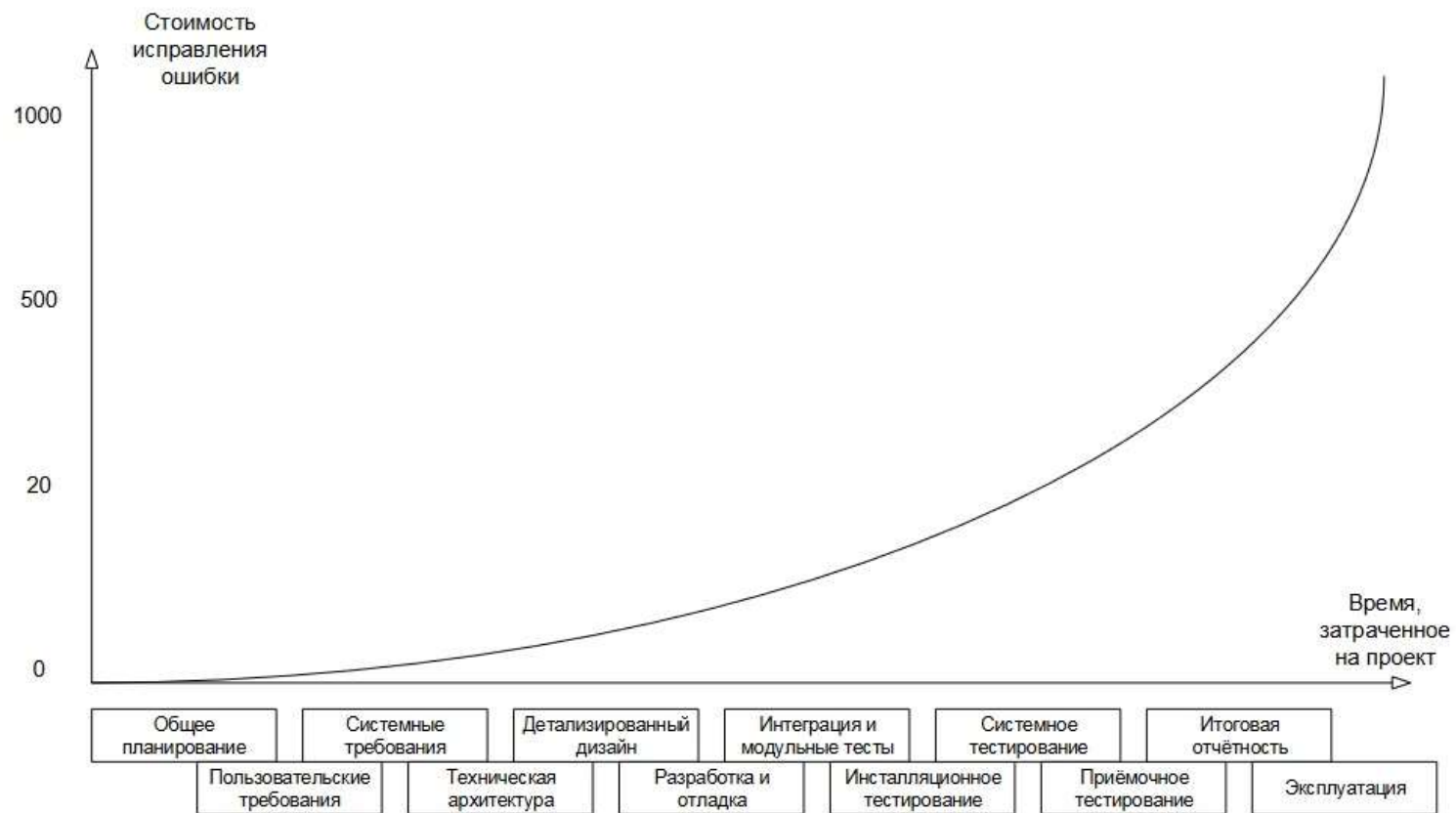
- **Тестирование документации и требований**
- Требование (requirement) — описание того, какие функции и с соблюдением каких условий должно выполнять приложение в процессе решения полезной для пользователя задачи.

- если поискать определения требований в литературе 10-20-30-летней давности, то можно заметить, что изначально о пользователях, их задачах и полезных для них свойствах приложения в определении требования не было сказано.
- Пользователь выступал некоей абстрактной фигурой, не имеющей отношения к приложению. В настоящее время такой подход недопустим, т.к. он не только приводит к коммерческому провалу продукта на рынке, но и многократно повышает затраты на разработку и тестирование.

- Важность требований
- Требования являются отправной точкой для определения того, что проектная команда будет проектировать, реализовывать и тестировать. Элементарная логика говорит нам, что если в требованиях что-то «не то», то и реализовано будет «не то», т.е. колоссальная работа множества людей будет выполнена впустую. Эту мысль иллюстрирует рисунок 2.2.а.
- Брайан Хэнкс, описывая важность требований, подчёркивает, что они:
 - Позволяют понять, что и с соблюдением каких условий система должна делать.
 - Предоставляют возможность оценить масштаб изменений и управлять изменениями.
 - Являются основой для формирования плана проекта (в том числе плана тестирования).
 - Помогают предотвращать или разрешать конфликтные ситуации.
 - Упрощают расстановку приоритетов в наборе задач.
 - Позволяют объективно оценить степень прогресса в разработке проекта.

- Вне зависимости от того, какая модель разработки ПО используется на проекте, чем позже будет обнаружена проблема, тем сложнее и дороже будет её решение.

А в самом начале («водопада», «спуска по букве v», «итерации», «витка спирали») идёт планирование и работа с требованиями. Если проблема в требованиях будет выяснена на этой стадии, её решение может свестись к исправлению пары слов в тексте, в то время как недоработка, вызванная пропущенной проблемой в требованиях и обнаруженная на стадии эксплуатации, может даже полностью уничтожить проект



- **Ещё одним аргументом** в пользу тестирования требований является то, что, по разным оценкам, в них зарождается от $\frac{1}{2}$ до $\frac{3}{4}$ всех проблем с программным обеспечением. В итоге есть риск, что получится так, как показано на рисунке 2.2.b. Поскольку мы постоянно говорим «документация и требования», а не просто «требования», то стоит рассмотреть перечень документации, которая должна подвергаться тестированию в процессе разработки ПО (хотя далее мы будем концентрироваться именно на требованиях).



Так клиент
объяснил, чего он
хочет



Так клиента понял
менеджер проекта



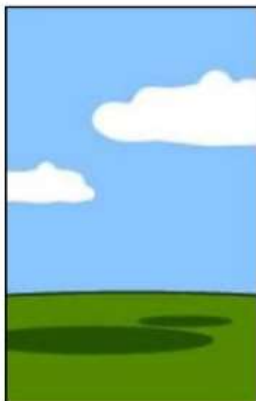
Так аналитик
описал проект



Так программист
реализовал проект



Так проект был
прорекламирован
консультантами



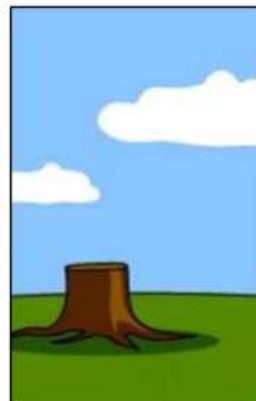
Так проект был
задокументирован



Так проект был
сдан в
эксплуатацию



В такую сумму
проект обошёлся
заказчику



Так работала
техническая
поддержка



Что на самом деле
было нужно
клиенту

- В общем случае документацию можно разделить на два больших вида в зависимости от времени и места её использования
- Продуктная документация
- Проектная документация

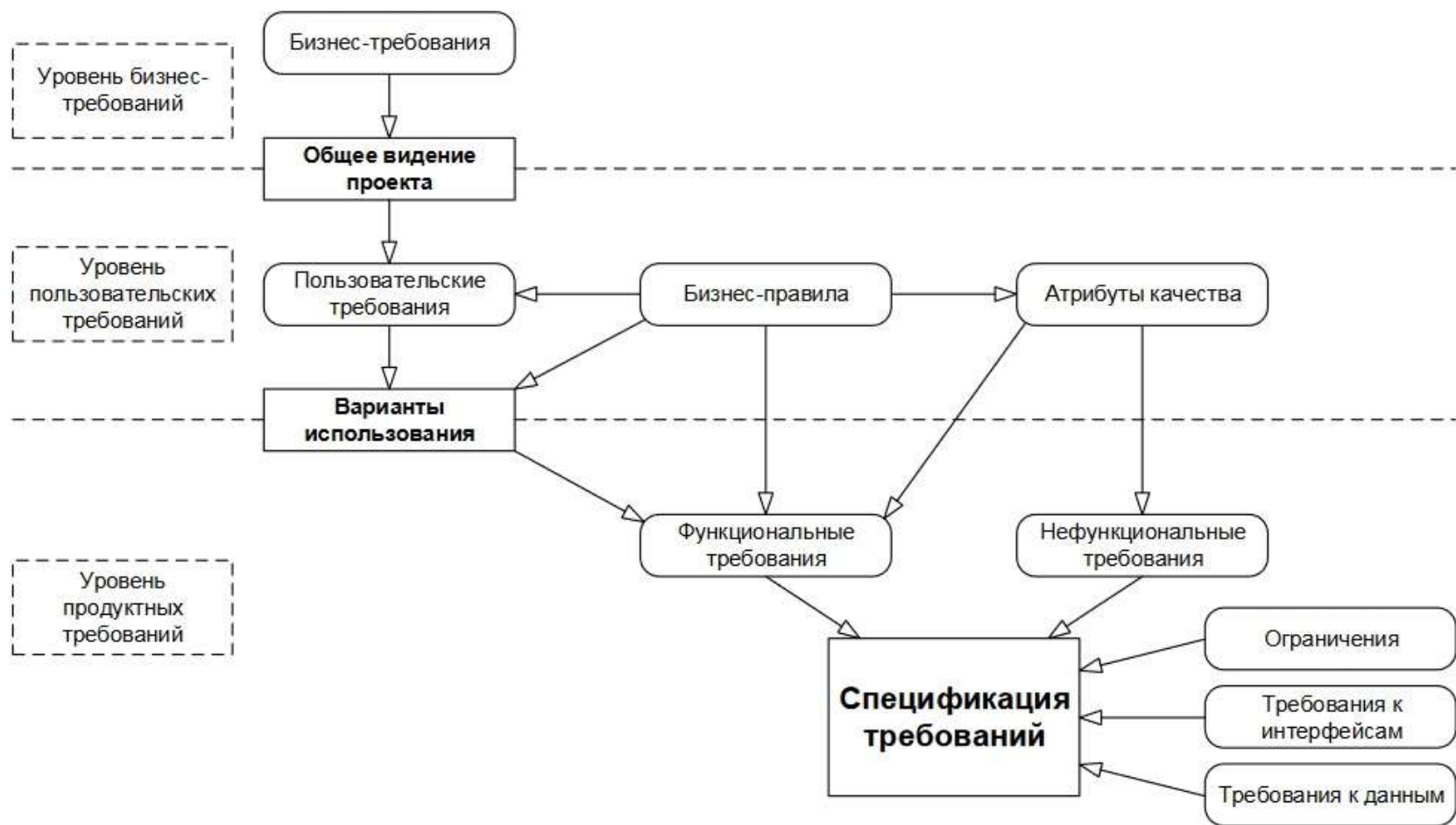
- **Продуктная документация** (product documentation, development documentation) используется проектной командой во время разработки и поддержки продукта.
- Она включает:
 - План проекта (project management plan) и в том числе тестовый план (test plan).
 - Требования к программному продукту (product requirements document, PRD) и функциональные спецификации (functional specifications document, FSD; software requirements specification, SRS).
- Архитектуру и дизайн (architecture and design).
- Тест-кейсы и наборы тест-кейсов (test cases, test suites).
- Технические спецификации (technical specifications), такие как схемы баз данных, описания алгоритмов, интерфейсов и т.д.

- **Проектная документация** (project documentation)
- включает в себя как продуктную документацию, так и некоторые дополнительные виды документации и используется не только на стадии разработки, но и на более ранних и поздних стадиях (например, на стадии внедрения и эксплуатации). Она включает:
- Пользовательскую и сопроводительную документацию (user and accompanying documentation), такую как встроенная помощь, руководство по установке и использованию, лицензионные соглашения и т.д.
- Маркетинговую документацию (market requirements document, MRD), которую представители разработчика или заказчика используют как на начальных этапах (для уточнения сути и концепции проекта), так и на финальных этапах развития проекта (для продвижения продукта на рынке).

- **Уровни и типы требований**

Форма представления, степень детализации и перечень полезных свойств требований зависят от уровней и типов требований

- **Бизнес-требования** (business requirements) выражают цель, ради которой разрабатывается продукт (зачем вообще он нужен, какая от него ожидается польза).
- Результатом выявления требований на этом уровне является общее видение (vision and scope) — документ, который, как правило, представлен простым текстом и таблицами. Здесь нет детализации поведения системы и иных технических характеристик, но вполне могут быть определены приоритеты решаемых бизнес-задач, риски и т.п.
- Несколько простых, изолированных от контекста и друг от друга примеров бизнес-требований:
- Нужен инструмент, в реальном времени отображающий наиболее выгодный курс покупки и продажи валюты.
- Необходимо в два-три раза повысить количество заявок, обрабатываемых одним оператором за смену.
- Нужно автоматизировать процесс выписки товарно-транспортных накладных на основе договоров



- **Пользовательские требования** (user requirements) описывают задачи, которые пользователь может выполнять с помощью разрабатываемой системы (реакцию системы на действия пользователя, сценарии работы пользователя). Поскольку здесь уже появляется описание поведения системы, требования этого уровня могут быть использованы для оценки объёма работ, стоимости проекта, времени разработки и т.д.
- Пользовательские требования оформляются в виде вариантов использования (use cases), пользовательских историй (user stories), пользовательских сценариев (user scenarios).
- Например:
- При первом входе пользователя в систему должно отображаться лицензионное соглашение.
- Администратор должен иметь возможность просматривать список всех пользователей, работающих в данный момент в системе.
- При первом сохранении новой статьи система должна выдавать запрос на сохранение в виде черновика или публикацию.

- **Бизнес-правила** (business rules) описывают особенности принятых в предметной области (и/или непосредственно у заказчика) процессов, ограничений и иных правил. Эти правила могут относиться к бизнес-процессам, правилам работы сотрудников, нюансам работы ПО и т.д. Несколько простых, изолированных от контекста и друг от друга примеров бизнес-правил:
 - Никакой документ, просмотренный посетителями сайта хотя бы один раз, не может быть отредактирован или удалён.
 - Публикация статьи возможна только после утверждения главным редактором.
 - Подключение к системе извне офиса запрещено в нерабочее время.

- **Атрибуты качества** (quality attributes) расширяют собой нефункциональные требования и на уровне пользовательских требований могут быть представлены в виде описания ключевых для проекта показателей качества (свойств продукта, не связанных с функциональностью, но являющихся важными для достижения целей создания продукта — производительность, масштабируемость, восстанавливаемость). Атрибутов качества очень много, но для любого проекта реально важными является лишь некоторое их подмножество.
Например:
- Максимальное время готовности системы к выполнению новой команды после отмены предыдущей не может превышать одну секунду.
- Внесённые в текст статьи изменения не должны быть потеряны при нарушении соединения между клиентом и сервером.
- Приложение должно поддерживать добавление произвольного количества неиероглифических языков интерфейса

- **Функциональные требования** (functional requirements) описывают поведение системы, т.е. её действия (вычисления, преобразования, проверки, обработку и т.д.) В контексте проектирования функциональные требования в основном влияют на дизайн системы.

Несколько простых, изолированных от контекста и друг от друга примеров функциональных требований:

- В процессе инсталляции приложение должно проверять остаток свободного места на целевом носителе.
- Система должна автоматически выполнять резервное копирование данных ежедневно в указанный момент времени.
- Электронный адрес пользователя, вводимый при регистрации, должен быть проверен на соответствие требованиям RFC822.

- **Нефункциональные требования** (non-functional requirements) описывают свойства системы (удобство использования, безопасность, надёжность, расширяемость и т.д.), которыми она должна обладать при реализации своего поведения.
Здесь приводится более техническое и детальное описание атрибутов качества. В контексте проектирования нефункциональные требования в основном влияют на архитектуру системы.
- Например
- При одновременной непрерывной работе с системой 1000 пользователей, минимальное время между возникновением сбоев должно быть более или равно 100 часов.
- Ни при каких условиях общий объём используемой приложением памяти не может превышать 2 ГБ.
- Размер шрифта для любой надписи на экране должен поддерживать настройку в диапазоне от 5 до 15 пунктов.

Следующие требования в общем случае могут быть отнесены к нефункциональным, однако их часто выделяют в отдельные подгруппы (рассмотрим 3 но их может быть намного больше)

- **Ограничения** (limitations, constraints) представляют собой факторы, ограничивающие выбор способов и средств (в том числе инструментов) реализации продукта.
- например
- Все элементы интерфейса должны отображаться без прокрутки при разрешениях экрана от 800x600 до 1920x1080.
- Не допускается использование Flash при реализации клиентской части приложения.
- Приложение должно сохранять способность реализовывать функции с уровнем важности «критический» при отсутствии у клиента поддержки JavaScript.

- **Требования к интерфейсам** (external interfaces requirements) описывают особенности взаимодействия разрабатываемой системы с другими системами и операционной средой.
Например
- Обмен данными между клиентской и серверной частями приложения при осуществлении фоновых AJAX-запросов должен быть реализован в формате JSON.
- Протоколирование событий должно вестись в журнале событий операционной системы.
- Соединение с почтовым сервером должно выполняться согласно RFC3207(«SMTP over TLS»)

- **Требования к данным** (data requirements) описывают структуры данных (и сами данные), являющиеся неотъемлемой частью разрабатываемой системы. Часто сюда относят описание базы данных и особенностей её использования.

Например

- Все данные системы, за исключением пользовательских документов, должны храниться в БД под управлением СУБД MySQL, пользовательские документы должны храниться в БД под управлением СУБД MongoDB.
- Информация о кассовых транзакциях за текущий месяц должна храниться в операционной таблице, а по завершении месяца переноситься в архивную.
- Для ускорения операций поиска по тексту статей и обзоров должны быть предусмотрены полнотекстовые индексы на соответствующих полях таблиц

- **Спецификация требований** (software requirements specification, SRS) объединяет в себе описание всех требований уровня продукта и может представлять собой весьма объёмный документ (сотни и тысячи страниц).
Поскольку требований может быть очень много, а их приходится не только единожды написать и согласовать между собой, но и постоянно обновлять, работу проектной команды по управлению требованиями значительно облегчают соответствующие инструментальные средства (requirements management tools)

ТЕСТИРОВАНИЯ В ПРОЦЕССЕ РАЗРАБОТКИ ПО

Аналитический отдел. В задачи аналитического отдела входят:

- определение концепций и функционального направления развития программного продукта;
- проведение предпроектного обследования;
- определение функциональных возможностей системы;
- определение (совместно с разработчиками) технических требований к системе;
- описание бизнес-процессов предметной области в терминах, понятных разработчикам
(постановки задач и спецификации на разработку);
- написание постановок задач и спецификаций на доработку программного средства при
изменении законодательства, требований клиентов, расширении функциональных возможностей продукта;
- контроль процесса реализации новых возможностей в программных продуктах компании.

Отдел документации. Часто данный отдел не выделяется в обособленную структуру, он

может входить, например, в состав аналитического отдела.

В задачи отдела входят написание технической документации для конечного пользователя,

отслеживание изменений в программном средстве и актуализация в документации.

Отдел разработки. Это ключевой отдел для фирмы. Если без остальных отделов зачастую можно обойтись, то без отдела разработки нельзя. В его задачи входят:

- определение (совместно с аналитиками) технических требований к системе;
- реализация базовых функций программного средства;
- расширение перечня функций программного средства (реализация доработок);
- исправление найденных ошибок;
- адаптация программного продукта для функционирования в других условиях (переход на новую СУБД, новый язык программирования и пр.);
- оптимизация программного продукта (увеличение быстродействия, надежности и пр.)

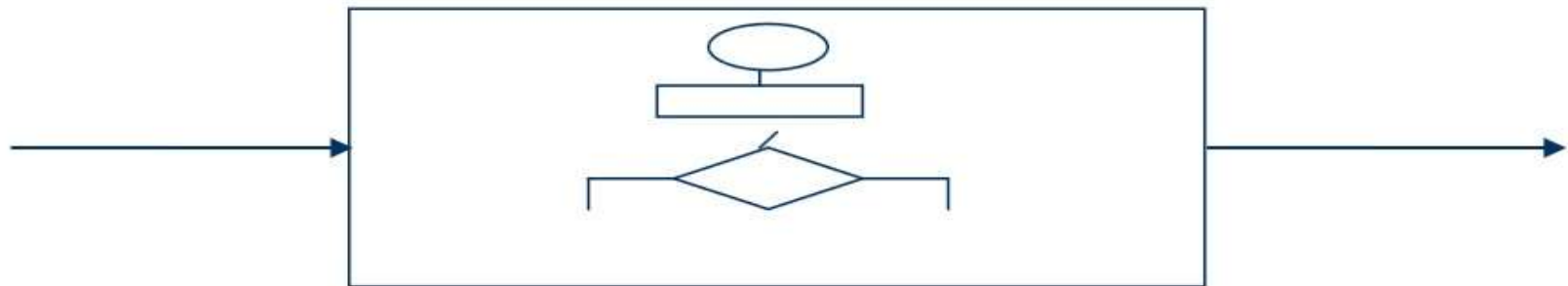
- **Отдел технической поддержки** (горячая линия). Осуществляет консультации пользователей по вопросам, связанным с установкой и эксплуатацией программного средства по различным каналам связи (телефон, почта, электронная почта).

Отдел тестирования. В задачи отдела входят:

- ☐ комплексный контроль качества;
- ☐ подготовка тестовой документации (планы тестирования и пр.);
- ☐ обнаружение и локализация ошибок в функционировании программных продуктов;
- ☐ фиксирование и отслеживание ошибок в функционировании программных средств;
- ☐ проверка соответствия документации программного продукта стандартам и реально реализованным функциям;
- ☐ участие в разработке и внедрении системы качества;
- ☐ автоматизация тестирования;
- ☐ оценка производительности разрабатываемых программных средств на различных программно-аппаратных платформах и их специфических конфигурациях

- **МЕТОДОЛОГИЯ ТЕСТИРОВАНИЯ**

В терминологии профессионалов тестирования (программного и некоторого аппаратного обеспечения), фразы «тестирование белого ящика» и «тестирование чёрного ящика» относятся к тому, имеет ли разработчик тестов доступ к исходному коду тестируемого программного обеспечения, или же тестирование выполняется через пользовательский интерфейс либо прикладной программный интерфейс, предоставленный тестируемым модулем. При тестировании методом белого ящика (white-box testing, также говорят — прозрачного ящика, оно же структурное тестирование), разработчик теста имеет доступ к исходному коду программ и может писать код, который связан с библиотеками тестируемого программного обеспечения. На рисунке 4.1 можно увидеть схематическое изображение данного метода.



- Тесты создаются на основе знаний о структуре самой системы и о том, как она работает.

Критерии полноты основаны на проценте элементов кода, которые отработали в ходе выполнения тестов. Для оценки степени соответствия требованиям могут привлекаться дополнительные знания о связи требований с определенными ограничениями на значения внутренних данных системы (например, на значения параметров вызовов, результатов и локальных переменных).

Тестировании методом чёрного ящика

тестировщик имеет доступ к программному обеспечению только через те же интерфейсы, что и заказчик или пользователь, либо через внешние интерфейсы, позволяющие другому компьютеру либо другому процессу подключиться к системе для тестирования.

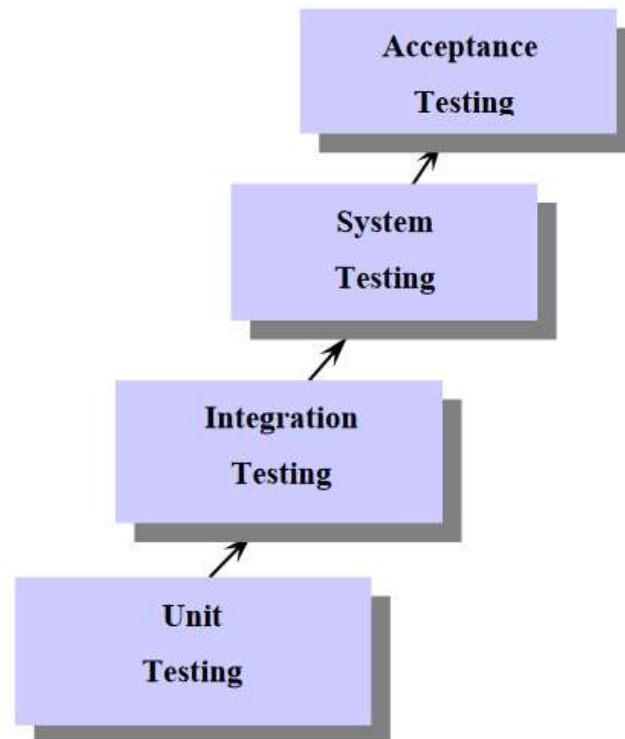
Тестирование черного ящика, нацеленное на проверку требований. Тесты для него и критерий полноты тестирования строятся на основе требований и ограничений, четко зафиксированных в спецификациях, стандартах, внутренних нормативных документах. Часто такое тестирование называется тестированием на соответствие (conformance testing).

Частным случаем его является *функциональное тестирование* — тесты для него, а также используемые критерии полноты проведенного тестирования определяют на основе требований к функциональности.

Помимо методов тестирования «белый ящик» и «черный ящик» различают тестирование методом **«серого ящика»**,

В данном случае у человека, который разрабатывает тест кейсы, есть некоторая информация о внутренней структуре приложения или о деталях реализации. Данный метод применяется в последнее время чаще предыдущих

УРОВНИ ТЕСТИРОВАНИЯ



• УРОВНИ ТЕСТИРОВАНИЯ

- Существует классификация видов тестирования, которая основана на том уровне детализации работ проекта, на который оно нацелено.
- Модульное тестирование (Unit-testing) — уровень тестирования, на котором тестируется минимально возможный для тестирования компонент, например, отдельный класс или функция.
На этом уровне применяются методы «белого ящика».
- В современных проектах **модульное тестирование** («юнит-тестинг») осуществляется разработчиками .
Модульное тестирование предназначено для проверки правильности отдельных модулей, независимо от их окружения. При этом проверяется, что если модуль получает на вход данные, удовлетворяющие определенным критериям корректности, то и результаты его корректны.

- Для описания критериев корректности входных и выходных данных часто используют программные контракты — предусловия, описывающие для каждой операции, на каких входных данных она предназначена работать, постусловия, описывающие для каждой операции, как должны соотноситься входные данные с возвращаемыми ею результатами, и инварианты, определяющие критерии целостности внутренних данных модуля.
- **Основной недостаток модульного тестирования** состоит в том, что проводить его можно, только когда проверяемый элемент программы уже разработан. Снизить влияние этого ограничения можно, подготавливая тесты (а это — наиболее трудоемкая часть тестирования) на основе требований заранее, когда исходного кода еще нет.

Модульное тестирование является важной составной частью отладочного тестирования, выполняемого разработчиками для отладки написанного ими кода.

- **Интеграционное тестирование** (Integration testing) – уровень тестирования, на котором отдельные программные модули объединяются и тестируются в группе. Обычно интеграционное тестирование проводится после модульного тестирования (юнит-тесты для модулей должны быть выполнены и найденные ошибки исправлены)
- Интеграционное тестирование в качестве входных данных использует модули, над которыми было проведено модульное тестирование, группирует их в более крупные множества, выполняет тесты, определённые в плане тестирования для этих множеств, и представляет их в качестве выходных данных и входных для последующего системного тестирования. При этом проверяется, что в ходе совместной работы модули обмениваются данными и вызовами операций, не нарушая взаимных ограничений на такое взаимодействие, например, предусловий вызываемых операций.

Интеграционное тестирование выполняется разработчиками используется при отладке, но на более позднем этапе разработки

- **Системное тестирование** (System testing) - это тестирование программного обеспечения, выполняемое на полной, интегрированной системе, с целью проверки соответствия системы исходным требованиям.
- Системное тестирование относится к методам тестирования «чёрного ящика», и, тем самым, не требует знаний о внутреннем устройстве системы.
Системное тестирование выполняется через внешние интерфейсы программного обеспечения и тесно связано с тестированием пользовательского интерфейса (или через пользовательский интерфейс), проводимым при помощи имитации действий пользователей над элементами этого интерфейса.
- Частными случаями этого вида тестирования являются тестирование графического пользовательского интерфейса (Graphical User Interface, GUI) и пользовательского интерфейса Web-приложений (WebUI).
Системное тестирование выполняется инженерами по тестированию

- **Приемочное тестирование** (Acceptance testing) - это тестирование готового продукта конечными пользователями на реальном окружении, в котором будет функционировать тестируемое приложение. Приемочные тесты разрабатываются пользователями, обычно, в виде сценариев. Для того, чтобы найти больше ошибок рекомендуется планировать не только системное тестирование и приемочное, но и модульное и интеграционное

- **Статическое тестирование** (Static testing) – тестирование, в ходе которого тестируемая программа (код) не выполняется (запускается). Анализ программы происходит на основе исходного кода, который вычитывается вручную, либо анализируется специальными инструментами.

Примеры статического тестирования:

- обзоры (Reviews);
- инспекции (Inspections);
- сквозные просмотры (Walkthroughs);
- аудиты (Audits)

- **Динамическое тестирование** (Dynamic testing) – тестирование, в ходе которого тестируемая программа (код) выполняется (запускается).
- **Альфа-тестирование** — тестирование в процессе разработки.
Бета-тестирование — тестирование выполняется пользователями (end-users).

Перед тем, как выпускается программное обеспечение, как минимум, оно должно проходить стадии альфа (внутреннее пробное использование) и бета (пробное использование с привлечением отобранных внешних пользователей) версий. Отчеты об ошибках, поступающие от пользователей этих версий продукта, обрабатываются в соответствии с определенными процедурами, включающими подтверждающие тесты (любого уровня), проводимые специалистами группы разработки. Данный вид тестирования не может быть заранее спланирован.

- **Регрессионное тестирование (Regression testing)** – тестирование функциональности, которая была уже протестирована до внесения какого-либо изменения . После внесения изменений в очередную версию программы, регрессионные тесты подтверждают, что сделанные изменения не повлияли на работоспособность остальной функциональности приложения. Регрессионное тестирование может выполняться как вручную, так и средствами автоматизации тестирования.
- Определение успешности регрессионных тестов (IEEE 610-90 “Standard Glossary of Software Engineering Terminology”) гласит: “повторное выборочное тестирование системы или компонент для проверки сделанных модификаций не должно приводить к непредусмотренным эффектам”.
- На практике это означает, что если система успешно проходила тесты до внесения модификаций, она должна их проходить и после внесения таковых. Основная проблема регрессионного тестирования заключается в поиске компромисса между имеющимися ресурсами и необходимостью проведения таких тестов по мере внесения каждого изменения. В определенной степени, задача состоит в том, чтобы определить критерии “масштабов” изменений, с достижением которых необходимо проводить регрессионные тесты.

- **«Смок-тест»** (Smoke Testing, «дымовое тестирование») в тестировании означает минимальный набор тестов на явные ошибки. Дымовой тест обычно выполняется самим программистом; не проходящую этот тест программу не имеет смысла отдавать на более глубокое тестирование

• ВИДЫ ТЕСТИРОВАНИЯ

- **Функциональное тестирование** (Functional testing) - цель данного тестирования состоит в том, чтобы убедиться в надлежащем функционировании объекта тестирования:
- каждое функциональное требование транслируется в тест-кейсы (используя техники «черного ящика») для того, чтобы проверить, что система функционирует в точности,
- как и описано в спецификации (функциональных требованиях к системе);
проверятся, все ли функциональные требования действительно
запрограммированы/реализованы.

- **Тестирование производительности** (Performance testing) - тестирование, которое проводится с целью определения, как быстро работает система или её часть под определённой нагрузкой.
- Также может служить для проверки и подтверждения других атрибутов качества системы, таких как масштабируемость, надёжность и потребление ресурсов.
- Существует особый подвид таких тестов, когда делается попытка достижения количественных пределов, обусловленных характеристиками самой системы и ее операционного окружения:
 - продемонстрировать, что система удовлетворяет критериям производительности при заданных условиях;
 - измерить, какая часть системы является причиной «плохой» производительности системы;
 - измерить время реакции на действие пользователя, время отклика на запрос, и т.д

- *Стрессовое тестирование* (Stress testing) - обычно используется для понимания пределов пропускной способности приложения.
- Этот тип тестирования проводится для определения надёжности системы во время экстремальных или диспропорциональных нагрузок и отвечает на вопросы о достаточной производительности системы в случае, если текущая нагрузка сильно превысит ожидаемый максимум.

- *Нагрузочное тестирование* (Load testing) - это простейшая форма тестирования производительности.
- Нагрузочное тестирование обычно проводится для того, чтобы оценить поведение приложения под заданной ожидаемой нагрузкой.

Этой нагрузкой может быть, например, ожидаемое количество одновременно работающих пользователей приложения, совершающих заданное число транзакций за интервал времени.

Такой тип тестирования обычно позволяет получить время отклика всех самых важных бизнес-транзакций. В случае наблюдения за базой данных, сервером приложений, сетью и т. д., этот тип тестирования может также идентифицировать некоторые узкие места приложения.

- **Тестирование стабильности** (Stability testing) проводится с целью убедиться в том, что приложение выдерживает ожидаемую нагрузку в течение длительного времени. При проведении этого вида тестирования осуществляется наблюдение за потреблением приложением памяти, чтобы выявить потенциальные утечки.
- Такое тестирование выявляет деградацию производительности, выражающуюся в снижении скорости обработки информации и/или увеличением времени ответа приложения после продолжительной работы по сравнению с началом теста

- **Тестирование удобства использования (Usability testing)** - исследование, выполняемое с целью определения, удобен ли некоторый искусственный объект (такой как веб-страница, пользовательский интерфейс или устройство) для его предполагаемого применения.
- Таким образом, проверка эргономичности измеряет эргономичность объекта или системы.
- Проверка эргономичности сосредоточена на определённом объекте или небольшом наборе объектов, в то время как исследования взаимодействие человек-компьютер в целом — формулируют универсальные принципы .
- Это метод оценки удобства продукта в использовании, основанный на привлечении
- пользователей в качестве тестировщиков, испытателей и суммировании полученных от них ВЫВОДОВ.

- При испытании многих продуктов пользователю предлагают в «лабораторных» условиях решить основные задачи, для выполнения которых этот продукт разрабатывался, и просят высказывать во время выполнения этих тестов свои замечания.
- Процесс тестирования фиксируется в протоколе (логе) и/или на аудио- и видеоустройства — с целью последующего более детального анализа.
- Если юзабилити-тестирование выявляет какие-либо трудности (например сложности в понимании инструкций, выполнении действий или интерпретации ответов системы), то разработчики должны доработать продукт и повторить тестирование.
- Наблюдение за тем, как люди взаимодействуют с продуктом, нередко позволяет найти для него более оптимальные решения. Если при тестировании используется модератор, то его задача — держать респондента сфокусированным на задачах (но при этом не "помогать" ему решать эти задачи).

Основную трудность после проведения процедуры юзабилити-тестирования нередко

- ***представляют большие объёмы и беспорядочность полученных данных.***
- Поэтому для последующего анализа важно зафиксировать:
- 1) речь модератора и респондента;
- 2) выражение лица респондента (снимается на видеокамеру);
- 3) изображение экрана компьютера, с которым работает респондент;
- 4) различные события, происходящие на компьютере, связанные с действиями пользователя:
- перемещение курсора и нажатия на клавиши мыши;
- использование клавиатуры;
- переходы между экранами (браузера или другой программы)

- Все эти потоки данных должны быть синхронизированы по тайм-кодам, чтобы при анализе
- их можно было бы соотносить между собой.
- Наряду с модератором в тестировании нередко участвуют наблюдатели. По мере
- обнаружения проблем они делают свои заметки о ходе тестирования так, чтобы после можно было
- синхронизировать их с основной записью. В итоге каждый значимый фрагмент записи теста
- оказывается прокомментирован в заметках наблюдателя. В идеале ведущий (т.е. модератор)
- представляет разработчика, наблюдатели — заказчика (например издателя), а испытатели —
- конечного пользователя (например покупателя).

- Существует еще один подход к usability-тестированию: для решения задачи предложенной пользователю разрабатывается "идеальный" сценарий решения этой задачи. Как правило, это сценарий, на который ориентировался разработчик. При выполнении задачи пользователями регистрируются их отклонения от задуманного сценария для последующего анализа.
- После нескольких итераций доработки программы и последующего тестирования можно получить удовлетворительный интерфейс с точки зрения пользователя

- *Тестирование интерфейса пользователя* (UI testing) - тестирование графического
- интерфейса пользователя для того, чтобы убедиться, что он соответствует принятым стандартам и их требованиям.
- Проверяется, как приложение обрабатывает ввод с клавиатуры и «мышки» и как
- отображаются элементы графического интерфейса (текст, кнопки, меню, списки и прочие элементы).

- **Тестирование безопасности** (security testing) - оценка уязвимости программного обеспечения к различным атакам.

- Компьютерные системы очень часто являются мишенью незаконного проникновения.

Под проникновением понимается широкий диапазон действий:

- попытки хакеров проникнуть в систему из спортивного интереса,
- месть рассерженных служащих,
- взлом мошенниками для незаконной наживы. и т.д.
- Тестирование безопасности проверяет фактическую реакцию защитных механизмов, встроенных в систему, на проникновение.
- В ходе тестирования безопасности испытатель играет роль взломщика. Ему разрешено все:
- попытки узнать пароль с помощью внешних средств;
- атака системы с помощью специальных утилит, анализирующих защиты;
- подавление, ошеломление системы (в надежде, что она откажется обслуживать других клиентов);
- целенаправленное введение ошибок в надежде проникнуть в систему в ходе восстановления;
- просмотр несекретных данных в надежде найти ключ для входа в систему.

- *Тестирование локализации* (Localization testing) - многогранная вещь, подразумевающая проверку множества аспектов, связанных с адаптацией продукта для пользователей из других стран.
- Например, тестирование локализации для пользователей из Японии может заключаться в проверке того, не выдаст ли система ошибку, если этот пользователь на сайте знакомств введет рассказ о себе символами *Kanji*, а не английским шрифтом.

- *Тестирование совместимости* (Compatibility testing) — вид нефункционального
- тестирования, основной целью которого является проверка корректной работы продукта в
- определенном окружении. Окружение может включать в себя следующие элементы:
- аппаратная платформа;
- сетевые устройства;
- периферия (принтеры, CD/DVD-приводы, веб-камеры и пр.);
- операционная система (Unix, Windows, MacOS, ...);
- базы данных (Oracle, MS SQL, MySQL, ...);
- системное программное обеспечение (веб-сервер, файрвол, антивирус, ...);
- браузеры (Internet Explorer, Firefox, Opera, Chrome, Safari).

- **ПРОЦЕСС ТЕСТИРОВАНИЯ**

- *Этапы и задачи*

Тестирование — это проверка соответствия программного обеспечения требованиям, осуществляемая с помощью наблюдения за его работой в специальных, искусственно построенных ситуациях. Такого рода ситуации называют тестовыми или просто тестами .

Тестирование — конечная процедура. Набор ситуаций, в которых будет проверяться тестируемое программное обеспечение, всегда конечен. Более того, он должен быть настолько мал, чтобы тестирование можно было провести в рамках проекта разработки программного обеспечения, не слишком увеличивая его бюджет и сроки.

Это означает, что при тестировании всегда проверяется очень небольшая доля всех возможных ситуаций.

Тестирование может использоваться для достаточно уверенного вынесения оценок о качестве программного обеспечения.

Для этого необходимо иметь **критерии полноты** тестирования, описывающие важность различных ситуаций для оценки качества, а также эквивалентности и зависимости между ними. Этот критерий может утверждать, что все равно в какой из ситуаций, А или В, проверять правильность работы программного обеспечения, или, если программа правильно работает в ситуации А, то, скорее всего, в ситуации В все тоже будет правильно.

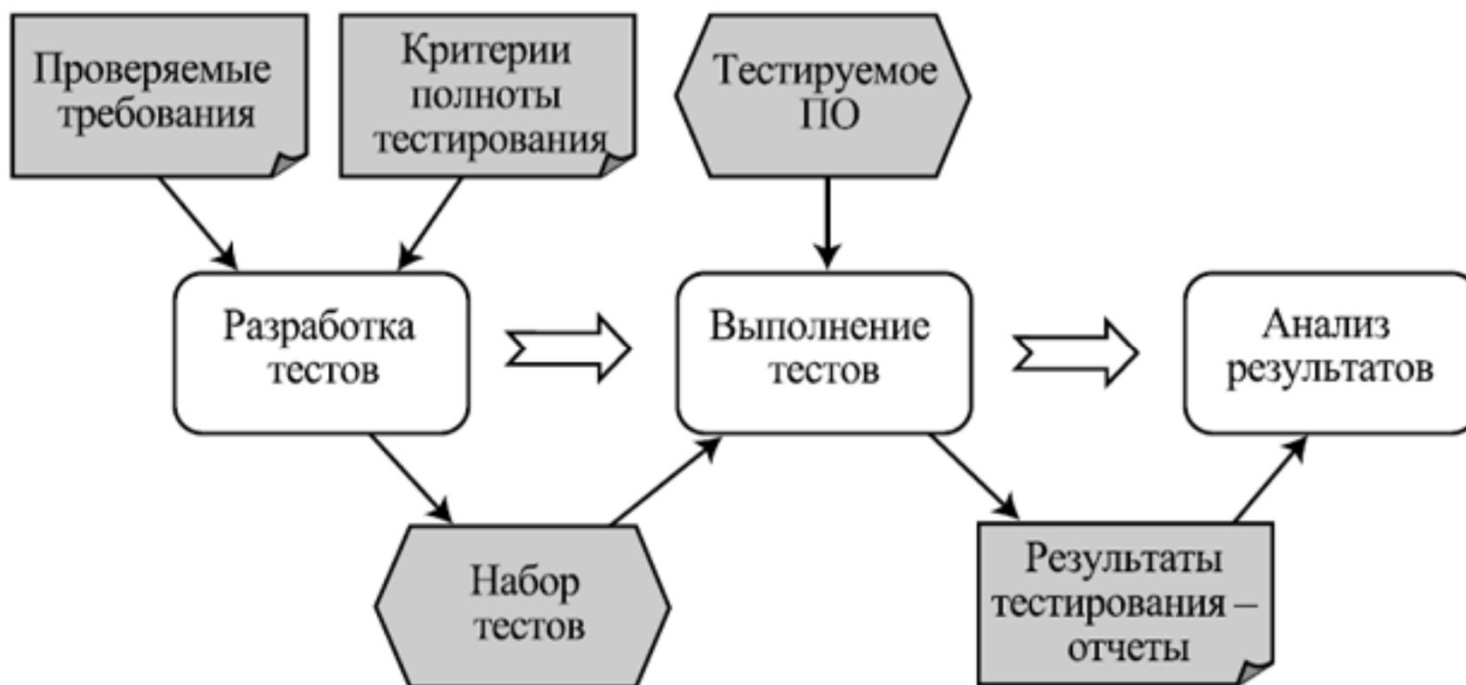
Часто критерий полноты тестирования задается при помощи определения эквивалентности ситуаций, дающей конечный набор классов ситуаций. В этом случае считают, что все равно, какую из ситуаций одного класса использовать в качестве теста. Такой критерий называют **критерием тестового покрытия**, а процент классов эквивалентности ситуаций, случившихся вовремя тестирования, — **достигнутым тестовым покрытием**.

Таким образом, основные задачи тестирования: построить такой набор ситуаций, который был бы достаточно представителен и позволял бы завершить тестирование с достаточной степенью уверенности в правильности программного обеспечения вообще, и убедиться, что в конкретной ситуации программное обеспечение работает правильно, в соответствии с требованиями.

Схема процесса тестирования.

Разработка тестов происходит на основе проверяемых требований и критерия полноты тестирования. Разработанные тесты формируются в тейс-кейс (набор тестов) и выполняем на ПО, которое нужно протестировать.

После прогона всех тестов анализируется результат, в результате чего можно выявить ошибки в программе.



Тестировать можно соблюдение любых требований, соответствие которым выявляется во время работы программного обеспечения.

Из характеристик качества по ISO 9126 этим свойством не обладают только атрибуты удобства сопровождения.

Поэтому выделяют виды тестирования, связанные с проверкой определенных характеристик и атрибутов качества — тестирование функциональности, надежности, удобства использования, переносимости и производительности, а также тестирование защищенности, функциональной пригодности и пр.

Кроме того, особо выделяют нагрузочное или стрессовое тестирование, проверяющее работоспособность программного обеспечения и показатели его производительности в условиях повышенных нагрузок — при большом количестве пользователей, интенсивном обмене данными с другими системами, большом объеме передаваемых или используемых данных и пр

- ***Планирование тестирования***

Вопросы, определяющие процесс планирования

Процесс тестирования находится в прямой зависимости от процесса разработки программного обеспечения, но при этом сильно отличается от него, поскольку преследует другие цели.

Разработка ориентирована на построение программного продукта, тогда как тестирование отвечает на вопрос, соответствует ли разрабатываемый программный продукт требованиям, в которых зафиксирован первоначальный замысел изделия (т.е. то, что заказал заказчик).

Вместе оба процесса охватывают виды деятельности, необходимые для получения качественного продукта. Ошибки могут быть привнесены на каждой стадии разработки.

Следовательно, каждому этапу разработки должен соответствовать этап тестирования.

Отношения между этими процессами таковы, что если что-то разрабатывается, то оно подвергается тестированию, а результаты тестирования используются для определения, соответствует ли это "что-то" набору предъявляемых требований.

Процесс тестирования возвращает выявленные им ошибки в процесс разработки.

Процесс разработки передает процессу тестирования новые и исправленные проектные версии.

Соответственно планирование тестирования тоже зависит от выбранной модели разработки.

Однако вне зависимости от модели разработки при планировании тестирования необходимо ответить на пять вопросов, определяющих этот процесс:

- **кто будет тестировать и на каких этапах?**

Разработчики продукта, независимая группа тестировщиков или совместно;

- **какие компоненты надо тестировать?** Будут ли подвергнуты тестированию все компоненты программного продукта или только компоненты, которые угрожают наибольшими потерями для всего проекта;

- **когда надо тестировать?** Будет ли это непрерывный процесс, вид деятельности, выполняемый в специальных контрольных точках, или вид деятельности, выполняемый на завершающей стадии разработки;

- **как надо тестировать?** Будет ли тестирование сосредоточено только на проверке того, что данный продукт должен выполнять, или также на том, как это реализовано;

- **в каком объеме тестировать?** Как определить, в достаточном ли объеме выполнено тестирование, или как распределить ограниченные ресурсы, выделенные под тестирование.

Все ответы на поставленные вопросы и многое другое фиксируется в документе – **Тест план.**

Тест план (Test Plan) - это документ, описывающий весь объем работ по тестированию,

начиная с описания объекта, стратегии, расписания, критериев начала и окончания тестирования,

до необходимого в процессе работы оборудования, специальных знаний, а также оценки рисков с вариантами их разрешения.

- ***Тест план содержит:***

- 1) перечень тестовых ресурсов;

- 2) перечень функций и подсистем, подлежащих тестированию;

3) тестовую стратегию:

- анализ функций и подсистем с целью определения слабых мест, требующих исчерпывающего тестирования, то есть участков функциональности, где появление дефектов наиболее вероятно;
- определение стратегии выбора входных данных для тестирования. Поскольку в реальных применениях множество входных данных программного продукта практически бесконечно, выбор конечного подмножества для проведения тестирования является сложной задачей. Для ее решения могут быть применены методы покрытия классов входных и выходных данных, анализ крайних значений, покрытие случаев использования и т.п. Выбранная стратегия должна быть обоснована и задокументирована;
- определение потребности автоматизации процесса тестирования. При этом решение об использовании существующей, либо о создании новой автоматизированной системы тестирования должно быть обосновано, а также продемонстрирована оценка затрат на создание новой системы или на внедрение уже существующей.

- 4) график (расписание) тестовых циклов;
- 5) указание конкретных параметров аппаратуры и программного окружения;
- 6) определение тестовых метрик, которые необходимо собирать и анализировать, таких
как покрытие набора требований, покрытие кода, количество и уровень серьезности дефектов, объем тестового кода и т.п.

Тест план должен как минимум отвечать на следующие вопросы:

1) что надо тестировать?

2) Описание объекта тестирования: системы, приложения, оборудование;

2) что будете тестировать?

Список функций и описание тестируемой системы и её компонент в отдельности;

3) как будете тестировать?

стратегия тестирования, а именно: методологии, виды тестирования и их применение по отношению к тестируемому объекту, приоритеты, автоматизация и пр.;

4) когда будете тестировать?

Последовательность проведения работ: фазы, циклы тестирования, процедура тестирования - подготовка (Test Preparation), тестирование (Testing), анализ результатов (Test Result Analysis) в разрезе запланированных фаз разработки;

5) где будете тестировать?

окружение тестируемой системы – описание;

необходимое для тестирования оборудование и программные средства.

6) кто будет тестировать?

роли и обязанности;

имена и фамилии.

7) критерии начала тестирования:

- готовность окружения;
 - готовность тест кейсов;
 - законченность разработки требуемого функционала;
 - выполнение юнит-тестов;
- билд построен и удовлетворяет определенным критериям.

8) критерии окончания тестирования:

- результаты тестирования удовлетворяют определенным критериям;
- требования к количеству открытых багов выполнены (определяются заранее).

9) критерии передачи системы для приемочного тестирования:

- приемочные тесты – где хранятся и когда выполняются;
- процедура приемки.

10) какие риски существуют и как мы ими будем управлять? Риски и их разрешение

11) метрики и отчеты:

- продуктовые метрики – кто собирает, с какой периодичностью;
- отчеты - кто создает, кому отправляет, и т.п.

12) расписание билдов.

Ответив в тест плане на вышеперечисленные вопросы, можно считать, что у на руках есть хороший черновик документа по планированию тестирования.

Далее, чтобы документ приобрел более менее серьезный вид, предлагается дополнить его следующими пунктами:

- окружение тестируемой системы (описание программно-аппаратных средств);
- необходимое для тестирования оборудование и программные средства (тестовый стенд и его конфигурация, программы для автоматизированного тестирования и т.д.);
- риски и пути их разрешения.

Виды тест планов

Чаще всего на практике приходится сталкиваться со следующими видами тест планов:

- 1) мастер тест план (Master Plan or Master Test Plan);**
- 2) тест план (Test Plan);**
- 3) план приемочных испытаний (Product Acceptance Plan)** - документ, описывающий набор действий, связанных с приемочным тестированием: стратегия, дата проведения, ответственные и т.д.;
- 4) план автоматизации (Test Automation Plan)** - документ, описывающий набор действий, связанных с автоматизацией тестированием: стратегия, правила, ответственные и т.д.

Явное отличие Master Test Plan от просто Test Plan в том, что **мастер тест план является более статичным** в силу того, что содержит в себе высокоуровневую информацию, которая не подвержена частому изменению в процессе тестирования и пересмотра требований. Сам же **детальный тест план**, который содержит более конкретную информацию по стратегии, видам

тестировании, расписанию выполнения работ, является "живым" документом, который постоянно

претерпевает изменения, отражающие реальное положение вещей на проекте.

В повседневной жизни на проекте может быть один Мастер Тест План и несколько детальных

тест планов, описывающих отдельные модули одного приложения

Тестовый случай (Test case). Виды. Структура.

Тестовый случай (Test Case) – это

- минимальный элемент тестирования (всего одна верификация\проверка);
- совокупность шагов, конкретных условий и параметров, необходимых для проверки реализации тестируемой функции или её части;
- описание определенных действий и условий, которые необходимы для того, чтобы выявить тот или иной баг.

Под тест кейсом понимается структура вида: **Action > Expected Result > Test Result.**

- пример test case

Action	Expecred Result	Test Result (passed/failed/blocked)
Open page "Login"	Login page is opened	Passed

Виды Тестовых Случаев

Тест кейсы разделяются по ожидаемому результату на позитивные и негативные:

- **позитивный тест кейс** использует только корректные данные и проверяет, что приложение правильно выполнило вызываемую функцию;
- **негативный тест кейс** оперирует как корректными так и некорректными данными (минимум 1 некорректный параметр) и ставит целью проверку исключительных ситуаций (срабатывание валидаторов), а также проверяет, что вызываемая приложением функция не выполняется при срабатывании валидатора.

Структура Тестовых Случаев (Test Case Structure)

Каждый тест кейс должен состоять из трех частей

Pre conditions	Список действий, которые приводят систему к состоянию пригодному для проведения основной проверки. Либо список условий, выполнение которых говорит о том, что система находится в пригодном для проведения основного теста состоянии.
Test case description	Список действий, переводящих систему из одного состояния в другое, для получения результата, на основании которого можно сделать вывод о удовлетворении реализации, поставленным требованиям
Post conditions	Список действий, переводящих систему в первоначальное состояние (состояние до проведения теста - initial state)

Примечание: Post Conditions не является обязательной частью. Эта часть актуальна при автоматизированном тестировании, когда за один прогон можно наполнить базу данных сотней или даже тысячей некорректных документов.

- **Пример тест кейса**

- do A1, verify B1
- do A2, verify B2
- do A3, verify B3
- В приведенном примере конечная проверка - B3. Это значит, что именно она является
- ключевой. Значит, A1 и A2 - это действия приводящие систему в тестопригодное состояние. А B1
- и B2 - условия того, что система находится в состоянии пригодном для тестирования

Action	Expected Result	Test Result (passed/failed/blocked)
Preconditions		
do A1	verify B1	
do A2	verify B2	
Test Case Description		
do A3	verify B3	
Postconditions		

- конечная проверка одна, т.е. в случае если тест провален (**test failed**) будет сразу ясно из-за чего. Т.к. если провальными окажутся проверки B1 и/или B2, то тест кейс будет заблокирован (**test blocked**), из-за того, что функцию не возможно привести в тестопригодное состояние (состояние пригодное для проведения тестирования), но это не значит, что тестируемая функция не работает.

Action	Expected Result	Test Result (passed/failed/blocked)
Preconditions		
do A1	verify B1	passed
do A2	verify B2	<i>failed</i>
Test Case Description		
do A3	verify B3	blocked
Postconditions		

- *Детализация описания тест кейсов*
- Уровень детализации тест кейсов должен быть таков, чтобы обеспечивать разумное
- соотношение времени прохождения к тестовому покрытию. Т.е. до тех пор пока покрытие тестами
- определенного функционала не меняется, можно уменьшать детализацию тест кейсов

Проверка отображения страницы		
Действие	Ожидаемый результат	Результат теста
Открыть страницу "Вход в систему"	- окно "Вход в систему" открыто; - название окна - Вход в систему; - логотип компании отображается в правом верхнем углу; - на форме 2 поля - Имя и Пароль; - кнопка Вход доступна; - ссылка "забыл пароль" - доступна.	...

Проверка: Проверьте, что отображаемая страница соответствует странице на картинке 1 (и прилагаем изображение страницы "Вход в систему")

Атрибут тест кейса	Описание
Test Case ID	Уникальное значение в пределах не только документа, но и всей фирмы □
Test Case Priority	Приоритет. Измеряется от 1 до n 1 – самый высокий n – самый низкий (для не очень больших проектов рационально выбирать n=4) □
IDEA	Описание идеи, проверяемой тест кейсом
SETUP and ADDITIONAL INFO	Входные параметры □
Revision History	История редактирования. Пример формата: <i>Created on <date> by<name></i> <i>Modified on<date>by<name></i> <i>Change</i> – какие изменения и зачем они
Execution Part □	Выполнимая часть тест кейса. Пример формата: Action – список команд EXPECTED RESULT – ожидаемый результат TEST RESULT – (passed, failed, blocked)

Пример тест кейса: пусть для тестирования возможности оплаты услуги на сайте необходимо выполнить следующий алгоритм:

- 1) открыть bsuir.by;
- 2) ввести в поле “Имя пользователя” значение “user”;
- 3) ввести в поле “Пароль” значение “password”;
- 4) нажать кнопку “Войти”;
- 5) ввести в поле “Искать людей” значение “Петр Петров”;
- 6) нажать кнопку “Поиск”;
- 7) нажать на страницу с Петр Петров;
- 8) в открывшейся странице нажать на ссылку “Отправить ”;
- 9) в открывшейся странице нажать на любую ссылку с подарком;
- 10) в открывшейся странице нажать на кнопку “Подарить”;
- 11) в открывшейся странице нажать на ссылку “Банковские карты”;
- 12) ввести в поле “Номер карты” значение карты VISA “1111-1111-1111-1111”;
- 13) ввести в поле “Действительно до” значение “07/15”;
- 14) ввести в поле “CVC” значение “111”;
- 15) нажать кнопку “Оплатить”;
- 16) записать номер заказа;
- 17) запросить базу данных “select res from payment where id = <номер заказа>”.

Ожидаемый результат: “10”

- Преимущество такой структуры тест кейса в отличие от первоначального списка заключается в
- том, что есть возможность протестировать по тому же сценарию другие данные (например: user2,
- password2, Джулия Робертс, 2222-2222-2222-2222).
- *Для выполнения одного и того же сценария на N наборах тестовых данных создается N*
- *тестов.*
- Этому правилу необходимо следовать. Таким образом, чтобы сделать подарок Ивану Иванову,
- нужно скопировать содержимое тест кейса VV12345, например, в тест кейс VV12346 и переписать
- только входные параметры.
- Такой вид тест кейса называется управляемый данными (data-driven).
- Проблемы сценария в примере:
- ☐ пункты 1-4 могут меняться в связи с миграцией сайта на новый домен, изменением
- интерфейса и т.д.;
- ☐ поиск друга в пунктах 5-7 “Петр Петров” может привести в никуда при удалении
- страницы;
- ☐ пункты 8-15 могут быть легко изменены за счет нового дизайна сайта.
- Следовательно, при любом таком изменении придется переписывать весь тест кейс, чтобы
- заново протестировать первоначальную задачу.

Test Case ID/Priority	VV12345	1
IDEA: Оплата картой VISA “подарка другу”		
SETUP and ADDITIONAL INFO:		
Аккаунт: user/password		
Имя друга: Петр Петров		
Номер карты: 1111-1111-1111-1111		
Срок действия: 07/15		
CVC: 111		
Запрос SQL: select res from payment where id = <номер заказа> ☐		
Revision History		
Created on 23/01/2014 by Иван Иванов	Новый тест кейс	
Execution Part		
ACTION	EXPECTED RESULT	TEST RESULT
1) открыть vk.com; 2) ввести в поле “Имя пользователя” значение “user”; 3) ввести в поле “Пароль” значение “password”; 4) нажать кнопку “Войти”; 5) ввести в поле “Искать людей” значение “Петр Петров”; 6) нажать кнопку “Поиск”; 7) нажать на страницу с Петр Петров; 8) в открывшейся странице нажать на ссылку “Отправить подарок”; 9) в открывшейся странице нажать на любую ссылку с подарком; 10) в открывшейся странице нажать на кнопку “Подарить”; 11) в открывшейся странице нажать на ссылку “Банковские карты”; 12) ввести в поле “Номер карты” значение карты VISA “1111-1111-1111-1111”; 13) ввести в поле “Действительно до” значение “07/15”; 14) ввести в поле “CVC” значение “111”; 15) нажать кнопку “Оплатить”; 16) записать номер заказа; 17) запросить базу данных “select res from payment where id = <номер заказа>”.		
	10	

Если же разбить задачу на несколько тест кейсов, например, за пункты 1-4 будет отвечать тест кейс “Вход в систему”, за пункты 5-7 “Поиск друга”, 8-15 – “Оплата подарка” (на внутреннем уровне каждого из них возможно еще более детальное разделение

Возможен вариант, когда все, что нужно – это выполнение команды номер 5, при условии что другие команды выполнены заранее

Test Case ID/Priority	VV12347	1
IDEA: Оплата картой VISA “подарка другу” SETUP and ADDITIONAL INFO: Аккаунт: user/password Имя друга: Петр Петров Номер карты: 1111-1111-1111-1111 Срок действия: 07/15 CVC: 111 Запрос SQL: select res from payment where id = <номер заказа> □		
Revision History		
Created on 23/01/2014 by Марина Гончарова	Новый тест кейс	
Modified on 23/01/2014 by Иванов Евгений	Упрощение шагов	
Execution Part		
ACTION	EXPECTED RESULT	TEST RESULT
1) войти в систему; 2) найти друга; 3) платить подарок; 4) записать номер заказа; 5) запросить базу данных “select res from payment where id = <номер заказа>”.	10	

Упрощение тест кейса

Test Case ID/Priority	VV12348	1
IDEA: Подтверждение оплаты SETUP and ADDITIONAL INFO: Номер заказа: параметр		
Revision History		
Created on 23/01/2014 by Марина Гончарова	Новый тест кейс	
Modified on 23/01/2014 by Иванов Евгений	Упрощение шагов	
Modified on 24/01/2014 by Сергей Галкин	Изменение структуры	
Execution Part		
ACTION	EXPECTED RESULT	TEST RESULT
Запросить базу данных "select res from payment where id = <номер заказа>."	10	

Неизвестным параметр < Номер заказа > после завершения выполнения теста получит свое значение, которое будет задокументировано [11].

- **ДЕФЕКТЫ. ПРИЧИНЫ, ОПИСАНИЕ, ОТСЛЕЖИВАНИЕ**

- **defect** — самое общее нарушение каких-либо требований или ожиданий, не обязательно проявляющееся вовсе (к дефектам относятся нарушения стандартов кодирования, недостаточная гибкость системы и пр.);
- **failure** — наблюдаемое нарушение требований, проявляющееся при каком-то реальном сценарии работы ПО. Это можно назвать проявлением ошибки;
- **fault** — ошибка в коде программы, вызывающая нарушения требований при работе (failures), то место, которое надо исправить. Хотя это понятие используется довольно часто, оно, вообще говоря, не вполне четкое, поскольку для устранения нарушения можно исправить программу в нескольких местах. Что именно надо исправлять, зависит от дополнительных условий, выполнение которых хотим при этом обеспечить, хотя в некоторых ситуациях наложение дополнительных ограничений не устраняет неоднозначность;
- **error** — используется в двух смыслах. *Первый смысл* — это ошибка в ментальной модели программиста, ошибка в его рассуждениях о программе, которая заставляет его делать ошибки в коде (faults). Это, собственно, ошибка, которую сделал человек в своем понимании свойств программы.

Второй смысл — это некорректные значения

- данных (выходных или внутренних), которые возникают при ошибках в работе
- программы.

Эти понятия некоторым образом связаны с основными источниками ошибок. Поскольку при разработке программ необходимо сначала понять задачу, затем придумать ее решение и закодировать его в виде программы, то, соответственно, основных источников ошибок три:

- **неправильное понимание задач.** Разработчики ПО не всегда понимают, что именно нужно сделать. Другим источником непонимания служит отсутствие его у самих пользователей и заказчиков — достаточно часто они могут просить сделать несколько не то, что им действительно нужно. Для предотвращения неправильного понимания задач программной системы служит анализ предметной области;
- **неправильное решение задач.** Зачастую, даже правильно поняв, что именно нужно сделать, разработчики выбирают неправильный подход к тому, как это делать. Выбираемые решения могут обеспечивать лишь некоторые из требуемых свойств, они могут хорошо подходить для данной задачи в теории, но плохо работать на практике, в конкретных обстоятельствах, в которых должно будет работать программное обеспечение. Помочь в выборе правильного решения может сопоставление альтернативных решений и тщательный анализ их на предмет соответствия всем требованиям, поддержание постоянной связи с пользователями и заказчиками, предоставление им необходимой информации о выбранных решениях, демонстрация прототипов, анализ пригодности выбираемых решений для работы в том контексте, в котором они будут использоваться;
- **неправильный перенос решений в код.** Имея правильное решение правильно понятой задачи, люди, тем не менее, способны сделать достаточно много ошибок при воплощении этих решений. Корректному представлению решений в коде могут помешать как обычные опечатки, так и забывчивость программиста или его нежелание
- отказаться от привычных приемов, которые не дают возможности аккуратно записать принятое решение. С ошибками такого рода можно справиться при помощи инспектирования кода, взаимного контроля, при котором разработчики внимательно читают код друг друга, опережающей разработки модульных тестов и тестирования.

Система отслеживания ошибок

- **Система отслеживания ошибок** (*bug tracking system*) — прикладная программа, разработанная с целью помочь разработчикам программного обеспечения (программистам, тестировщикам и др.) учитывать и контролировать ошибки (баги), найденные в программах, пожелания пользователей, а также следить за процессом устранения этих ошибок и выполнения или невыполнения пожеланий .

Главный компонент такой системы — база данных, содержащая сведения об обнаруженных дефектах. Эти сведения фиксируются в **отчете об ошибке**.

- **Отчет об ошибке (баг репорт)** - это документ, описывающий ситуацию или последовательность действий приведшую к некорректной работе объекта тестирования, с указанием причин и ожидаемого результата.

- **шаблона баг репорта**

Шапка	
Короткое описание (Summary)	Короткое описание проблемы, явно указывающее на причину и тип ошибочной ситуации.
Проект (Project)	Название тестируемого проекта
Компонент приложения (Component)	Название части или функции тестируемого продукта
Номер версии (Version)	Версия на которой была найдена ошибка
Серьезность (Severity)	Наиболее распространена пятиуровневая система градации серьезности дефекта: – s1 Блокирующий (Blocker); – s2 Критический (Critical); – s3 Значительный (Major); – s4 Незначительный (Minor); – s5 Тривиальный (Trivial).
Приоритет (Priority)	Приоритет дефекта: – p1 Высокий (High); – p2 Средний (Medium); – p3 Низкий (Low). (подробнее в разделе Серьезность и приоритет ошибки)
Статус (Status)	Статус бага. Зависит от используемой процедуры и жизненного цикла бага (bug workflow and life cycle)
Автор (Author)	Создатель баг репорта
Назначен на (Assigned To)	Имя сотрудника, назначенного на решение проблемы
Окружение	
ОС / Сервис Пак и т.д. / Браузера + версия / ...	Информация об окружении, на котором был найден баг: операционная система, сервис пак, для WEB тестирования - имя и версия браузера и т.д.

Описание	
Шаги воспроизведения (Steps to Reproduce)	Шаги, по которым можно легко воспроизвести ситуацию, приведшую к ошибке.
Фактический Результат (Result)	Результат, полученный после прохождения шагов к воспроизведению
Ожидаемый результат (Expected Result)	Ожидаемый правильный результат
Дополнения	
Прикрепленный файл (Attachment)	Файл с логами, скриншот или любой другой документ, который может помочь прояснить причину ошибки или указать на способ решения проблемы

- ***Серьезность и приоритет ошибки***

- Разные системы отслеживания ошибок предлагают разные пути описания серьезности и приоритета баг репорта, неизменным остается лишь смысл, вкладываемый в эти поля.
- **серьезность (Severity)** - это атрибут, характеризующий влияние ошибки на работоспособность приложения;
- **приоритет (Priority)** - это атрибут, указывающий на очередность выполнения задачи или устранения ошибки. Можно сказать, что это инструмент менеджера по планированию работ. Чем выше приоритет, тем быстрее нужно исправить дефект.

Градации серьезности дефекта (severity):

- **s1 Блокирующая (Blocker).** Блокирующая ошибка, приводящая приложение в нерабочее состояние, в результате которого дальнейшая работа с тестируемой системой или ее ключевыми функциями становится невозможна. Решение проблемы необходимо для дальнейшего функционирования системы;
- **s2 Критическая (Critical).** Критическая ошибка, неправильно работающая ключевая бизнес логика, дыра в системе безопасности, проблема, приведшая к временному падению сервера или приводящая в нерабочее состояние некоторую часть системы, без возможности решения проблемы, используя другие входные точки. Решение проблемы необходимо для дальнейшей работы с ключевыми функциями тестируемой системой;
- **s3 Значительная (Major).** Значительная ошибка, часть основной бизнес логики работает некорректно. Ошибка не критична или есть возможность для работы с тестируемой функцией, используя другие входные точки;

- **s4 Незначительная (Minor).** Незначительная ошибка, не нарушающая бизнес логику тестируемой части приложения, очевидная проблема пользовательского интерфейса.
- **s5 Тривиальная (Trivial).** Тривиальная ошибка, не касающаяся бизнес логики приложения, плохо воспроизводимая проблема, малозаметная посредством пользовательского интерфейса, проблема сторонних библиотек или сервисов, проблема, не оказывающая никакого влияния на общее качество продукта.

Градации приоритета ошибки (Priority):

- **p1 Высокий (High).** Ошибка должна быть исправлена как можно быстрее, т.к. ее наличие является критической для проекта;
- **p2 Средний (Medium).** Ошибка должна быть исправлена, ее наличие не является критичной, но требует обязательного решения;
- **p3 Низкий (Low).** Ошибка должна быть исправлена, ее наличие не является критичной, и не требует срочного решения.

Порядок исправления ошибок по их приоритетам: **High > Medium > Low.**

Баг репорт - это технический документ и в связи с этим язык описания проблемы должен быть техническим.

Должна использоваться правильная терминология при использовании названий элементов пользовательского интерфейса (editbox, lightbox, combobox, link, text area, button, menu, popup menu, title bar, system tray и т.д.), действий пользователя (click link, press the button, select menu item и т.д.) и полученных результатах (window is opened, error message is displayed, system crashed и т.д.).

Требования к обязательным полям баг репорта

Обязательными полями баг репорта являются: короткое описание (*Bug Summary*), серьезность (*Severity*), шаги к воспроизведению (*Steps to reproduce*), результат (*Actual Result*), ожидаемый результат (*Expected Result*). Ниже приведены требования и примеры по заполнению этих полей.

Короткое описание. В одном предложении нужно уместить смысл всего баг репорта, а именно: коротко и ясно, используя правильную терминологию сказать что и где не работает.

Например:

приложение зависает, при попытке сохранения текстового файла размером больше 50Мб;
данные на форме "Профайл" не сохраняются после нажатия кнопки "Сохранить".

- **Шаги к воспроизведению/Результат/Ожидаемый результат.** Очень важно четко описать все шаги, с упоминанием всех вводимых данных (имени пользователя, данных для заполнения формы) и промежуточных результатов.

Например:

1) войдите в системы: Пользователь Тестер1, пароль xxxXXX:

- вход в систему осуществлен.

2) кликните линк Профайл:

- страница Профайл открылась.

3) введите Новое имя пользователя: Тестер2;

4) нажмите кнопку Сохранить.

Результат. На экране появилась ошибка. Новое имя пользователя не было сохранено

Ожидаемый результат. Страница профайл перегружилась. Новое значение имени пользователя сохранено.

Основные ошибки при написании баг репортов

Недостаточность предоставленных данных. Не всегда одна и та же проблема проявляется при всех вводимых значениях и под любым вошедшим в систему пользователем, поэтому настоятельно рекомендуется вносить все необходимые данные в баг репорт.

Определение серьезности. Очень часто происходит либо завышение, либо занижение серьезности дефекта, что может привести к неправильной очередности при решении проблемы.

Язык описания. Часто при описании проблемы используются неправильная терминология или сложные речевые обороты, которые могут ввести в заблуждение человека, ответственного за решение проблемы.

Отсутствие ожидаемого результата. В случаях, если вы не указали, что же должно быть требуемым поведением системы, вы тратите время разработчика, на поиск данной информации, тем самым замедляете исправления дефекта. Вы должны указать пункт в требованиях, написанный тест кейс или же ваше личное мнение, если эта ситуация не была документирована.

Поле	Ответственный за заполнение поля
<i>Короткое описание (Summary)</i>	Автор баг репорта (обычно это Тестировщик)
Проект (Project)	Автор баг репорта (обычно это Тестировщик)
Компонент приложения (Component)	Автор баг репорта (обычно это Тестировщик)
Номер версии (Version)	Автор баг репорта (обычно это Тестировщик)
<i>Серьезность (Severity)</i>	Автор баг репорта (обычно это Тестировщик), однако данный атрибут может быть изменен вышестоящим менеджером
Приоритет (Priority)	Менеджер проекта или менеджер ответственный за разработку компонента, на который написан баг репорт
Статус (Status)	Автор баг репорта (обычно это Тестировщик), но многие системы баг трекинга выставляют статус по умолчанию
Автор (Author)	Устанавливается по умолчанию, если нет, то указывается имя автора баг репорта
Назначен на (Assigned To)	Менеджер проекта или менеджер ответственный за разработку компонента, на который написан баг репорт
ОС / Сервис Пак и т.д. / Браузера + версия / ...	Автор баг репорта (обычно это Тестировщик)
<i>Шаги воспроизведения (Steps to Reproduce)</i>	Автор баг репорта (обычно это Тестировщик)
<i>Фактический Результат (Result)</i>	Автор баг репорта (обычно это Тестировщик)
<i>Ожидаемый результат (Expected Result)</i>	Автор баг репорта (обычно это Тестировщик)
Прикрепленный файл (Attachment)	Автор баг репорта (обычно это Тестировщик), а также любой член командной группы, считающий, что прикрепленные данные помогут в исправлении бага

Жизненный цикл бага

Обнаружен (submitted). Тестировщик находит баг и представляет его на рассмотрение в систему отслеживания ошибок. С этого момента баг начинает свою официальную жизнь и о его существовании знают необходимые люди.

Назначен (assigned). Тестировщик или ведущий разработчик рассматривает баг и назначает его направление кому-то команды разработчиков.

Исправлен (fixed). Разработчик, которому было назначено исправление дефекта, исправляет его и сообщает о том, что задание выполнено

Проверен (verified). Тестировщик, который обнаружил ошибку проверяет на новом билде (в котором исправление данной ошибки заявлено), исправлен ли баг на самом деле. И только в том случае, если ошибка не проявится на новом билде, тестировщик меняет статус бага на *Verified*.

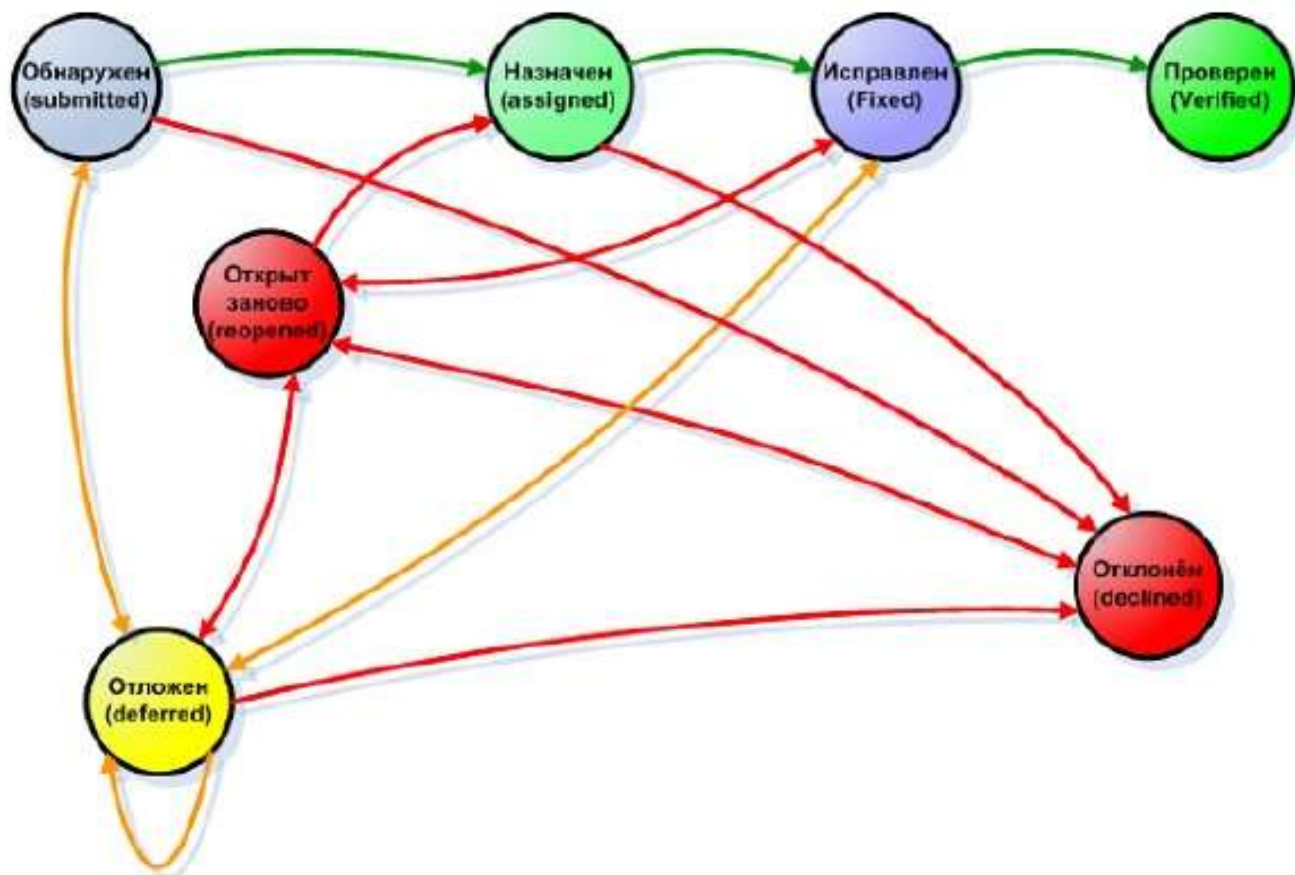
Открыт заново (reopened). Если баг проявляется на новом билде, тестировщик снова открывает этот дефект. Баг приобретает статус *Reopened*.

Отклонен (declined). Баг может быть отклонен. Во-первых, потому что для заказчика какие-то ошибки перестают быть актуальными. Во-вторых, это может случиться по вине тестировщика из-за плохого знания продукта, требований (дефекта на самом деле нет).

Отложен (deferred). Если исправление конкретного бага сейчас не очень важно или заказчик пока думает, или мы ждем какую-то информацию, от которой зависит исправление бага, тогда баг приобретает статус *Deferred*.

Закрытые (closed) баги. Закрытым считается баг в состояниях *Проверен (verified)* и *Отклонен (declined)*.

Открытые (open) баги. Открытыми являются баги в состояниях *Обнаружен (submitted)*, *Назначен (assigned)*, *Открыт заново (reopened)*. Иногда к открытым относят и баги в состояниях *Исправлен (fixed)* и *Отложен (deferred)*.



- **UML** – унифицированный язык моделирования (Unified Modeling Language) – это система обозначений, которую можно применять для объектно-ориентированного анализа и проектирования. Его можно использовать для визуализации, спецификации, конструирования и документирования программных систем. Словарь UML включает три вида строительных блоков:
 - Диаграммы.
 - Сущности.
 - Связи.

- **Сущности** – это абстракции, которые являются основными элементами модели, **связи** соединяют их между собой, а **диаграммы** группируют представляющие интерес наборы сущностей.

Диаграмма – это графическое представление набора элементов, чаще всего изображенного в виде связанного графа вершин (сущностей) и путей (связей). Язык UML включает **13** видов диаграмм, среди которых на первом месте в списке — диаграмма классов, о которой и пойдет речь.

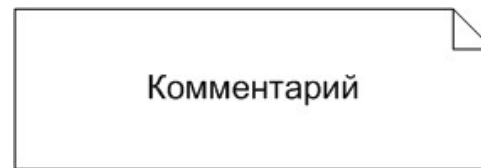
Диаграммы классов показывают набор классов, интерфейсов, а также их связи. Диаграммы этого вида чаще всего используются для моделирования объектно-ориентированных систем. Они предназначены для статического представления системы.

Большинство элементов UML имеют уникальную и прямую графическую нотацию, которая дает визуальное представление наиболее важных аспектов элемента.

- **Сущности**
- Диаграммы классов оперируют тремя видами сущностей UML:
- Структурные.
- Поведенческие.
- Аннотирующие.
- **Структурные сущности** – это «имена существительные» в модели UML. В основном, статические части модели, представляющие либо концептуальные, либо физические элементы. Основным видом структурной сущности в диаграммах классов является **класс**.
- **Поведенческие сущности** – динамические части моделей UML. Это «глаголы» моделей, представляющие поведение модели во времени и пространстве.
- Основной из них является **взаимодействие** – поведение, которое заключается в обмене сообщениями между наборами объектов или ролей в определенном контексте для достижения некоторой цели. Сообщение изображается в виде линии со стрелкой, почти всегда сопровождаемой именем операции.



- **Аннотирующие сущности** – это поясняющие части UML-моделей, иными словами, комментарии, которые можно применить для описания, выделения и пояснения любого элемента модели. Главная из аннотирующих сущностей – **примечание**. Это символ, служащий для описания ограничений и комментариев, относящихся к элементу либо набору элементов. Графически представлен прямоугольником с загнутым углом; внутри помещается текстовый или графический комментарий.



Структурные сущности — классы

Класс – это описание набора объектов с одинаковыми атрибутами, операциями, связями и семантикой.

Графически класс изображается в виде прямоугольника, разделенного на 3 блока горизонтальными линиями:

- имя класса
- атрибуты (свойства) класса
- операции (методы) класса.

Для атрибутов и операций может быть указан один из трех типов видимости:

- — — **private** (частный)
- # — **protected** (защищенный)
- + — **public** (общий)

Видимость для полей и методов указывается в виде левого символа в строке с именем соответствующего элемента.

Определение понятия архитектуры ПО

Архитектура программного обеспечения (software architecture) – это представление, которое даёт информацию о компонентах ПО, обязанностях отдельных компонентов и правилах организации связей между компонентами. Необходимость архитектуры обоснована сложностью программного обеспечения. Продуманная архитектура облегчает разработку и дальнейшее развитие ПО. Она служит базисом, каркасом создаваемой системы, интегрируя отдельные компоненты и создавая высокоуровневую модель системы.

Набор принципов, используемых в архитектуре, формирует *архитектурный стиль* (software architecture style). Применение архитектурного стиля сходно употреблению шаблона проектирования, но не на уровне компонента (модуля или класса), а на уровне всей создаваемой системы ПО. Как и шаблоны проектирования, архитектурные стили упрощают коммуникацию разработчиков и предлагают готовые решения целого класса абстрактных проблем. В таблице 1 представлено короткое описание основных архитектурных стилей.

Основные архитектурные стили

Архитектурный стиль	Описание
Клиент-серверная модель	Разделение системы на два приложения – клиент и сервер. При работе клиент посылает запросы на обслуживание серверу
Компонентная архитектура	Деление системы на компоненты, которые могут быть повторно использованы и не зависят друг от друга. Каждый компонент снабжается известным интерфейсом для коммуникаций
Многоуровневая архитектура	Разделение функций приложения на группы (уровни), которые организованы в виде стекового набора
Шина сообщений	Система, которая может посылать и передавать информационные сообщения в определённом формате по общему коммуникационному каналу. Благодаря этому организуется взаимодействие систем без указаний конкретных получателей сообщений
Многозвенная архитектура	Разделение функций подобно многоуровневой архитектуре, но группировка происходит не только на логическом, а и на физическом уровне – отдельным группам соответствует отдельный компьютер (сервер, кластер)
Объектно-ориентированная архитектура	Представление системы в виде набора взаимодействующих объектов
Выделенное представление	Выделение в системе отдельных групп функций для взаимодействия с пользователями и обработки данных
Архитектура, ориентированная на сервисы	Каждый компонент системы представлен в виде независимого сервиса, предоставляющего свои функции по стандартному протоколу

Клиент-серверная модель

Клиент-серверная модель (client-server model) описывает отношение между двумя компьютерными программами, в котором одна программа – *клиент* – выполняет запросы к другой программе – *серверу*. Эта модель решает, в основном, задачу развёртывания приложения. С использованием клиент-серверной модели созданы многие приложения для работы с базами данных, электронной почтой и для доступа к веб-ресурсам.

Перечислим основные принципы данного архитектурного стиля:

1. Клиент инициирует один или несколько запросов, ожидает ответа на них, А затем обрабатывает ответы.
2. В определённый момент времени клиент подключён к одному серверу для обработки запросов (реже – к небольшой группе серверов).
3. Клиент работает с пользователем напрямую, применяя графический интерфейс.
4. Сервер не инициирует запросов.
5. Обычно для выполнения запросов клиенты проходят аутентификацию на сервере.

Главными преимуществами клиент-серверной модели являются:

–**Высокая безопасность.** Все данные хранятся на сервере, обеспечивающем больший уровень безопасности, нежели отдельный клиент.

–**Централизованный доступ к данным.** Так как данные хранятся только на сервере, ими легко управлять (например, обеспечить обновление).

– **Устойчивость и лёгкость сопровождения.**

Роль сервера могут выполнять несколько физических компьютеров, объединённых в сеть. Благодаря этому клиент не замечает сбоев или замены отдельного серверного компьютера.

Рассмотрим некоторые варианты клиент-серверной модели. В системах *клиент-очередь-клиент* (client-queue-client или passive queue) сервер исполняет роль очереди для данных клиентов. То есть, клиенты используют сервер только для обмена данными между собой. *Пиринговые приложения* (peer-to-peer application) – это вариация системы клиент-очередь-клиент, в которой любой клиент может играть роль сервера. *Сервера приложений* (application server) служат для размещения и выполнения программ, которыми управляет клиент.

Компонентная архитектура

Ключевым понятием компонентной архитектуры является компонент

(component). Это программный объект, спроектированный так, чтобы удовлетворять следующим требованиям:

1. Компонент допускает повторное использование в различных системах.
2. Компонент не хранит информации, специфичной для конкретного ПО, в котором он используется.
3. Допускается создание новых компонентов на основе существующих.
4. Компонент имеет известный интерфейс для взаимодействия, но скрывает детали своей внутренней реализации.
5. Компоненты проектируются так, чтобы иметь минимальные зависимости от других компонентов.

Типичным примером компонентов являются элементы пользовательского интерфейса (элементы управления).

Компонентная архитектура сосредоточена на выделении отдельных компонентов и организации взаимодействия между ними. Этот стиль решает задачи структурирования приложений и обеспечивает следующие преимущества:

- **Лёгкость развёртывания.** Когда для компонента доступна новая версия, старая версия заменяется без влияния на остальные компоненты.
- **Уменьшение стоимости.** При разработке можно применять готовые компоненты сторонних производителей.
- **Повторное использование.** Одни и те же компоненты могут использоваться в нескольких приложениях.
- **Уменьшение технической сложности.** Обычно компоненты, составляющие одно приложение, развёрнуты в рамках одного программного контейнера. Этот контейнер управляет временем жизни компонентов, активацией компонентов, передачей сообщений между компонентами и так далее.

Многоуровневая архитектура

Многоуровневая архитектура (multilayered architecture) сосредоточена на иерархическом распределении отдельных частей системы при помощи эффективного разделения отношений. Каждая часть соотносится с определённым *уровнем* (layer), для каждого уровня заданы выполняемые им функции, уровни выстроены в стековую структуру (то есть находятся один поверх другого). Например, типичная многоуровневая архитектура веб-приложения включает уровень представления (компоненты пользовательского интерфейса), уровень бизнес-логики (обработка данных) и уровень доступа к данным. При этом уровень представления считается высшим, за ним идёт уровень бизнес-логики, а за уровнем бизнес-логики – уровень доступа к данным.

Сформулируем основные принципы многоуровневой архитектуры:

1. Проектирование чётко устанавливает разграничение функций между уровнями.
2. Нижние уровни независимы от верхних уровней.
3. Верхние уровни вызывают функции нижних уровней, но при этом взаимодействуют только соседние уровни иерархии.

Использование многоуровневой архитектуры обеспечивает следующие преимущества:

– **Изоляция.** Разработка и обновление ПО могут быть изолированы рамками одного уровня.

– **Производительность.** Распределение уровней на отдельные физические компьютеры повышает производительность и отказоустойчивость.

– **Тестируемость.** Уровни допускают независимое тестирование. Многоуровневая архитектура активно применяется при создании бизнес - приложений и сайтов, особенно приложений масштаба предприятия. При этом обычно используется следующий набор уровней (рис. 1):

Уровень представления (presentation layer) ответственен за взаимодействие с пользователем, ввод и вывод информации.

Бизнес-уровень или уровень бизнес-логики (business logic layer) обрабатывает информацию, реализуя конкретные бизнес-правила.

Уровень доступа к данным (data access layer) обеспечивает загрузку и сохранение информации, используя источник данных (файл, база данных) или внешний сервис.

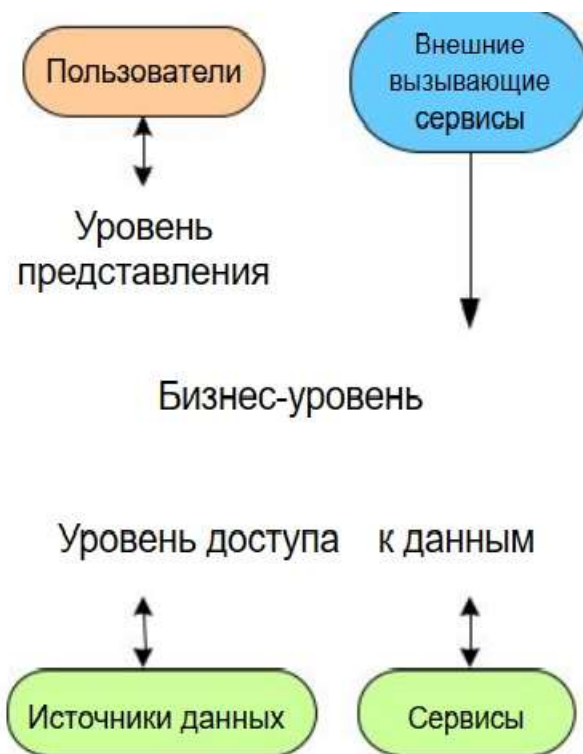


Рис. 1. Типичные уровни бизнес-приложения.

Для каждого уровня дополнительно можно выделить типичный набор компонентов

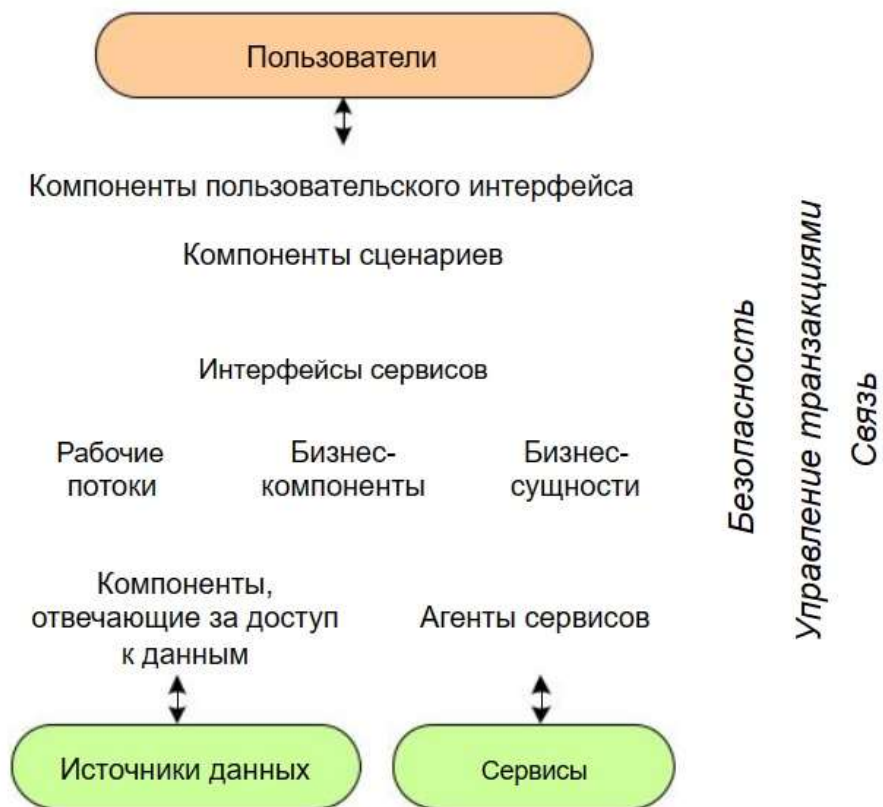


Рис. 2. Компоненты отдельных уровней.

1. Компоненты пользовательского интерфейса (UI Components).

Они предназначены для вывода информации и для ввода данных пользователями. Эти компоненты являются элементами оконного или веб-интерфейса.

2. Компоненты сценариев (UI Process Components). Обычно система подчиняется определённым правилам взаимодействия с ней. Например, в приложении для продажи товаров реализуется следующий сценарий: пользователь выбирает категорию товара из списка категорий, система показывает список товаров, затем пользователь выбирает товар, и система показывает данные по товару. Для того чтобы упорядочить такие сценарии и повторно их использовать, они объединяются в специальные объекты. Кроме этого, создаётся специальный механизм, который позволяет пользователю работать с системой только по определённым сценариям.

3. Рабочие потоки (Business Workflows). После получения данных от пользователя они должны быть использованы при выполнении бизнес-процессов (рабочих потоков). Бизнес-процессы состоят из шагов, которые должны быть выполнены в определённом порядке. Например, система должна подсчитать общую сумму заказа, проверить данные кредитной карты и организовать доставку товара. При этом заранее неизвестно, сколько времени потребуется на выполнение этих шагов. Поэтому нужен механизм управления этими операциями.

4. *Бизнес-компоненты* (Business Components). В этих компонентах реализуются бизнес-правила и решаются основные задачи. Другими словами, в них реализуется бизнес-логика приложения. Например, в системе продажи товаров нужно реализовать способ подсчёта общей стоимости покупки и назначить соответствующую плату за доставку.

5. *Агенты сервисов* (Service Gateways). Когда бизнес-компоненту понадобится функциональность внешнего сервиса, он должен обратиться к некоторому объекту, представляющему этот сервис в системе. На этих агентов часто ложится задача преобразования формата данных внешнего сервиса к формату вызывающего компонента. Например, бизнес-компоненты могут использовать одного агента сервиса для работы с сервисом проверки кредитных карт и другого агента для взаимодействия с сервисом обработки данных от курьера.

6. *Интерфейсы сервисов* (Service Interfaces). Чтобы сервисы могли воспользоваться функциональностью друг друга, а приложения – функциональностью сервисов, должен быть определён протокол обмена данными и сообщениями между сервисами и приложениями-потребителями. Этот протокол объявляется как интерфейс сервиса. Часто эти интерфейсы называют бизнес-фасадом.

7. *Компоненты, отвечающие за доступ к данным* (Data Access Components).

Программам нужно где-то хранить данные и в какой-то момент выполнения бизнес-процесса к ним обращаться. Имеет смысл абстрагироваться от конкретного вида данных конкретного приложения в пользу универсальных компонентов, представляющих данные. Эти компоненты размещаются в слое доступа к данным. Например, чтобы показать данные о товаре пользователю, приложению понадобится получить данные о товаре из БД. Для этого будет задействован слой доступа к данным и сама БД.

8. *Бизнес-сущности* (Business Entities). Приложению понадобится передавать данные между компонентами и уровнями. Например, список товаров должен быть передан из слоя доступа к данным в слой представления. Поэтому нужен способ представления в системе бизнес-сущностей из предметной области. Для этого могут использоваться готовые структуры данных или могут быть созданы специальные классы.

Шина сообщений

Архитектура, основанная на шине сообщений (message bus), подразумевает наличие общего коммуникационного канала, используя который компоненты обмениваются информацией. Компонент помещает сообщение в коммуникационный канал, после этого сообщение рассылается всем заинтересованным компонентам. Данная архитектура направлена на решение коммуникационных задач.

Принципы архитектуры с использованием шины сообщений следующие:

1. Все коммуникации между компонентами выполняются только при помощи информационных сообщений.
2. Сообщения имеют стандартный формат. Это позволяет интегрировать компоненты, разработанные на разных платформах.
3. Общая логика приложения изменяется путём удаления или добавления компонентов, подключённых к шине.
4. Как правило, коммуникация происходит в асинхронном режиме. Преимущества шины сообщений:

– **Расширяемость.** Компоненты, подключённые к шине, добавляются и удаляются без воздействия на другие подключённые компоненты.

– **Уменьшение сложности.** Для взаимодействий с системой компонент должен реализовать только логику работы с общей шиной.

– **Масштабируемость.** При увеличении нагрузки на один из подключённых компонентов достаточно добавить к шине копию этого компонента, которая будет обрабатывать часть сообщений. Вариациями стиля шина сообщений являются *шина сервисов масштаба предприятия* (enterprise service bus) и *шина интернет-сервисов* (internet service bus). В первом случае имеется в виду наличие средств для конвертирования сообщений, циркулирующих по шине, в различные форматы. Во втором случае шина объединяет компоненты, распределённые по всемирной сети. Это подразумевает идентификацию клиентов при помощи *универсального идентификатора ресурсов* (uniform resource identifier, URI) и особые политики безопасности.

Многозвенная архитектура

Многозвенная архитектура (multitier architecture) – это архитектурный стиль развёртывания приложений, подразумевающий разделение компонентов на функциональные группы, подобно тому, как это происходит в многоуровневой архитектуре. Группа (реже – несколько групп) формируют *звено* (tier) – часть приложения, которая физически обособлена, выполняется в отдельном процессе или на отдельном физическом компьютере.

Многозвенная архитектура характеризуется следующими принципами:

1. Это архитектурный стиль развёртывания многоуровневой архитектуры.
2. Звенья зависят только от своих непосредственных соседей. Звено n знает, как обрабатывать запросы от звена $n+1$, передавать запросы к звену $n-1$ и интерпретировать полученные результаты.
3. Уровень развёртывается в отдельное звено, если функциями этого уровня пользуются внешние приложения и сервисы. В противном случае размещение уровня в отдельном звене возможно, но не обязательно.

Преимущества многозвенной архитектуры:

- **Удобство сопровождения.** Физическая изоляция звеньев облегчает замену оборудования.
- **Масштабируемость.** При увеличении нагрузки на одно из звеньев возможно лёгкое увеличение количества оборудования в звене.
- **Увеличение работоспособности и доступность.** Этот показатель возрастает благодаря физической изолированности и независимости оборудования.

В качестве примера использования данного архитектурного стиля приведём типичное веб-приложение с повышенными требованиями к безопасности обрабатываемых данных. В таком веб-приложении компоненты бизнес-логики можно разместить на отдельном физическом сервере, который связан с веб-сервером по интрасети. Веб-сервер принимает запросы пользователей из внешней сети, перенаправляет их на обработку серверу с бизнес-логикой, обработанные данные представляет в виде веб-страниц. Если слой доступа к данным также размещается на отдельном компьютере, то мы получим достаточно распространённый вариант – *трёхзвенную архитектуру* (рис. 3).

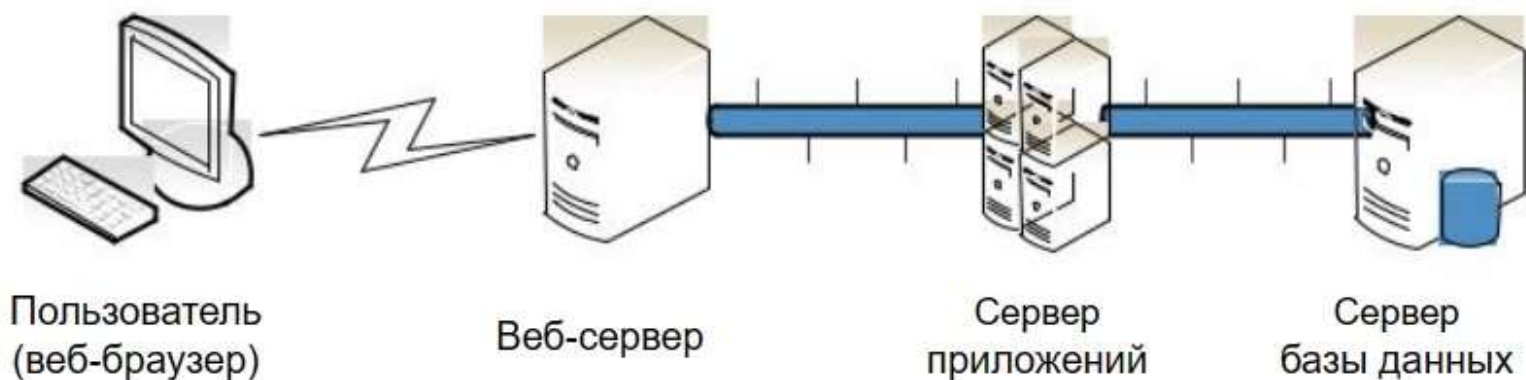


Рис. 3. Пример трёхзвенной архитектуры.

Объектно-ориентированная архитектура

При использовании объектно-ориентированной архитектуры (objectoriented architecture) система воспринимается как набор взаимодействующих объектов. Каждый такой объект содержит данные и необходимое поведение, а коммуникация между объектами происходит через открытые интерфейсы путём отправки и приёма сообщений.

Главные принципы объектно-ориентированной архитектуры, в целом, соответствуют принципам ООП:

- 1.Использование абстракций (базовые классы, интерфейсы) для сокрытия деталей конкретных реализаций.
- 2.Использование агрегирования – построение сложных объектов, включающих простые объекты.
- 3.Инкапсуляция.
- 4.Наследование.
- 5.Полиморфизм.
- 6.Уменьшение связанности объектов друг с другом благодаря применению интерфейсов и объектных фабрик.

Преимуществами объектно-ориентированной архитектуры являются высокая тестируемость систем, расширяемость, способность к повторному использованию.

Рассмотрим одну из вариаций объектно-ориентированной архитектуры, известную как *проектирование, основывающееся на домене* (*domain-driven design, DDD*). DDD используется для разработки уровня домена – одного из уровней в многоуровневой архитектуре приложений. Уровень домена описывает модель, с которой работает система, включая данные и бизнес-логику. При создании уровня домена DDD сосредотачивается на следующих объектах:

1. *Сущности* (entities). Основные объекты модели, обладающие уникальным идентификатором. Например, при разработке системы интернет-магазина сущностями могут быть покупатель, заказ, товар.
2. *Объекты-значения* (value objects). Объекты, которые представляют сгруппированный набор атрибутов. Например, адрес, состоящий из города, улицы, номера дома. Объекты-значения используются как части сущностей и не обладают уникальным идентификатором.
3. *Агрегаторы* (aggregates). Это особый вид сущностей, содержащий вложенные сущности. Например, заказ может содержать список заказываемых товаров. Агрегируемые сущности не имеют самостоятельного значения и должны быть доступны только через агрегатор.
4. *Хранилища* (repositories). Методы или классы, используемые для сохранения сущностей во внешних источниках данных и для получения сущностей из источников.
5. *Фабрики* (factories). Методы или классы, используемые для создания новых сущностей и агрегаторов.

Выделенное представление

Выделенное представление (separated presentation) – это стиль обработки запросов или действий пользователя, а также манипулирования элементами интерфейса и данными. Стиль подразумевает отделение элементов интерфейса от логики приложения.

Ключевыми принципами данного архитектурного стиля являются:

1. Выделение отдельных функций и ролей для задач обработки запросов, изменения данных и представления данных.

2. Для интеграции отдельных компонентов может использоваться событийная модель.

Основным преимуществом рассматриваемого стиля является улучшенная возможность организации тестирования отдельных компонентов системы.

В качестве примера использования выделенного представления рассмотрим шаблон *модель-представление-контроллер* (Model-View-Controller, MVC). Этот шаблон имеет три основных компонента (рис. 4):

– *Модель* представляет данные, с которыми работает приложение. Модель также реализовывает логику обработки данных согласно заданным бизнес -правилам и обеспечивает чтение и сохранение данных во внешних источниках.

- *Представление* обеспечивает способ отображение данных модели.
- *Контроллер* обрабатывает внешние запросы и координирует изменение модели и актуальность представления.

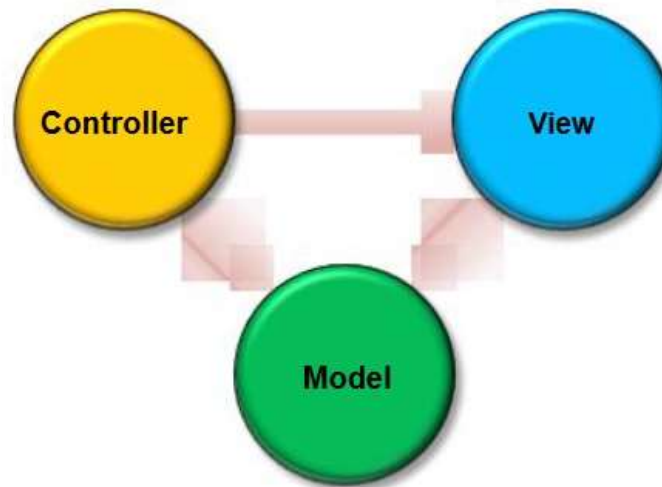


Рис. 4. Модель-представление-контроллер.

Важно отметить, что как представление, так и контроллер зависят от модели. Однако модель не зависит ни от представления, ни от контроллера. Это одно из ключевых достоинств подобного разделения. Оно позволяет построить модель независимо от визуального представления, а также создавать несколько различных представлений и контроллеров для одной модели.

Архитектура, ориентированная на сервисы

Архитектура, ориентированная на сервисы (service-oriented architecture, SOA), предоставляет требуемые функции в виде набора сервисов. Сервисы используют стандартные протоколы для вызова своих функций, публикации в сети и обнаружения. Отдельный сервис должен рассматриваться как независимое приложение, а не как компонент или объект. Основной задачей при использовании SOA является определение интерфейса сервиса и схемы передаваемых при вызове сервиса данных.

Базовые принципы SOA:

1. Сервисы автономны и независимы друг от друга.
2. Сервисы распределены в локальной или глобальной сети. Местоположение сервиса не важно – главное, чтобы сеть доступа к сервису поддерживала требуемые протоколы.
3. Сервисы публикуют контракты использования и схемы для данных обмена, но скрывают внутренние классы своей реализации.

Преимуществами архитектуры, ориентированной на сервисы, являются высокая абстракция и возможность использования сервисов в различных приложениях.

Шаблон проектирования MVC предполагает разделение данных приложения, пользовательского интерфейса и управляющей логики на три отдельных компонента:

***модель, представление и контроллер** – таким образом, что модификация каждого компонента может осуществляться независимо.*

Модель (Model) предоставляет данные предметной области представлению и реагирует на команды контроллера, изменяя свое состояние.

Представление (View) отвечает за отображение данных предметной области (модели) пользователю, реагируя на изменения модели.

Контроллер (Controller) интерпретирует действия пользователя, оповещая модель о необходимости изменений.

• Пользователь взаимодействует с моделью.

Представление — «окно», через которое пользователь воспринимает модель. Когда вы делаете что-то с представлением (скажем, щелкаете на кнопке воспроизведения), представление сообщает контроллеру, какая операция была выполнена. Контроллер должен обработать это действие.

• Контроллер обращается к модели с запросами об изменении состояния.

Контроллер получает действия пользователя и интерпретирует их. Если вы щелкаете на кнопке, контроллер должен разобраться, что это значит и какие операции с моделью должны быть выполнены при данном действии.

• Контроллер также может обратиться к представлению с запросом об изменении.

Когда контроллер получает действие от представления, в результате его обработки он может обратиться к представлению с запросом на изменение (скажем, заблокировать некоторые кнопки или команды меню).

• Модель оповещает представление об изменении состояния.

Когда в модели что-то изменяется (вследствие действий пользователя или других внутренних изменений — скажем, перехода к следующей песне в списке), модель оповещает представление об изменении состояния.

• Представление запрашивает у модели информацию состояния.

Представление получает отображаемую информацию состояния непосредственно от модели. Например, когда модель оповещает представление о начале воспроизведения новой песни, представление запрашивает название песни и отображает его. Представление также может запросить у модели информацию состояния в результате запроса на изменение состояния со стороны контроллера.

MVC с точки зрения шаблонов .

Стратегия

*Представление и контроллер реализуют классический шаблон **Стратегия**.*

Представление — объект со сменной стратегией, контроллер эту стратегию предоставляет. Представление интересует только визуальные аспекты приложения, а все решения относительно поведения интерфейса делегируются контроллеру.

Применение шаблона Стратегия также сохраняет логическую изоляцию представления от модели, потому что все взаимодействия с моделью для выполнения пользовательских запросов осуществляются контроллером. Представлению о них ничего не известно.

Представление

Изображение состоит из вложенных окон, панелей, кнопок, надписей и т. д.

*Каждый компонент является комбинацией (окно) или листом (кнопка). Когда контроллер приказывает представлению обновиться, он обращается к верхнему компоненту, а шаблон **Компоновщик** делает все остальное.*

Модель

*Модель реализует шаблон **Наблюдатель** для оповещения заинтересованных объектов об изменениях состояния. Шаблон Наблюдатель обеспечивает полную независимость модели от представлений и контроллеров. Он позволяет использовать разные представления с одной моделью, или даже несколько представлений одновременно.*

Основная суть DDD

У нас есть домен (**domain**), который является бизнес задачей (например “счет на оплату билетов в кино”).

Работа с доменом начинается с анализа предметной области и составления требований. Этим занимаются Domain experts. Они прекрасно знают как работает домен и все его бизнес-процессы.

Далее происходит разбиение домена на под-домены (**subdomain**).

Subdomain — это те действующие элементы проекта, которые представляют из себя основной функционал (программное выражение наших бизнес ценностей).

Далее идет основной бизнес процесс (**Core domain**).

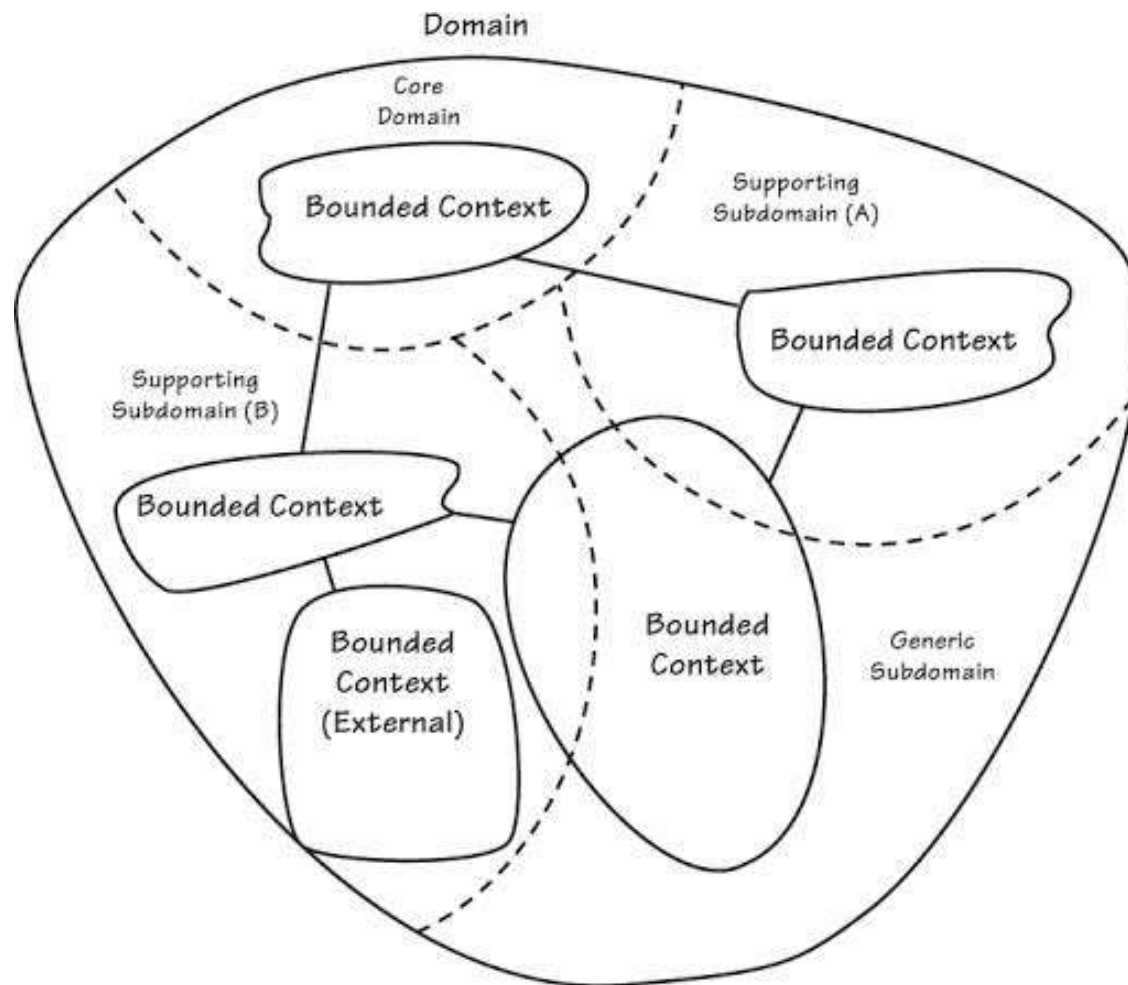
Core domain — основной бизнес процесс. Это не core domain, который в коде, некая сущность с основной логикой. Это весь процесс в целом. Он отвечает за бизнес ценность самого продукта.

Supporting subdomain — важная составляющая, от которых зависит наш бизнес, но которые не имеют прямого отношения к **Core domain**.

Generic subdomain — это составляющая, которая помогает бизнес процессу, на что мы можем не тратить свои ресурсы при проектировании.

Bounded context — это разграничение контекстов (это бизнес функционал, который описан BA).

Ниже я стырил картинку из интернета, на которой наглядно изображены все под-домены.



DDD позволяет упростить взаимодействие между всеми аспектами разработки продукта *от этапа появления идеи до тестирования уже полностью готового продукта.*

На протяжении всего этапа проектирования и разработки:

- вся команда будет общаться на едином языке, все будут понимать суть того что происходит в том или ином контексте,
- внесение доработок будет упрощено за счет распределения по контекстам,
- будет происходить раскидывание контекстов на те или иные группы между разработчиками и т.д.

• Линукс, сетевая инфраструктура Линикс, ядро, компоненты, проект реальный и тесты к нему.

Основные термины и концепции операционных систем

Обзор ядра Linux

Основные термины и понятия операционных систем¶

Пользователь против ядра¶

Ядро и пользователь — два термина, которые часто используются в операционных системах. Их определение довольно прямолинейно:

ядро — это часть операционной системы, которая работает с более высокими привилегиями, тогда как **пользователь** (пространство) обычно означает приложения, работающие с низкими привилегиями.

Однако эти термины сильно перегружены и могут иметь очень специфические значения в некоторых контекстах. Пользовательский режим и режим ядра — это термины, которые могут конкретно относиться к режиму выполнения процессора.

Код, работающий в режиме ядра, может полностью управлять ЦП, в то время как код, работающий в пользовательском режиме, имеет определенные ограничения. Например, локальные прерывания ЦП могут быть отключены или разрешены только при работе в режиме ядра. Если такая операция предпринята во время работы в пользовательском режиме, будет сгенерировано исключение, и ядро возьмет на себя его обработку.

Некоторые процессоры могут иметь даже более высокие привилегии, чем режим ядра, например, режим гипервизора, который доступен только для кода, работающего в гипервизоре (монитор виртуальной машины)

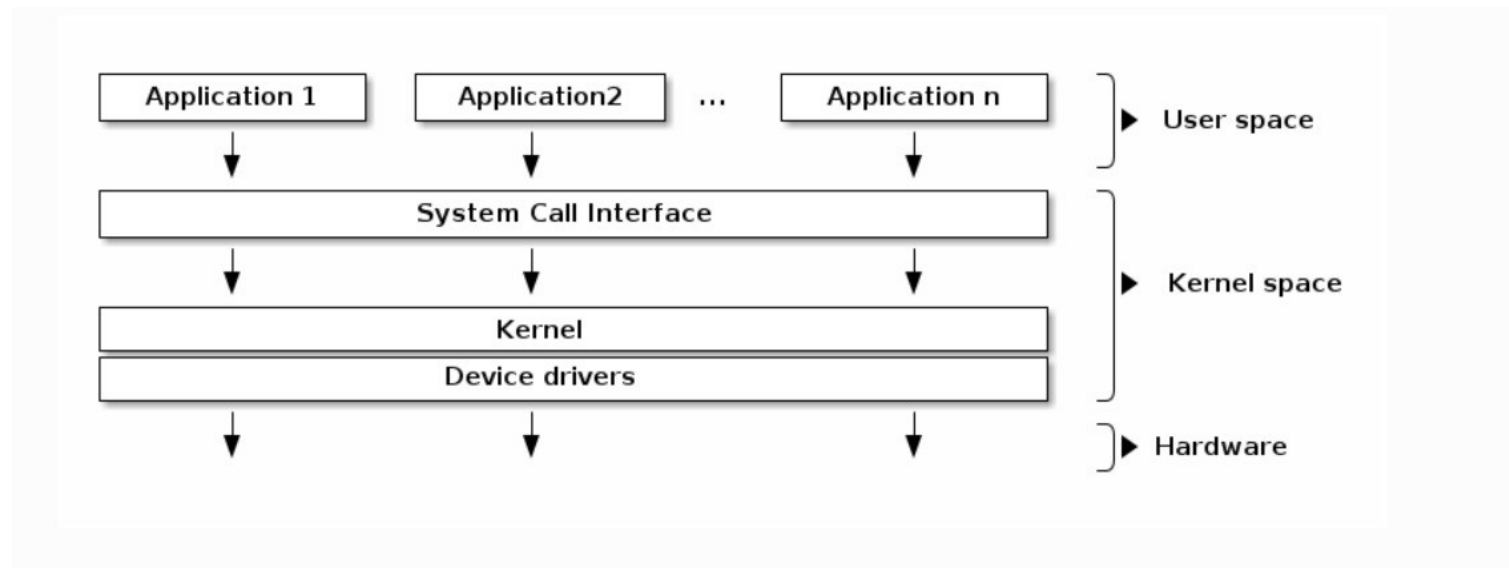
Пространство пользователя и пространство ядра могут конкретно относиться к защите памяти или к виртуальным адресным пространствам, связанным либо с ядром, либо с пользовательскими приложениями.

Грубо говоря, пространство ядра — это область памяти, зарезервированная для ядра, а пространство пользователя — это область памяти, зарезервированная для конкретного пользовательского процесса.

Доступ к пространству ядра защищен, поэтому пользовательские приложения не могут получить к нему прямой доступ, в то время как к пользовательскому пространству можно получить прямой доступ из кода, работающего в режиме ядра.

Типичная архитектура операционной системы¶

В типичной архитектуре операционной системы (см. рисунок ниже) ядро операционной системы отвечает за доступ и совместное использование оборудования безопасным и справедливым образом несколькими приложениями.



Ядро предлагает набор API-интерфейсов, которые выдают приложения, которые обычно называются «системными вызовами».

Эти API-интерфейсы отличаются от API-интерфейсов обычных библиотек, поскольку они являются границей, на которой режим выполнения переключается с пользовательского режима на режим ядра.

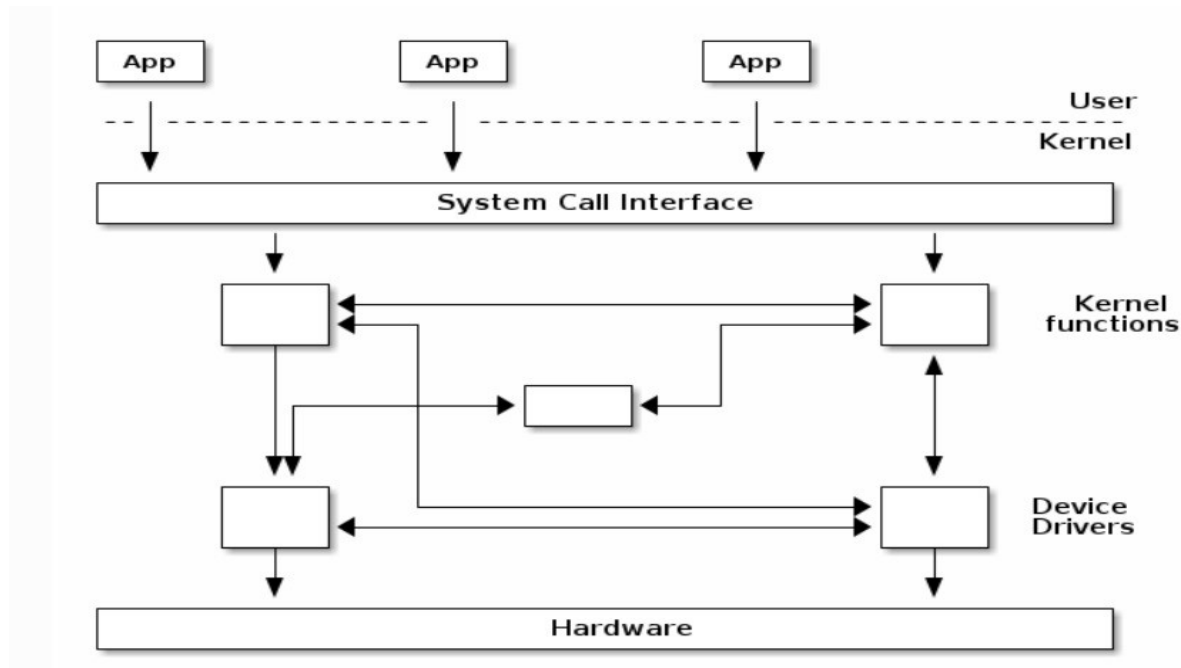
Для обеспечения совместимости приложений системные вызовы изменяются редко.

Linux особенно применяет это (в отличие от API-интерфейсов ядра, которые могут меняться по мере необходимости). Сам код ядра можно логически разделить на код ядра и код драйверов устройств.

Код драйверов устройств отвечает за доступ к конкретным устройствам, в то время как основной код ядра является общим. Ядро может быть дополнительно разделено на несколько логических подсистем (например, доступ к файлам, сеть, управление процессами и т. д.).

Монолитное ядро¶

Монолитное ядро — это ядро, в котором нет защиты доступа между различными подсистемами ядра и в котором общедоступные функции могут напрямую вызываться между различными подсистемами.



Тем не менее, большинство монолитных ядер обеспечивают логическое разделение между подсистемами, особенно между ядром ядра и драйверами устройств с относительно строгими API (но не обязательно зафиксированными в сборке), которые должны использоваться для доступа к службам, предлагаемым одной подсистемой или драйверами устройств. Это, конечно, зависит от конкретной реализации ядра и архитектуры ядра.

Монолитная ОС (UNIX или Novell NetWare) использует монолитное ядро, которое компонуется как одна программа, работающая в привилегированном режиме.

Монолитное ядро представляет собой набор процедур, каждая из которых может вызвать каждую.

Все процедуры работают в привилегированном режиме.

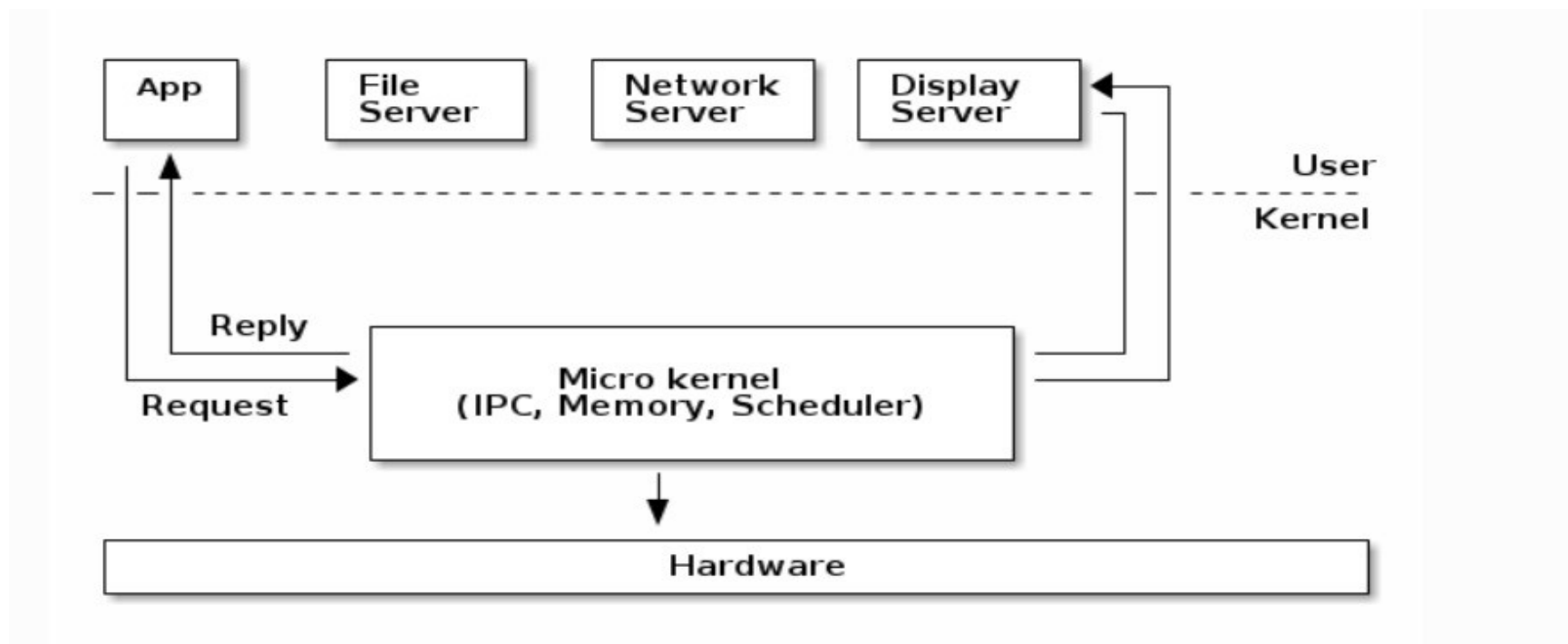
Таким образом, монолитное ядро – это такая схема операционной системы, при которой все ее компоненты являются составными частями одной программы, используют общие структуры данных и взаимодействуют друг с другом путем непосредственного вызова процедур.

Для монолитной операционной системы ядро совпадает со всей системой. Во многих операционных системах с монолитным ядром сборка ядра, то есть его компиляция, осуществляется отдельно для каждого компьютера, на который устанавливается операционная система. При этом можно выбрать список оборудования и программных протоколов, поддержка которых будет включена в ядро. Так как ядро является единой программой, перекомпиляция – это единственный способ добавить в него новые компоненты или исключить неиспользуемые.



Микроядро¶

Микроядро — это ядро, в котором большие части ядра защищены друг от друга и обычно работают как службы в пользовательском пространстве. Поскольку значительные части ядра теперь работают в пользовательском режиме, оставшийся код, работающий в режиме ядра, значительно меньше, отсюда и термин «микроядро».



В микроядерной архитектуре ядро содержит ровно столько кода, который позволяет передавать сообщения между различными работающими процессами.

На практике это означает реализацию планировщика и механизма IPC (межпроцессовое взаимодействие) в ядре, а также базовое управление памятью для настройки защиты между приложениями и службами.

Одним из преимуществ этой архитектуры является то, что сервисы изолированы, поэтому ошибки в одном сервисе не повлияют на другие сервисы.

Таким образом, в случае сбоя службы мы можем просто перезапустить ее, не затрагивая всю систему.

Однако на практике этого трудно добиться, поскольку перезапуск службы может повлиять на все приложения, которые зависят от этой службы (например, если файловый сервер выйдет из строя, все приложения с открытыми файловыми дескрипторами столкнутся с ошибками при доступе к ним).

Эта архитектура навязывает модульный подход к ядру и обеспечивает защиту памяти между службами, но за счет снижения производительности. То, что является простым вызовом функции между двумя службами на монолитных ядрах, теперь требует прохождения IPC и планирования, что приведет к снижению производительности.

Микроядерная архитектура является альтернативой классическому способу построения операционной системы. Под классической архитектурой в данном случае понимается рассмотренная выше структурная организация ОС, в соответствии с которой все основные функции операционной системы, составляющие многослойное ядро, выполняются в привилегированном режиме.

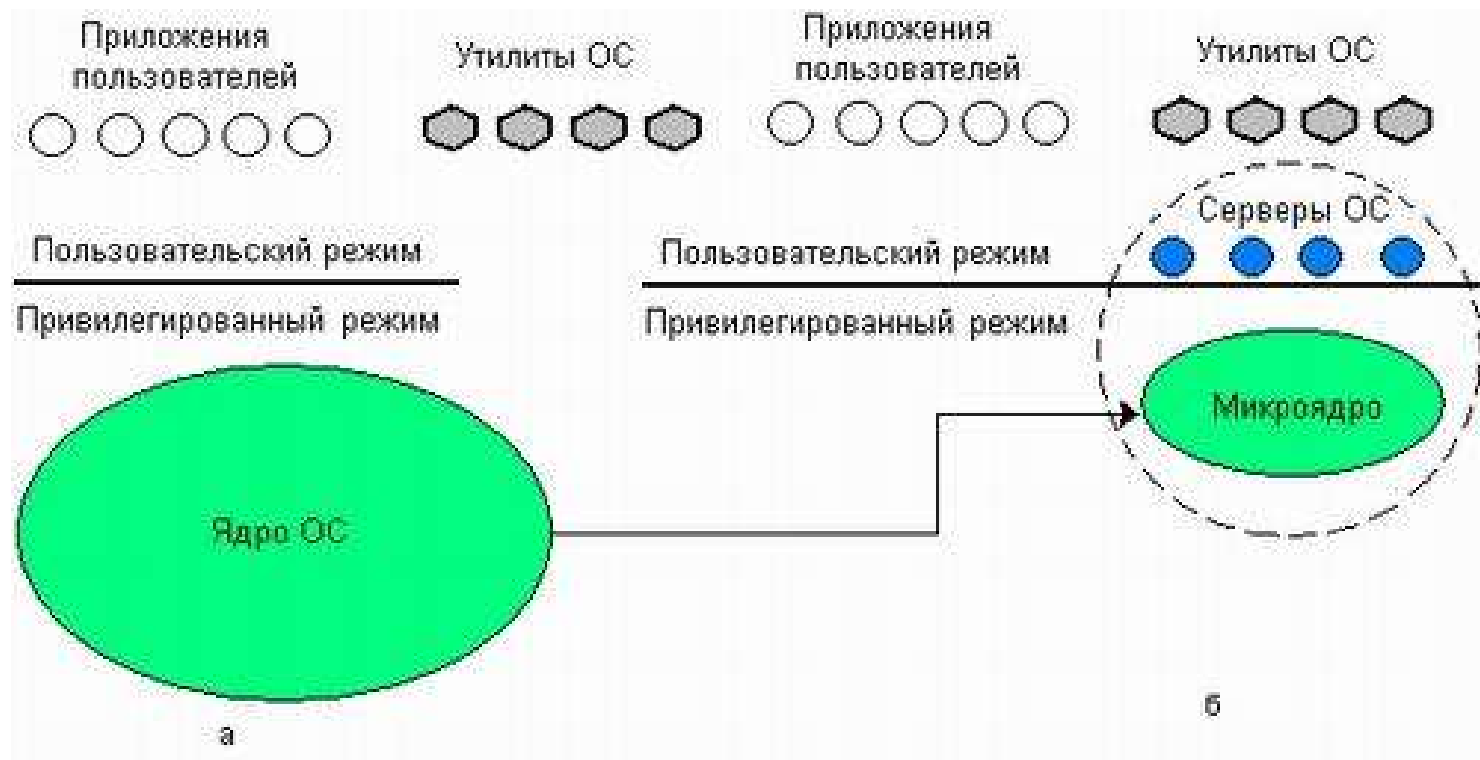
При этом некоторые вспомогательные функции ОС оформляются в виде приложений и выполняются в пользовательском режиме наряду с обычными пользовательскими программами (становясь системными утилитами или обрабатывающими программами).

Каждое приложение пользовательского режима работает в собственном адресном пространстве и защищено тем самым от какого-либо вмешательства других приложений.

Код ядра, выполняемый в привилегированном режиме, имеет доступ к областям памяти всех приложений, но сам полностью от них защищен.

Приложения обращаются к ядру с запросами на выполнение системных функций.

Суть микроядерной архитектуры состоит в следующем. В привилегированном режиме остается работать только очень небольшая часть ОС, называемая микроядром. Микроядро защищено от остальных частей ОС и приложений. В состав микроядра обычно входят машинно-зависимые модули, а также модули, выполняющие базовые (но не все!) функции ядра по управлению процессами, обработке прерываний, управлению виртуальной памятью, пересылке сообщений и управлению устройствами ввода-вывода, связанные с загрузкой или чтением регистров устройств. Набор функций микроядра обычно соответствует функциям слоя базовых механизмов обычного ядра. Такие функции операционной системы трудно, если не невозможно, выполнить в пространстве пользователя.



Перенос основного объема функций ядра в пользовательское пространство

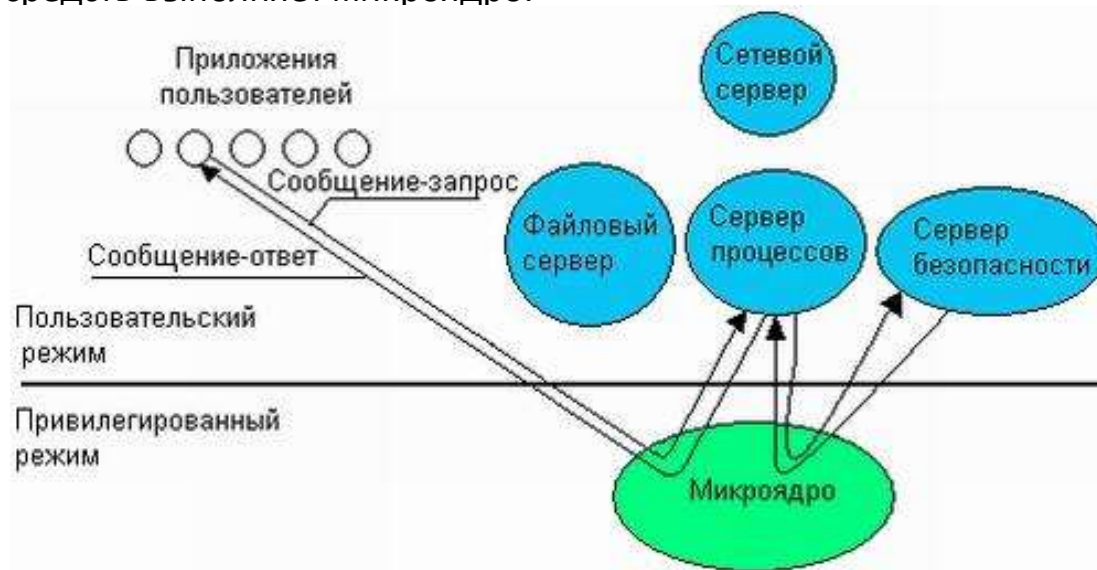
Все остальные более высокоуровневые функции ядра оформляются в виде приложений, работающих в пользовательском режиме. Однозначного решения о том, какие из системных функций нужно оставить в привилегированном режиме, а какие перенести в пользовательский, не существует. В общем случае многие менеджеры ресурсов, являющиеся неотъемлемыми частями обычного ядра — файловая система, подсистемы управления виртуальной памятью и процессами, менеджер безопасности и т. п., — становятся «периферийными» модулями, работающими в пользовательском режиме.

Работающие в пользовательском режиме менеджеры ресурсов имеют принципиальные отличия от традиционных утилит и обрабатывающих программ операционной системы, хотя при микроядерной архитектуре все эти программные компоненты также оформлены в виде приложений.

Утилиты и обрабатывающие программы вызываются в основном пользователями. Ситуации, когда одному приложению требуется выполнение функции (процедуры) другого приложения, возникают крайне редко. Поэтому в операционных системах с классической архитектурой отсутствует механизм, с помощью которого одно приложение могло бы вызвать функции другого.

Совсем другая ситуация возникает, когда в форме приложения оформляется часть операционной системы. По определению, основным назначением такого приложения является обслуживание запросов других приложений, например создание процесса, выделение памяти, проверка прав доступа к ресурсу и т. д. Именно поэтому менеджеры ресурсов, вынесенные в пользовательский режим, называются серверами ОС, то есть модулями, основным назначением которых является обслуживание запросов локальных приложений и других модулей ОС. Очевидно, что для реализации микроядерной архитектуры необходимым условием является наличие в операционной системе удобного и эффективного способа вызова процедур одного процесса из другого. Поддержка такого механизма и является одной из главных задач микроядра.

Схематично механизм обращения к функциям ОС, оформленным в виде серверов, выглядит следующим образом . Клиент, которым может быть либо прикладная программа, либо другой компонент ОС, запрашивает выполнение некоторой функции у соответствующего сервера, посылая ему сообщение. Непосредственная передача сообщений между приложениями невозможна, так как их адресные пространства изолированы друг от друга. Микроядро, выполняющееся в привилегированном режиме, имеет доступ к адресным пространствам каждого из этих приложений и поэтому может работать в качестве посредника. Микроядро сначала передает сообщение, содержащее имя и параметры вызываемой процедуры нужному серверу, затем сервер выполняет запрошенную операцию, после чего ядро возвращает результаты клиенту с помощью другого сообщения. Таким образом, работа микроядерной операционной системы соответствует известной модели клиент-сервер, в которой роль транспортных средств выполняет микроядро.



Реализация системного вызова в микроядерной архитектуре

Преимущества и недостатки микроядерной архитектуры

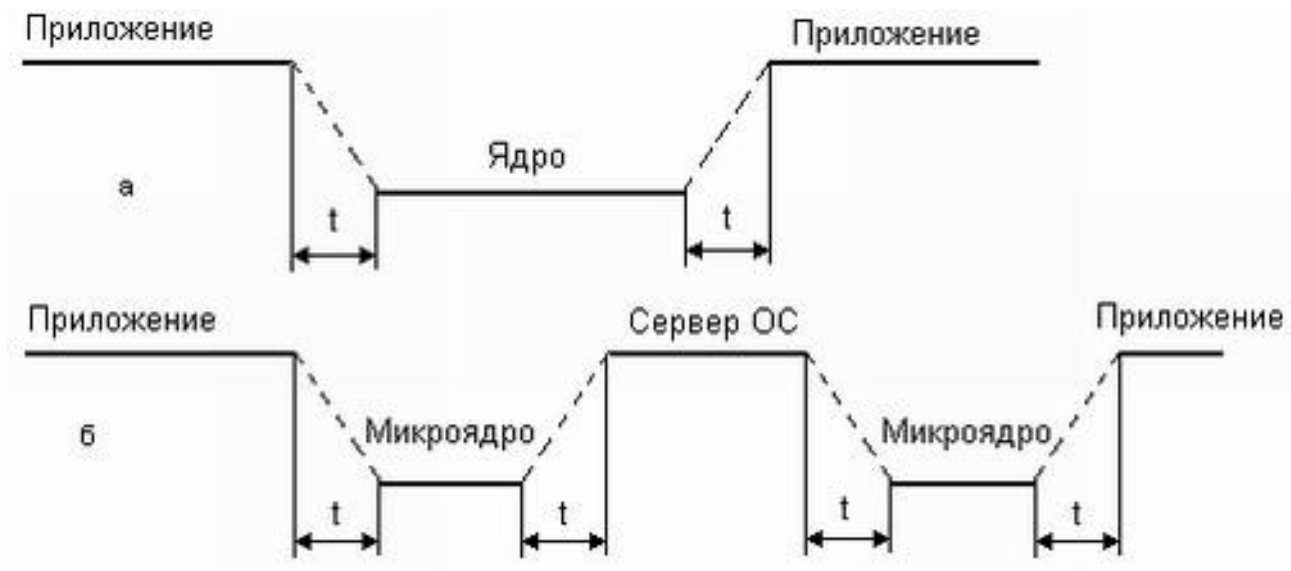
Операционные системы, основанные на концепции микроядра, в высокой степени удовлетворяют большинству требований, предъявляемых к современным ОС, обладая переносимостью, расширяемостью, надежностью и создавая хорошие предпосылки для поддержки распределенных приложений. За эти достоинства приходится платить снижением производительности, и это является основным недостатком микроядерной архитектуры. Высокая степень переносимости обусловлена тем, что весь машинно-зависимый код изолирован в микроядре, поэтому для переноса системы на новый процессор требуется меньше изменений и все они логически сгруппированы вместе.

Расширяемость присуща микроядерной ОС в очень высокой степени. В традиционных системах даже при наличии многослойной структуры нелегко удалить один слой и поменять его на другой по причине множественности и размытости интерфейсов между слоями. Добавление новых функций и изменение существующих требует хорошего знания операционной системы и больших затрат времени. В то же время ограниченный набор четко определенных интерфейсов микроядра открывает путь к упорядоченному росту и эволюции ОС. Добавление новой подсистемы требует разработки нового приложения, что никак не затрагивает целостность микроядра. Микроядерная структура позволяет не только добавлять, но и сокращать число компонентов операционной системы, что также бывает очень полезно. Например, не всем пользователям нужны средства безопасности или поддержки распределенных вычислений, а удаление их из традиционного ядра чаще всего невозможно. Обычно традиционные операционные системы позволяют динамически добавлять в ядро или удалять из ядра только драйверы внешних устройств — ввиду частых изменений в конфигурации подключенных к компьютеру внешних устройств подсистема ввода-вывода ядра допускает загрузку и выгрузку драйверов «на ходу», но для этого она разрабатывается особым образом (например, среда STREAMS в UNIX или менеджер ввода-вывода в Windows NT). При микроядерном подходе конфигурируемость ОС не вызывает никаких проблем и не требует особых мер — достаточно изменить файл с настройками начальной конфигурации системы или же остановить не нужные больше серверы в ходе работы обычными для остановки приложений средствами.

Использование микроядерной модели повышает надежность ОС. Каждый сервер выполняется в виде отдельного процесса в своей собственной области памяти и таким образом защищен от других серверов операционной системы, что не наблюдается в традиционной ОС, где все модули ядра могут влиять друг на друга. И если отдельный сервер терпит крах, то он может быть перезапущен без останова или повреждения остальных серверов ОС. Более того, поскольку серверы выполняются в пользовательском режиме, они не имеют непосредственного доступа к аппаратуре и не могут модифицировать память, в которой хранится и работает микроядро. Другим потенциальным источником повышения надежности ОС является уменьшенный объем кода микроядра по сравнению с традиционным ядром — это снижает вероятность появления ошибок программирования.

Модель с микроядром хорошо подходит для поддержки распределенных вычислений, так как использует механизмы, аналогичные сетевым: взаимодействие клиентов и серверов путем обмена сообщениями. Серверы микроядерной ОС могут работать как на одном, так и на разных компьютерах. В этом случае при получении сообщения от приложения микроядро может обработать его самостоятельно и передать локальному серверу или же переслать по сети микроядру, работающему на другом компьютере. Переход к распределенной обработке требует минимальных изменений в работе операционной системы — просто локальный транспорт заменяется на сетевой.

Производительность. При классической организации ОС выполнение системного вызова сопровождается двумя переключениями режимов, а при микроядерной организации — четырьмя. Таким образом, операционная система на основе микроядра при прочих равных условиях всегда будет менее производительной, чем ОС с классическим ядром. Именно по этой причине микроядерный подход не получил такого широкого распространения, которое ему предсказывали.



Смена режимов при выполнении системного вызова

Серьезность этого недостатка хорошо иллюстрирует история развития Windows NT. В версиях 3.1 и 3.5 диспетчер окон, графическая библиотека и высокоуровневые драйверы графических устройств входили в состав сервера пользовательского режима, и вызов функций этих модулей осуществлялся в соответствии с микроядерной схемой. Однако очень скоро разработчики Windows NT поняли, что такой механизм обращений к часто используемым функциям графического интерфейса существенно замедляет работу приложений и делает данную операционную систему уязвимой в условиях острой конкуренции. В результате в версию Windows NT 4.0 были внесены существенные изменения — все перечисленные выше модули были перенесены в ядро, что отдалило эту ОС от идеальной микроядерной архитектуры, но зато резко повысило ее производительность.

Этот пример иллюстрирует главную проблему, с которой сталкиваются разработчики операционной системы, решившие применить микроядерный подход, — что включать в микроядро, а что выносить в пользовательское пространство. В идеальном случае микроядро может состоять только из средств передачи сообщений, средств взаимодействия с аппаратурой, в том числе средств доступа к механизмам привилегированной защиты. Однако многие разработчики не всегда жестко придерживаются принципа минимизации функций ядра, часто жертвуя этим ради повышения производительности. В результате реализации ОС образуют некоторый спектр, на одном краю которого находятся системы с минимально возможным микроядром, а на другом — системы, подобные Windows NT, в которых микроядро выполняет достаточно большой объем функций.

Микроядра против монолитных ядер¶

микроядра лучше из-за модульной конструкции, которую обеспечивает микроядро.

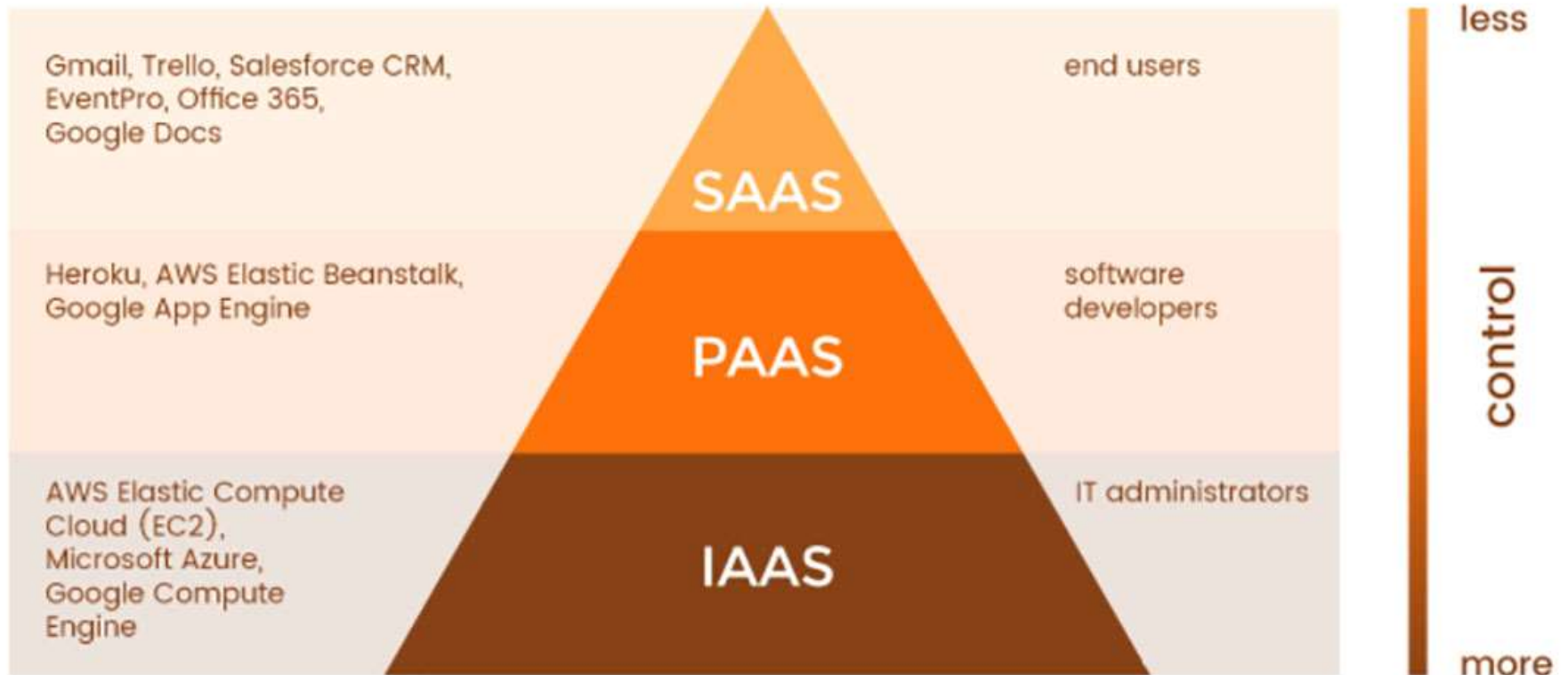
Однако монолитные ядра также могут быть модульными, и для достижения этой цели современные монолитные ядра используют несколько подходов:

Компоненты могут быть включены или отключены во время компиляции.

Поддержка загружаемых модулей ядра (во время выполнения)

Организация ядро в логические независимые подсистемы.

Строгие интерфейсы, но с низкой производительностью: макросы, встроенные функции, указатели функций.



Что такое IaaS: облачная инфраструктура как услуга

Практически ни одна компания не обходится без IT-инфраструктуры — даже небольшой фирме нужны серверы для хранения баз данных или инструменты, объединяющие компьютеры сотрудников в общую сеть. Компания может закупить серверы и настроить эту инфраструктуру у себя, но это достаточно долго и дорого. А может ничего не покупать, если взять в аренду облачные сервисы IaaS.

IaaS — это Infrastructure as a Service, то есть инфраструктура как услуга. К инфраструктуре относят вычислительные ресурсы: виртуальные серверы, хранилища, сети. Это что-то вроде виртуальных «компьютеров», на которые можно установить что угодно: операционную систему, программное обеспечение, приложения.

Поставщик IaaS (инфраструктуры как сервиса), или облачный провайдер, уже все купил и собрал, обеспечил серверы электричеством и интернетом. Вам остается только подключиться к этим вычислительным мощностям через интернет и использовать их для своих целей.

IaaS включает в себя:

1. Виртуальные серверы (VPS/VDS), на которые можно устанавливать различные программы. Иногда провайдер предлагает серверы сразу с операционными системами, чтобы на них можно было быстрее развернуть нужные приложения.
 2. Сетевые настройки, которые обеспечивают связь виртуальных серверов друг с другом, внешними серверами, принадлежащими компании-клиенту, и интернетом. К ним относят:
 1. доступность серверов друг для друга и для внешней сети, маршрутизацию сетевых соединений серверов;
 2. балансировщики нагрузки, которые предотвращают перегрузки серверов, распределяя между ними входящий трафик;
 3. VPN — технологию шифрования данных, передаваемых компанией между облаком и ее физическим дата-центром;
 3. Управление доступом пользователей. Например, можно ограничить доступ к отдельным виртуальным машинам или разрешить просмотр данных, но запретить вносить в них изменения.
 4. Облачные хранилища для хранения файлов, данных или бэкапов. От обычных облачных дисков, с которыми имеют дело отдельные пользователи, они отличаются почти неограниченным объемом хранения и быстрой скоростью доступа к данным.
 5. Сервисы резервного копирования и катастрофоустойчивости, которые страхуют вашу инфраструктуру от падений и потери данных при выходе из строя ее отдельных узлов.
- Когда вы арендуете инфраструктуру IaaS, то получаете виртуальные серверы, сетевые настройки и облачные хранилища, за которые отвечает провайдер. При этом у вас есть доступ к операционной системе виртуальных серверов и большая свобода в настройке и установке приложений.

Примеры IaaS — как можно использовать облачную инфраструктуру:

1.Перенос IT-систем в облако. Самое очевидное применение IaaS — отказ от физических серверов и перенос вашей IT-инфраструктуры в облако. С IaaS вы сможете делать все то же, что с физическим дата-центром — хранить данные, настроить CRM-систему, развернуть сайт или любые бизнес-серверы.

2.Экономия на инфраструктуре. Если вы сами обслуживаете серверы, вам нужно платить за помещение, оборудование, лицензионное ПО — это большие капитальные расходы. Кроме того, в облаке провайдер берет на себя предоставление виртуальной инфраструктуры с оговоренным в SLA временем простоя (даунтаймом). Чтобы обеспечить такой же аптайм в собственном частном ЦОДе, а также управлять физической инфраструктурой и виртуализацией, нужен отдельный штат специалистов. Это дополнительные расходы сверх капитальных. Если собственный ЦОД для компании — не профильный бизнес, компания не умеет оценивать риски и дополнительные и скрытые расходы, у нее нет соответствующей экспертизы, тогда дешевле развернуть IT-инфраструктуру в облаке.

3.Быстрый запуск бизнеса. Облачная инфраструктура IaaS снижает расходы бизнеса на старте, так как помогает уменьшить капитальные вложения, обойтись без закупки оборудования и организации дата-центра.

4.Расширение инфраструктуры. Облачные технологии IaaS можно использовать и для расширения уже существующей инфраструктуры. К примеру, у вас есть физические серверы, но их мощности не хватает для нынешних задач. Вместо того чтобы покупать новые, можно подключиться к IaaS-платформе, и получить там нужные мощности.

1. Инфраструктура для компаний со скачками спроса. IaaS подходит, если у компании нелинейный спрос на ресурсы. Например, у вас есть физические серверы, на которых развернут интернет-магазин. В период праздников или распродаж сайт падает — покупателей приходит слишком много, и серверы не выдерживают. Если использовать облачную инфраструктуру, при увеличении нагрузки можно сразу получить дополнительные вычислительные мощности. А когда нагрузка упадет, вернуться к плановому потреблению мощностей. В случае обычной физической инфраструктуры нужно докупать дополнительные серверы — пользоваться ими вы будете только на пике нагрузки, а обслуживать постоянно. Это не говоря о том, что процедура закупки оборудования обычно занимает недели, а облачное масштабирование — минуты или даже секунды.

2. Разработка и тестирование. Как и в собственной локальной инфраструктуре, в облачной можно организовать отдельные среды разработки, тестирования и «боевой» нагрузки, в которой запущено уже готовое приложение. Но, в отличие от собственной инфраструктуры, в облаке тестовые среды можно разворачивать мгновенно. После завершения тестов лишние среды можно свернуть и не переплачивать за простаивающие ресурсы. Всё это упрощает и ускоряет процесс тестирования ПО и позволяет экономить на покупке тестовых серверов.

Для построения в облаке сложной инфраструктуры IaaS для крупных проектов может потребоваться [помощь провайдера в миграции и администрировании](#). Но для большинства проектов управление [облачной инфраструктурой](#) не требует непосредственного участия провайдера. А если возникнут сложности, всегда можно обратиться в техподдержку.

Что такое PaaS: облачная платформа как услуга

Провайдеры облачных услуг могут предоставлять уже настроенные инструменты (платформы) под разные задачи. Такие инструменты называют **PaaS — это Platform as a Service**, платформа как услуга.

Ключевое отличие PaaS от IaaS в том, что здесь у вас есть определенные инструменты, например: система управления базами данных, среда машинного обучения или обработки big data, промышленный IoT. Их нужно настроить под потребности компании, но не надо строить с нуля. Это позволяет экономить время разработчиков — например, им не нужно возиться с разработкой базы данных, можно просто загрузить в нее информацию и работать.

При этом у вас нет доступа к операционной системе, настройкам виртуальных серверов, которые лежат в основе PaaS, а также к низкоуровневым настройкам самой платформы. Провайдер берет на себя их оптимальную конфигурацию и снимает с вас необходимость следить за настройками, обновлениями, масштабированием и безопасностью. Вы получаете доступ только к интерфейсам самой платформы.

Вот несколько **примеров PaaS**:

1. Базы данных. В облака можно перенести все или часть баз данных компании. В случае с IaaS пользователь получает только пространство на дисках, и должен сам выбрать систему управления базами данных, установить ее и настроить, обеспечить защиту данных и резервное копирование. В PaaS СУБД уже установлена, нужно лишь настроить ее для себя и загрузить данные. За работоспособность и резервное копирование отвечает провайдер.

2. Разработка приложений в контейнерах. Это современный стандарт разработки приложений. Суть в том, что все компоненты, которые нужны для запуска приложения, упаковывают в отдельные виртуальные контейнеры. Их можно быстро вызывать, запускать приложения и при необходимости добавлять вычислительные мощности для быстрого масштабирования и устойчивости к высоким нагрузкам.

3. Аналитика больших данных. PaaS помогают обрабатывать как исторические большие данные, то есть массивы информации, собранные компанией за какое-то время, так и данные в режиме реального времени. Для этого используют такие инструменты как Apache Hadoop, Apache Spark, Apache Kafka и другие. Они уже установлены и настроены в облаке, вам остается только выбрать нужную конфигурацию.

4. Машинное обучение. Такая платформа как сервис позволяет быстро разрабатывать приложения на основе глубокого обучения под нужды компании: системы компьютерного зрения для распознавания лиц, номеров автомобилей и других объектов, системы аудиоаналитики. К примеру, у вас есть большая база данных с фотографиями сотрудников, и вы хотите настроить на объекте безбарьерный вход — пропускать сотрудников через турникеты, распознавая их лица. Для этого вы разрабатываете собственное приложение, а потом «учите» нейросеть узнавать сотрудников с помощью PaaS, где уже установлены инструменты для распознавания лиц.

PaaS-сервисы экономят время на конфигурацию инфраструктуры, нужной компании. Вы можете подключить нужный набор сервисов, главное, убедиться, что у облачного провайдера есть нужные сейчас и в перспективе решения.

Что такое SaaS: облачные сервисы для конкретных прикладных задач

SaaS — это полностью настроенная и готовая к работе программа, выполняющая определенные функции. Единственное отличие технологии SaaS от программы на смартфоне или компьютере в том, что сам софт находится в облаке. Доступ к нему идет через интернет, а сама программа работает на мощностях виртуальных серверов, поэтому не нагружает ваш компьютер или смартфон.

SaaS — [это Software as a Service](#), программное обеспечение как сервис. IaaS и PaaS для SaaS-сервисов могут выступать как инфраструктура и среды разработки и развертывания: разработчики программного обеспечения используют облачные мощности, чтобы разрабатывать, запускать и хранить SaaS-приложения, обеспечивать к ним доступ пользователям.

Примеры SaaS — это большинство сервисов в интернете: электронная почта, CRM-системы, планировщики задач, веб-конструкторы для создания сайтов, платформы для ведения блогов. То есть все облачные программы, которые позволяют решать конкретные задачи.

Если у вас есть задача, которая полностью решается платформой SaaS, то дешевле и быстрее воспользоваться ей, а не разрабатывать собственное решение.

Так, даже крупная компания вряд ли будет разрабатывать свой почтовый клиент — есть готовые почтовые сервисы, которые можно приобрести по подписке и функционала которых достаточно для работы.

IaaS, PaaS и SaaS — что это и когда нужны

IaaS. Вычислительные мощности для запуска своих решений и развертывания IT-инфраструктуры компании.

PaaS. Позволяет использовать уже готовые и настроенные платформы для специализированных задач.

SaaS. Модель, когда в облаках размещены готовые сервисы для конкретных прикладных функций — отправки почты, ведения базы клиентов, создания сайта.

Виды виртуализации

Классификация виртуализации носит условный характер, её условно можно разделить на две фундаментально различающиеся категории:

- **виртуализация платформ.** Продуктом этого вида виртуализации являются виртуальные машины – некие программные абстракции, запускаемые на платформе реальных аппаратно-программных систем.
- **виртуализация ресурсов.** Данный вид виртуализации преследует своей целью комбинирование или упрощение представления аппаратных ресурсов для пользователя и получение неких пользовательских абстракций оборудования, пространств имен, сетей и т.п.

Виртуализация платформ

В зависимости от степени виртуализации аппаратных ресурсов, виртуализацию платформ делят на следующие виды:

- виртуализация серверов;
- виртуализация на уровне ОС;
- виртуализация приложений;
- виртуализация представлений.

Виртуализация серверов сводится к:

- полной эмуляции (симуляции);
- частичной эмуляции (нативной или встроенной виртуализации);
- частичной виртуализации;
- паравиртуализация.

Полная эмуляция (симуляция). Этот вид виртуализации построен на основе полной эмуляции всего аппаратного обеспечения определенной платформы. Такой подход позволяет эмулировать различные аппаратные архитектуры. Например, на платформе x86 это позволяет запустить операционную систему, разработанную для процессоров IBM PowerPC.

Главным и основным минусом такого подхода является очень низкое быстродействие гостевой системы (в 100—1000 раз меньше, чем физического компьютера), так как происходит полная эмуляция всех инструкций виртуального процессора, а инструкции физического процессора совсем не используются.

Полная эмуляция используется исключительно при разработке системного программного обеспечения и в образовательных целях. Например, при разработке программного обеспечения под процессоры, которые еще не реализованы физически; для низкоуровневой отладки операционных систем, так как симуляция позволяет отслеживать все инструкции виртуального процессора, что нельзя сделать в системах, которые частично или полностью используют реальный процессор.

Программные платформы симуляции: VMware Workstation, VMware Server, VMware ESX Server, Virtual Iron, Microsoft Hyper-V Server, Microsoft Virtual PC, Sun VirtualBox, Parallels Desktop и другие.

Частичная эмуляция (нативная виртуализация) базируется на принципе эмуляции только необходимого количества ресурсов, чтобы виртуальная машина могла быть запущена изолированно. Например, при полной эмуляции различных архитектур, гостевая система работает с определенной специфической системой команд процессора, отличной от системы команд процессора хостовой системы. Каждую команду процессору гостевой системы нужно транслировать в соответствующую команду хостовой системы, что невероятно уменьшает быстродействие. При использовании нативной виртуализации никакой трансляции команд не происходит, так как гостевая операционная система разработана под ту же архитектуру, на которой работает хостовая система. Это позволяет значительно повысить быстродействие гостевой системы и максимально приблизить его к быстродействию реальной системы.

Для повышения быстродействия нативной виртуализации применяется специализированная программная прослойка – **гипервизор**. Гипервизор является посредником между гостевой операционной системой и физическим аппаратным обеспечением. Он позволяет гостевой системе напрямую обращаться к аппаратным ресурсам, что и является секретом высокого быстродействия данного вида виртуализации.

Частичная эмуляция является самым распространенным видом виртуализации в наше время. Основным ее недостатком является зависимость виртуальных машин от конкретной аппаратной архитектуры.

Платформы нативной виртуализации: VMware Workstation, VMware Server, VMware ESX Server, Virtual Iron, Virtual PC, VirtualBox, Parallels Desktop и другие.

Частичная виртуализация и виртуализация адресного пространства

Подход частичной виртуализации заключается в симулировании нескольких экземпляров аппаратного обеспечения (особенно часто используется виртуализация адресного пространства). Это позволяет совместно использовать ресурсы и изолировать процессы, но не позволяет создавать экземпляры гостевых систем. Т.е. при частичной виртуализации виртуальные машины не создаются, а происходит изоляция некоторых ресурсов и процессов на уровне хостовой операционной системы. Каждая гостевая система является приложением, запущенным как процесс внутри изолированного адресного пространства.

Недостатком частичной виртуализации является невозможность разделить гостевую и хостовую системы, так как они выполняются внутри ядра хостовой системы.

Платформы частичной виртуализации: UML (User-mode Linux), при котором гостевое ядро запускается внутри основного ядра и выполняется в его контексте.

Паравиртуализация. Этот метод не вынуждает эмулировать аппаратное обеспечение, однако требует от гостевой системы наличия специального программного интерфейса для взаимодействия с хостовой системой. Такой подход требует значительной модификации ядра гостевой, так как все взаимодействие ее с аппаратным обеспечением сводится к совершению специальных вызовов (гипервызовов) к хостовой системе. При этом хостовая система должна иметь гипервизор, который принимает и обрабатывает гипервызовы из гостевой системы. Быстродействие гостевых систем с паравиртуализацией считается максимальным. В отличие от нативной виртуализации, в случае паравиртуализации нет нужды эмулировать BIOS, процессор, жесткие диски и другие устройства. Это и дает выигрыш в производительности паравиртуализированных систем на уровне 5-15% по сравнению с нативными. За такой прирост производительности нужно расплачиваться понижением гибкости (не всегда найдется нужная модифицированная операционная система) и безопасности (в следствии близости гостевых систем к аппаратному уровню могут возникать различные коллизии) таких решений. С ростом популярности виртуализации, появлением в современных процессорах встроенной поддержки технологий виртуализации Intel VT-x) и AMD-V, решений для паравиртуализации в чистом виде практически не осталось.

Платформы паравиртуализации: VMware ESX Server, Xen, Microsoft Hyper-V. Эти продукты поддерживают паравиртуализацию, частичную виртуализацию и их смесь.

– Виртуализация на уровне ядра операционной системы (ОС). Данный метод позволяет в рамках одной ОС выделить ресурсы для выполнения независимых гостевых ОС. Каждая гостевая система имеет изолированную от хостовой системы среду выполнения, что обеспечивает достаточную безопасность. Данный метод позволяет разбить физические ресурсы компьютера на независимые разделы (контейнеры или песочницами), средствами операционной системы. Такая виртуализация используется при организации систем для хостинга, когда в рамках одной операционной системы необходимо предоставлять клиентам несколько виртуальных серверов.

Платформы виртуализации: Linux-VServer, OpenVZ, Virtuozzo, FreeBSD Jails, Solaris Containers.

– Виртуализация приложений значительно отличается от всех остальных видов. Целевое приложение (а не вся операционная система) помещается в специальный контейнер, который эмулирует работу операционной системы с данным приложением. Контейнер содержит все необходимые приложению ресурсы: системные файлы, файлы реестра и конфигурационные файлы. При этом в контейнер помещаются исключительно файлы, необходимые для нормальной работы целевого приложения, что делает размеры контейнера минимальными. Использование изолированного контейнера позволяет решать конфликты несовместимости программ между собой или между операционными системами. Например, благодаря виртуализации приложений становится возможным установить и запустить несколько экземпляров одной программы разных версий. Microsoft Office 2003 и Microsoft Office 2007 могут одновременно работать без каких-либо конфликтов, Adobe Photoshop CS2 может работать рядом с Adobe Photoshop CS4. Данный метод также позволяет запускать программные продукты, предназначенные для более поздних версий операционной системы в более ранних версиях с сохранением полной совместимости. Например, программу для Windows Vista становится возможным запустить в Windows 2000.

Платформы виртуализации приложений: Microsoft SoftGrid, VMware ThinApp, Altiris Software Virtualization Solution.

КАК РАБОТАЕТ ВИРТУАЛИЗАЦИЯ?

В неvirtуальных условиях, архитектуры x86 строго контролируют, какие процессы могут работать в каждом из четырех тщательно определенных уровней привилегий (начиная с Кольца 0 (Ring 0) по Кольцо 3).

Как правило, только ядро операционной системы хоста имеет какой-либо шанс получить доступ к инструкциям, хранящимся в кольце под номером 0. Однако, поскольку вы не можете предоставить нескольким виртуальным машинам, которые работают на одном физическом компьютере, равный доступ к кольцу 0, не вызывая больших проблем, необходим диспетчер виртуальных машин (или «гипервизор»), который бы эффективно перенаправлял запросы на такие ресурсы, как память и хранилище, на виртуализированные системы, эквивалентные им.

При работе в аппаратной среде без виртуализации SVM или VT-x все это выполняется с помощью процесса, известного как ловушка, эмуляция и двоичная трансляция. На виртуализированном оборудовании такие запросы, как правило, перехватываются гипервизором, адаптируются к виртуальной среде и возвращаются в виртуальную машину.

Виртуализация и кольца привилегий



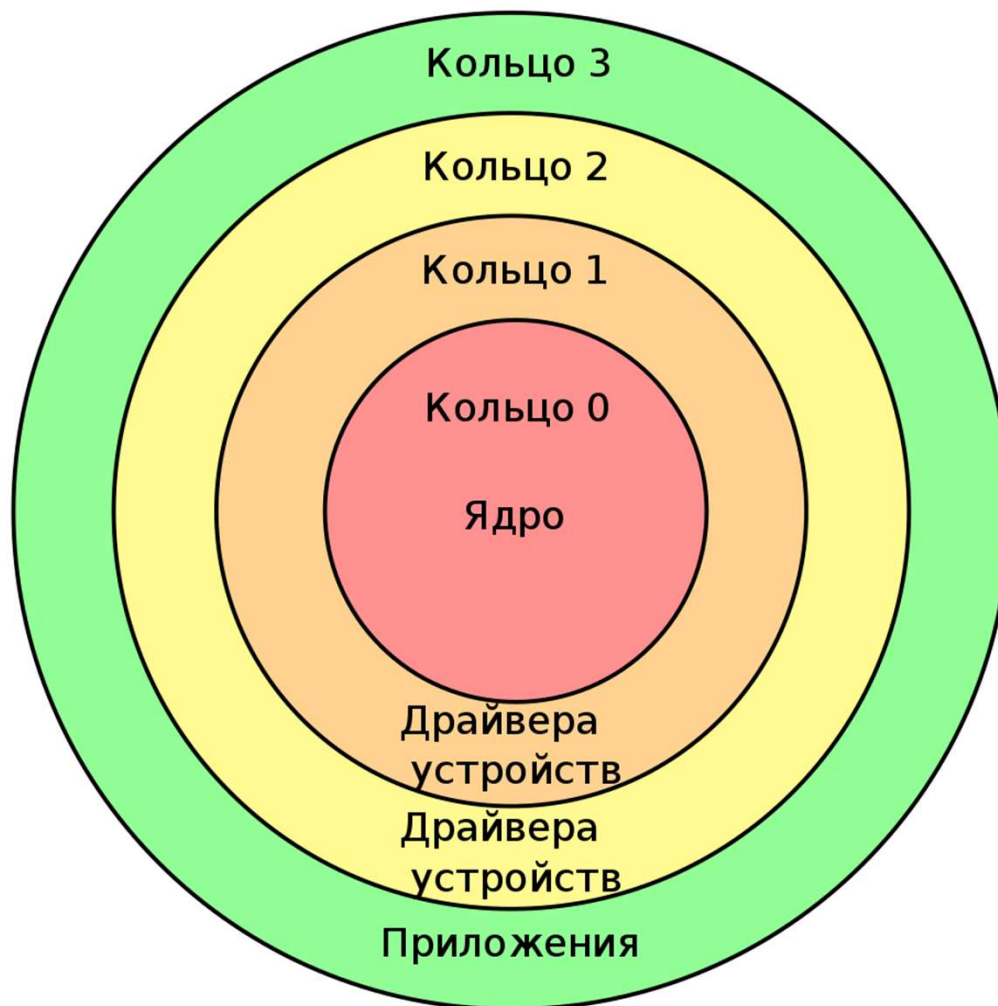
Простое добавление нового программного уровня для обеспечения такого уровня организации взаимодействия приведет к значительной задержке практически во всех аспектах производительности системы. Одним из успешных решений было решение ввести новый набор инструкций в ЦП, которые создают, так называемое, «кольцо 1», которое действует как кольцо 0 и позволяет гостевой ОС работать без какого-либо влияния на другие несвязанные операции.

На самом деле, при правильной реализации виртуализация позволяет большинству программных кодов работать как обычно, без каких-либо перехватов.

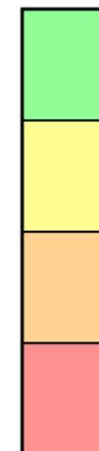
Несмотря на то, что эмуляция часто играет роль поддержки при развертывании виртуализации, она все же работает несколько иначе. В то время как виртуализация стремится разделить существующие аппаратные ресурсы между несколькими пользователями, эмуляция ставит перед собой цель заставить одну конкретную аппаратную/программную среду имитировать ту, которой на самом деле не существует, чтобы у пользователей была возможность запускать процессы, которые изначально было невозможно запустить. Для этого требуется программный код, который имитирует желаемую исходную аппаратную среду, чтобы обмануть ваше программное обеспечение, заставив его думать, что оно на самом деле работает где-то еще.

Эмуляция может быть относительно простой в реализации, но она почти всегда несет за собой значительные потери производительности.

Согласно сложившимся представлениям, существует два класса гипервизоров: **Type-1** и **Type-2**.



Минимум
привилегий



Максимум
привилегий

Термином **режим супервизора** (Supervisor mode) разработчики и производители процессоров, как правило, называют наиболее привилегированный режим работы процессора. Чаще всего этот режим используется для выполнения кода ядра операционной системы. Обычно этот режим по функциональности соответствует 0-му кольцу защиты (Ring 0) в x86-процессорах, то есть предоставляет неограниченный доступ ко всем возможностям процессора, работе с периферией и так далее. Код, работающий в данном режиме, как правило, занимается управлением доступными аппаратными ресурсами, разделением их использования между отдельными задачами (процессами) и так далее, что и привело к такому названию режима.

Некоторые фирмы-разработчики и производители процессоров, например ARM, не используют классификацию режимов работы процессора в виде колец защиты. Тем не менее, у большинства современных процессоров (кроме простейших), как правило, присутствует несколько режимов работы, отличающихся друг от друга доступными в данном режиме привилегиями.

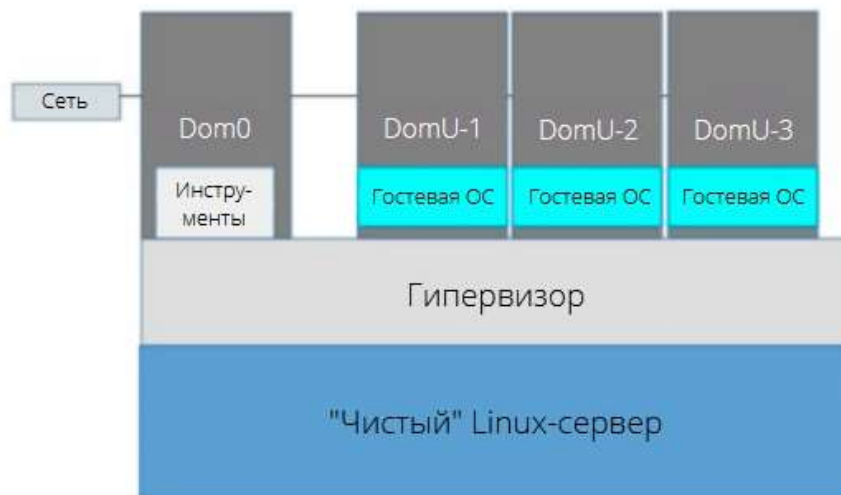
Некоторые современные процессоры могут предоставлять дополнительный режим работы, известный как **режим гипервизора** (Hypervisor mode). Как правило, данный режим реализуется с целью поддержки технологий виртуализации на уровне аппаратного обеспечения. Это позволяет достигнуть не только одновременного выполнения нескольких задач, но и одновременного выполнения нескольких операционных систем на одном процессоре без существенных потерь производительности и без изменения самих операционных систем. Как правило, при использовании данного режима полный доступ ко всем ресурсам возможен именно из режима гипервизора. В таком случае режим супервизора уже не является максимально привилегированным и ограничивает многие привилегированные операции. При выполнении привилегированных операций операционными системами в режиме супервизора управление передается специальной программе — [гипервизору](#). Гипервизор осуществляет арбитраж использования имеющихся аппаратных ресурсов несколькими операционными системами аналогично тому, как сами операционные системы осуществляют распределение ресурсов между несколькими задачами. По сути, гипервизор обычно является небольшим ядром, которое управляет распределением ресурсов между несколькими операционными системами и работает уровнем ниже, чем сами операционные системы. В силу этого в терминологии x86 данный режим часто условно называют кольцом -1 (Ring -1).

Режим системного управления (System Management Mode) — самый привилегированный режим выполнения на процессорах архитектуры [x86/x86-64^{\[1\]}](#) (впервые появился в [386SL](#)). Режим SMM (условно называемый «Кольцо -2») более привилегированный, чем «Кольцо 0» и аппаратный гипервизор ([VT/AMD-y](#)) «Кольцо -1». В этом режиме приостанавливается нормальное выполнение кода, и начинается выполнение специального кода из системного ОЗУ (SMRAM), недоступного в других режимах. Этот код получает доступ ко всей системной памяти, включая ядро и память гипервизора.

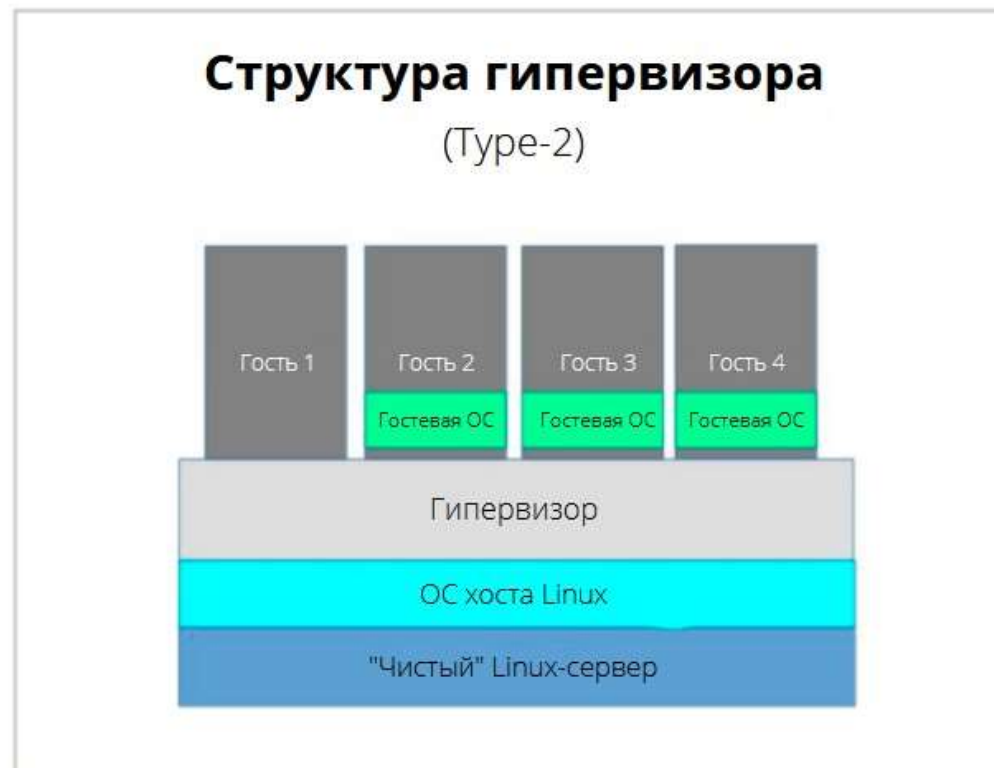
• **Bare-metal гипервизоры (исполняемые на «голом железе») (Type-1)**, загружаются как операционная система машины и – иногда через основную привилегированную виртуальную машину – сохраняют полный контроль над аппаратным обеспечением хоста, запуская каждую гостевую ОС как системный процесс. XenServer и VMWare ESXi – яркие примеры современных гипервизоров Type-1. В последнее время использование термина «гипервизор» распространилось на все технологии виртуализации хостов, хотя раньше оно использовалось только для описания систем Type-1. Первоначально более общим термином, охватывающим все типы систем, был «Мониторы виртуальных машин». То, в какой степени люди используют термин «мониторы виртуальных машин» все это время, наводит меня на мысль, что они подразумевают «гипервизор» во всех его интерпретациях.

Структура гипервизора

(Type-1)



• **Гипервизоры, размещенные на виртуальном узле (Type-2)** сами по себе являются просто процессами, работающими поверх обычного стека операционной системы. Гипервизоры Type-2 (включая VirtualBox и, в некотором роде, KVM) отделяют системные ресурсы хоста для гостевых операционных систем, создавая иллюзию частной аппаратной среды.



ВИРТУАЛИЗАЦИЯ: ПАРАВИРТУАЛИЗАЦИЯ ИЛИ АППАРАТНАЯ ВИРТУАЛИЗАЦИЯ

Виртуальные машины полностью виртуализированы. Иными словами, они думают, что они обычные развертывания операционной системы, которые живут собственной счастливой жизнью на собственном оборудовании. Поскольку им не нужно взаимодействовать со своей средой как-то иначе, чем с автономной ОС, то они могут работать с готовыми немодифицированными программными стеками. Однако раньше за такое сходство приходилось платить, потому что преобразование аппаратных сигналов через уровень эмуляции занимало дополнительное время и циклы.

В случае с паравиртуализацией (PV – Paravirtualization) паравиртуальные гости хотя бы частично осведомлены о своей виртуальной среде, в том числе и том, что они используют аппаратные ресурсы совместно с другими виртуальными машинами. Эта осведомленность означает, что хостам PV не нужно эмулировать хранилище и сетевое оборудование, и делает доступными эффективные драйверы ввода-вывода. На первых порах это позволяло гипервизорам PV достигать более высокой производительности для операций, требующих подключения к аппаратным компонентам.

Тем не менее, для того, чтобы предоставить гостевой доступ к виртуальному кольцу 0 (т.е. кольцу -1), современные аппаратные платформы – и, в частности, архитектура Intel Ivy Bridge – представили новую библиотеку наборов инструкций ЦП, которая позволила аппаратной виртуализации (HVM – Hardware Virtual Machine) обойти узкое место, связанное с ловушкой и эмуляцией, и в полной мере воспользоваться преимуществами аппаратных расширений и немодифицированных операций ядра программного обеспечения.

Также значительно повысить производительность виртуализации может последняя технология Intel – таблицы расширенных страниц (EPT – Extended Page Tables).

В связи с этим, в большинстве случаев можно обнаружить, что HVM обеспечивает более высокую производительность, переносимость и совместимость.

КОНТЕЙНЕРНАЯ ВИРТУАЛИЗАЦИЯ

Как мы уже видели ранее, виртуальная машина гипервизора – это полноценная операционная система, чья связь с аппаратными ресурсами «основополагающей четверки» полностью виртуализирована – она думает, что работает на собственном компьютере.

Гипервизор устанавливает виртуальную машину из того же ISO-образа, который вы загружаете и используете для установки операционной системы непосредственно на пустой физический жесткий диск.

Контейнер в свою очередь фактически представляет собой приложение, запускаемое из скриптообразного шаблона, которое считает себя операционной системой. В контейнерных технологиях, таких как LXC и [Docker](#), контейнеры – это не что иное, как программные и ресурсные (файлы, процессы, пользователи) средства, которые зависят от ядра хоста и представления аппаратных ресурсов «основополагающей четверки» (т.е. ЦП, ОЗУ, сеть и хранилище) для всего, то они делают.

Контейнерная архитектура

